

Лекция 1. Об архитектуре

Рабочая тетрадь

Тетрадь следует за списком экзаменационных вопросов: часть *N* — практическая проработка вопроса *N*. Вопрос о понятии декомпозиции (модульность, связность, сопряжение, сложность) в этой лекции не раскрывается и здесь опущен — он относится к материалу о декомпозиции и ООП. Материал сверх текста лекции помечен [+].

1. Понятие архитектуры, профессия «Архитектор»

Определение: архитектура ПО

Архитектура программной системы — это совокупность важнейших решений об организации системы: из каких *компонентов* она состоит, как компоненты связаны друг с другом и с окружением, какие *ограничения* действуют на компоненты и связи, и какова *мотивация* этих решений. Формального общепринятого определения нет. Стандарт **ISO/IEC/IEEE 42010** «Architecture description» определяет архитектуру как *fundamental concepts or properties of a system in its environment embodied in its elements, relationships, and in the principles of its design and evolution*.

Ключевые тезисы. Архитектура у системы есть *всегда*, даже если её никто не проектировал и не описывал — просто она может оказаться плохой. Архитектура *меняется и эволюционирует* вместе с системой (это не разовая деятельность «до программирования»). Архитектура — во многом *гуманитарная* область: она создаётся для людей, чтобы упростить понимание системы, поэтому «правильной» и «неправильной» архитектуры не бывает — бывают удачные и не удовлетворяющие требованиям.

[+] Актуальная редакция стандарта. Лекция ссылается на ISO/IEC/IEEE 42010:2011. В ноябре 2022 вышла **вторая редакция 42010:2022**, которая отменяет и заменяет редакцию 2011. Главные терминологические изменения: «system of interest» → «entity of interest» (чтобы стандарт можно было применять и к не-программным сущностям — предприятиям, продуктовым линейкам), а «architecture framework» → «architecture description framework» (ADF). Базовая концептуальная модель (вид — точка зрения — заинтересованное лицо) сохранилась.

Зачем нужна архитектура: программа → продукт → системный продукт

Ценность представляет не «программа» (код, работающий у разработчика на хороших входных данных), а **программный продукт**: отестированный, документированный, с налаженным сопровождением, корректно обрабатывающий ошибки на любых входах. Брукс оценивает продукт примерно втрое дороже программы, а **системный программный продукт** (компонент, интегрированный в комплекс) — ещё втрое дороже продукта; итого знаменитое соотношение **1:3:9**.

Пример: соотношение трудоёмкости («квадрат Брукса»)

Программа — одна точка (1). **Программный продукт** = программа + модульные/интеграционные тесты (часто строк тестов *больше*, чем кода) + документация + обработка ошибок + процесс выпуска версий ≈ 3 . **Системный продукт** = продукт + описанные интерфейсы и протоколы + интеграционное тестирование (сложность которого может расти *экспоненциально* от числа интегрируемых приложений) ≈ 9 .

Вывод: как только требования к ПО выходят за пределы «сел и написал», появляются нефункциональные характеристики (надёжность, сопровождаемость, интеграция) — и именно тут архитектура окупается.

Архитектура — это то, что в принципе позволяет создавать ПО размером больше $\sim 100\,000$ строк. Современные системы огромны (см. таблицу), и без грамотной структуры большой проект разваливается под собственной сложностью ещё в разработке.

Unix v1.0 (1971)	10 K LOC	Microsoft Office (2013)	45 M LOC
Quake 3 engine	310 K LOC	MS Visual Studio 2012	50 M LOC
Windows 3.1 (1992)	2.5 M LOC	Windows Vista (2007)	50 M LOC
Linux kernel 2.6.0	5.2 M LOC	Mac OS X 10.4	86 M LOC

Четыре причины тратить усилия на явную архитектуру:

1. **Старт реализации.** С первого варианта архитектуры начинается кодирование — продуманная структура *в разы* сокращает затраты на код.
2. **Инструмент управления проектом.** Без декомпозиции на подзадачи и компоненты нельзя ни оценить проект, ни решить, браться ли за него; крупнозернистая декомпозиция определяет состав команды и даже структуру организации.
3. **Средство переиспользования.** Архитектура выделяет кандидатов на переиспользование, задаёт требования и критерии выбора сторонних компонентов, инкапсулирует свою функциональность.

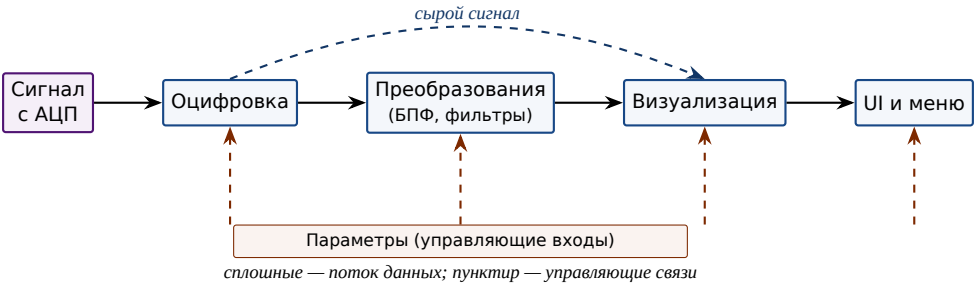
4. **Рассуждение о качествах до кода.** Несколько вариантов архитектуры можно проанализировать и выбрать лучший до написания первой строчки (иногда даже количественно — оценить производительность, память).

Разбор: ПО осциллографа (сквозной пример лекции)

Компания делает цифровые осциллографы и настраивает их под клиента. С ростом функциональности «поправить прошивку» стало дорого — понадобилась архитектура. Авторы перебрали стили:

1. **ООП-модель предметной области.** Сущности выделены аккуратно, но их *слишком много*; модель не помогала разбить систему на модули и не подсказывала, куда отнести функциональность (например, измерение — часть данных или отдельный объект?). Отброшена, но улучшила понимание домена.
2. **Уровневая (слоистая) архитектура:** оборудование → оцифровка → цифровая обработка → визуализация → UI, каждый слой общается только с соседними. Понятна и структурирует систему, но на деле *всем слоям нужно знать про все* (UI настраивает параметры оцифровки; рисовать надо и «сырой» сигнал). Преимущества слоёв теряются.
3. **Каналы и фильтры (pipe & filter):** функциональность как цепочка преобразований данных. Ближе к взгляду инженера-пользователя, легко показать сырой сигнал. Но пользовательские настройки ложились плохо.
4. **Модифицированные каналы и фильтры:** фильтрам добавили *второй тип входа* — управляющие сигналы (настройки) помимо данных. Теперь настройки — это управляющие входы, отображать можно что угодно, а система легко расширяема: добавил тип фильтра и его связи — получил новую функцию. На этой архитектуре и остановились.

Мораль: выводы о сопровождаемости и расширяемости делаются по *структуре*, без единой строчки кода и даже без выбора языка; те же стили (уровни, ООП-модель, каналы-фильтры) не специфичны для осциллографов — это **архитектурные стили**.



Профессия «Архитектор»

В крупных проектах архитектор (или архитектурная группа) разрабатывает и описывает архитектуру, доносит её до заинтересованных лиц, контролирует реализацию и поддерживает архитектуру в актуальном состоянии. Профстандарт «**Архитектор программного обеспечения**» (код 06.003) выделяет трудовые функции: согласование требований с точки зрения архитектуры; выбор и моделирование архитектурного решения; разработка архитектурных разделов проектной и эксплуатационной документации; контроль реализации и испытаний; сопровождение эксплуатации. То есть архитектор — **ключевой технический специалист**: помимо проектирования он ведёт техническое руководство, контролирует качество на всех фазах и часто отвечает за коммуникацию между разработчиками, руководством и заказчиком (важны и технические, и социальные навыки).

	Архитектор	Разработчик
Кругозор	Широкий: домен + много технологий	Глубокий в используемой технологии
Погружение	Быстро вникает в предметную область	Достаточно знать свой инструмент
Зона ответственности	Что делает компонент и как связан с другими	Как компонент реализован внутри
Граница	выше уровня паттернов (GoF)	паттерны и ниже — его дело

Граница ответственности проходит примерно на уровне паттернов проектирования (книга Гаммы и др., «Банды четырёх», GoF): какие паттерны взять для реализации компонента — дело программиста; что компонент должен делать и как связан с остальными — дело архитектора.

DO — хороший архитектор

Пишет код вместе с командой (особенно интерфейсы, протоколы, интеграционные тесты)

Доверяет программистам реализацию

Помнит, что архитектуру можно и нужно менять

Опирается на нефункциональные требования при выборе стиля

DON'T — антипаттерны роли

Сидит в «башне из слоновой кости» и только раздаёт указания — теряет связь с кодом

Микроменеджит вплоть до методов классов — команде неинтересно, люди уходят

Считает архитектуру разовым документом «до программирования»

Оценивает себя только умением «фигачить код» (как на ACM ICPC)

Осторожно: типичные заблуждения

- «Архитектура — это лишнее, нужно только большим неповоротливым организациям». Для мелких приложений (одностраничные сайты, простые мобильные) — возможно; но всё, что больше ~100 K LOC, без архитектуры разваливается.
- «Правильная» и «неправильная» архитектура. Почти любой «ужас» ценой усилий можно заставить работать; объективно сравнить варианты тяжело, часто команда полагается на интуицию и вкус архитектора.
- Архитектор не должен терять руки: *сначала* обычно надо поработать разработчиком — часть умений приходит только с опытом.

[+] Современные практики работы архитектора

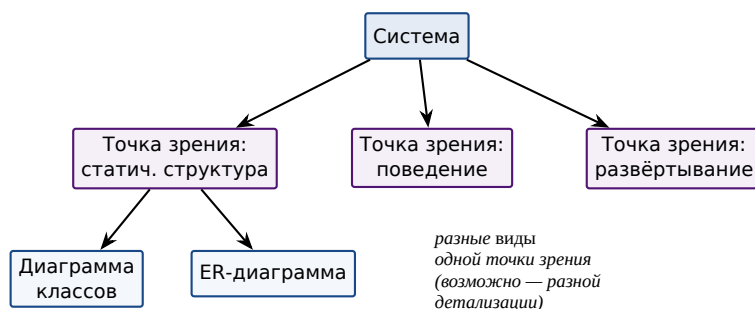
- **ADR (Architecture Decision Records)** — короткие версионизируемые markdown-записи «контекст → решение → последствия» рядом с кодом в репозитории. Прямо отвечают на требование стандарта фиксировать мотивацию решений; инструменты — `adr-tools`, `Log4brains`.
- **C4-модель** (Simon Brown) — лёгкий способ рисовать архитектуру на 4 уровнях детализации: System Context, Container, Component, Code; хорошо ложится на идею «видов разной зернистости».
- **arc42** — открытый шаблон архитектурной документации (12 разделов), де-факто популярен в Европе.
- **Закон Конвея**: структура системы повторяет структуру коммуникаций организации — формальное обоснование тезиса лекции «крупнозернистая декомпозиция определяет структуру организации». Отсюда практика «обратного манёвра Конвея» (*inverse Conway*): сначала строят команды под желаемую архитектуру.

2. Архитектурные виды

Определение: вид и точка зрения

Архитектурное описание — совокупность артефактов, документирующих архитектуру. Поскольку система сложна, её описывают по частям, каждая с какой-то стороны (аналогия: у здания есть поэтажный план, схема электрики, схема водопровода).

- **Архитектурный вид (architectural view)** — описание системы с одной определённой стороны. Вид может состоять из нескольких описаний разной формальности (неформальные, полуформальные, на формальном языке) и разного уровня детализации (модель здания целиком — и детальный план каждого помещения).
- **Точка зрения (architectural viewpoint)** — совокупность видов, рассматривающих систему единообразно. Например, диаграмма классов и диаграмма «сущность-связь» — два разных вида одной точки зрения (статическая структура).



IEEE 1016-2009 «Software Design Descriptions» перечисляет 12 точек зрения. Хорошая новость: все они необязательны — стандарт требует лишь общие сведения (назначение, границы, контекст) и те виды, которые архитектор считает важными.

Контекст	Что система делает; окружение; система как «чёрный ящик»; корень иерархии видов
Композиция	Крупные части и их назначение; импаст-анализ, переиспользование, оценка, планирование
Логическая структура	Классы, интерфейсы и отношения между ними
Зависимости	Связи по данным: что разделяется, порядок выполнения; узкие места, интеграционное тестирование
Информ. структура	Персистентные данные: что хранить, схема БД, доступ
Использование шаблонов	Документирование применённых локальных паттернов
Интерфейсы	Внешние и внутренние интерфейсы (UI — отдельный вид)
Структура системы	Рекурсивное описание внутренней структуры компонентов
Взаимодействия	Почему, когда, как и на каком уровне идёт взаимодействие
Динамика состояний	Состояния и переходы в реактивных системах
Алгоритмы	Детальное поведение сущностей, логика методов
Ресурсы	Использование внешних ресурсов (аппаратных, сторонних сервисов)

Разбор: виды осциллографа

Три варианта архитектуры из примера — это три *вида* (и разные *точки зрения*): ООП-модель описывает **статическую структуру** (сущности и связи); каналы-фильтры — **потоки данных** и их преобразования; уровни — ту же статику, но *крупнозернисто* (нет смысла дробить слои на подслои — это другой *уровень детализации*). Если бы слоистая архитектура подошла, она всё равно состояла бы минимум из двух видов: «слои с высоты» и модель классов внутри каждого слоя.

Когда какой вид брать (decision guidance):

- **Старт проекта** → Контекст (границы, акторы, чёрный ящик), затем Композиция (крупные части).
- **Распределение работ, оценка, переиспользование** → Композиция.
- **Информационная система с БД** → Логическая структура + Информационная структура.
- **Реактивная система (приборы, протоколы)** → Динамика состояний (автоматы).
- **Производительность, интеграция** → Зависимости.
- **Правило: не документировать всё подряд.** Лишние виды стареют и вводят в заблуждение; берут только то, что реально помогает понимать и сопровождать систему.

UML в стандартах упоминается постоянно (сам 42010 описывает свои понятия диаграммами классов UML). **Читать** UML-диаграммы должен уметь каждый, даже вечный junior; **рисовать** — обязательный навык архитектора, даже если в конкретной организации ими не пользуются.

[+] Готовые наборы видов

- **Модель «4+1»** (Philippe Kruchten, 1995) — классика: Logical, Process, Development, Physical виды + Scenarios (use cases), связывающие остальные. Стабильный, до сих пор преподаётся.
- **C4** (Simon Brown) — современная замена тяжёлым UML-наборам для повседневной коммуникации: четыре уровня масштаба вместо десятка типов диаграмм.
- **ISO/IEC/IEEE 42010:2022** оформляет всё это терминологически: *viewpoint*, *view*, *model kind*, объединённые в *architecture description framework* (ADF).

3. Роль архитектуры в жизненном цикле; prescriptive и descriptive

Распространённое заблуждение: архитектор работает только на фазе проектирования — получает требования, рисует диаграммы, пишет design document, «перебрасывает через стену» программистам и уходит. Так иногда и делают («кочующие архитекторы» — одна команда обслуживает много проектов, это эффективно по стоимости). **Современный взгляд:** архитектура — это *непрерывная деятельность в центре* процесса разработки, нужная на всех фазах (как контроль версий или управление проектом), и она ещё и направляет остальные деятельности.

Требования как вход архитектуры

Требования — исходные данные для проектирования. **Наибольшее влияние оказывают нефункциональные требования:** написать *работающую* программу можно миллионом способов; сложно сделать её *ещё и* быстрой, надёжной, сопровождаемой. Поэтому и стиль, и архитектура выбираются под приоритеты НФТ.

Функциональные	Что система должна делать
Нефункциональные	Как она это делает: эффективность (время/ресурсы), масштабируемость, удобство использования, надёжность, безопасность, сопровождаемость/расширяемость
Ограничения	Технические (например, разработка под iPhone ограничивает инструментарий) и бизнес (например, успеть к пресс-конференции через месяц)

Разбор: надёжность — это не одно число

Наработка на отказ (MTBF) и время восстановления (MTTR) — разные вещи с разной важностью в разных ситуациях.

- **Межпланетный зонд:** MTTR неинтересно (ошибся антенной — потерян навсегда), а MTBF критично (если за 20 лет полёта ПО в среднем падает раз в два года — запускать бессмысленно).
- **Сотовый телефон:** наоборот — один пропущенный из-за сбоя вызов не страшен, можно перезвонить (MTTR мал и не важен).
- **Failsafe** (корректное поведение при сбое) — третье, отдельное свойство: ядерный реактор или банковская система при *любой* критической ошибке должны завершаться *безопасно*.

Prescriptive и descriptive архитектура; дрейф и эрозия

Определение: предписывающая и описывающая архитектура

Предписывающая (prescriptive) архитектура — спроектированная, «как задумано» (as-designed). **Описывающая (descriptive) архитектура** — фактическая, «как построено» (as-built), та, что реально воплощена в коде. Между ними со временем расходятся два процесса:

- **Архитектурный дрейф (architectural drift)** — появление в фактической архитектуре важных решений, которых *не было* в предписывающей. Часто они адекватны, но не задокументированы и локальны — никто не думал об их глобальных последствиях.
- **Архитектурная эрозия (architectural erosion)** — *нарушение* в фактической архитектуре принципов, заложенных в предписывающей. Возникает при багфиксах, рефакторингах, добавлении функций; вносит хаоса гораздо больше, чем дрейф, и превращает систему в «большой ком грязи» (big ball of mud).

	Дрейф (drift)	Эрозия (erosion)
Суть	Добавление решений вне плана	Нарушение заложенных принципов
Вред	Меньше: чаще адекватно, но недокументировано	Больше: разрушает структуру системы
Аналогия	Незаписанные новые правила	Рост энтропии
Лечение	Документировать, осмыслить глобально	Контроль архитектора, рефакторинги (замедляют, но не останавливают)

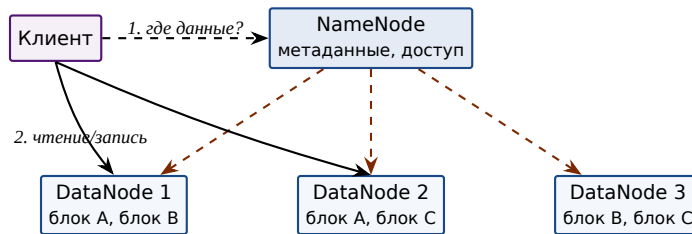
Осторожно: эрозия как энтропия

Архитектурные ограничения трудно проверять автоматически, поэтому эрозия на практике почти неизбежна. Каждый рефакторинг, каждый багфикс, каждая новая строка имеют *цену* в виде потенциальной эрозии. Бороться «в лоб» бесполезно, как с энтропией; хорошая чётко описанная архитектура и постоянное внимание лишь *замедляют* сползание в хаос. В пределе изменения становятся слишком дороги — тогда нужно **восстановление архитектуры** по коду, её рефакторинг и глобальный рефакторинг кода (в запущенных случаях — реинжиниринг); пока идёт эта работа, развивать систему нельзя.

Разбор: Apache Hadoop — prescriptive против descriptive

Предписывающая HDFS (as-designed): кластером управляет NameNode (хранит метаданные, управляет доступом). Файлы хранятся не целиком, а *блоками*; блоки распределены по DataNode и *реплицированы* (выход одного узла из строя не теряет данные). Клиент спрашивает у NameNode, *где* его данные, и работает с DataNode напрямую.

Описывающая (as-built): восстановление архитектуры по исходникам (Hadoop ≈ 200 К значимых строк, ≈ 1700 классов, Java) заняло у двух аспирантов ≈ 120 часов + 8 часов инженера из Yahoo. Реальность *гораздо* сложнее диаграммы: выделено **68 компонентов** (21 — утилиты, 47 — бизнес-логика). Ключевой факт: **58 из 68** компонентов *не соответствовали* структуре пакетов исходников (18 имели реализацию в разных пакетах, 40 содержали в «своих» пакетах куски чужих компонентов). *Вывод:* по структуре папок архитектуру *не* восстановить.



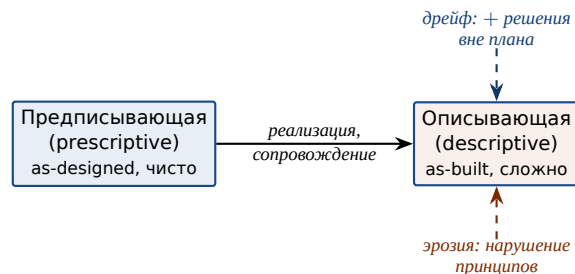
пунктир от NameNode — управление/heartbeat; блоки реплицированы (А,В,С на двух узлах)

[+] Где лекция устарела: проверяемые ограничения и судьба Hadoop

1. Ограничения можно проверять автоматически — fitness functions. Тезис лекции «архитектурные ограничения редко удаётся формализовать для инструментов» сегодня во многом снят. Книга *Building Evolutionary Architectures* (Ford, Parsons, Kua; 2-е изд. 2022) ввела **архитектурные fitness-функции** — «юнит-тесты для архитектуры», которые в CI/CD ловят дрейф и эрозию: запрет циклических зависимостей, контроль направления связей между слоями, правила именования и границы пакетов. Инструменты: **ArchUnit** (Java), **NetArchTest** (.NET), **dependency-cruiser** (JS/TS), **OPA** (policy-as-code). Пример правила на ArchUnit:

```
// Fail the build if any class outside the Service layer
// is accessed by Controller bypassing the allowed layering.
layeredArchitecture().consideringAllDependencies()
    .layer("Controller").definedBy("..controller..")
    .layer("Service").definedBy("..service..")
    .layer("Persistence").definedBy("..persistence..")
    .whereLayer("Persistence").mayOnlyBeAccessedByLayers("Service")
    .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller");
// runs as a normal JUnit test in the CI pipeline
```

2. Hadoop сегодня. Сам пример датирован: *MapReduce* стал legacy — для обработки индустрия перешла на **Apache Spark** (вычисления в памяти, на порядок-два быстрее на итеративных задачах). *HDFS* ещё используется для on-prem «озёр данных», но современный стандарт — *облачное объектное хранилище* (S3, GCS) + Spark, что разделяет хранение и вычисление. Сам Hadoop живёт и поддерживается (релиз 3.4.2, август 2025), но как *хранилище и управление ресурсами* (HDFS, YARN), а не как движок MapReduce.



Итоговый чек-лист

- ☐ Могу дать рабочее определение архитектуры (компоненты, связи, ограничения, мотивация) и процитировать смысл ISO/IEC/IEEE 42010 **[+]** (актуальная редакция — 2022).
- ☐ Объясняю цепочку программа → продукт → системный продукт (1 : 3 : 9) и почему архитектура окупается на больших системах (>100 K LOC).
- ☐ Называю четыре причины явно описывать архитектуру.
- ☐ Помню сквозной пример осциллографа: ООП-модель → слои → каналы-фильтры → фильтры с управляющими входами, и какие качества решал каждый шаг.
- ☐ Описываю профессию архитектора: функции профстандарта 06.003, «ключевой техн. специалист», широкий кругозор, должен писать код, без «башни» и без микроменеджмента; граница с программистом — на уровне паттернов GoF.
- ☐ Различаю **вид (view)** и **точку зрения (viewpoint)**; помню, что виды бывают разной формальности и детализации; знаю, что в IEEE 1016 обязательны лишь основные сведения.
- ☐ Привожу примеры точек зрения и понимаю принцип «не документировать лишнее».
- ☐ Объясняю, что архитектура — непрерывная деятельность в центре цикла, а не фаза «до кода».
- ☐ Помню, что архитектуру сильнее всего формируют *нефункциональные* требования; различаю MTBF, MTTR и failsafe.

- ☐ Чётко развожу **prescriptive/descriptive** и **дрейф/эрозию**; понимаю восстановление архитектуры и реинжиниринг.
- ☐ Разбираю пример Hadoop (NameNode/DataNode; 58 из 68 компонентов $\neq$ структуре пакетов).
- ☐ **【+】** Знаю современный ответ на «ограничения нельзя проверять» — fitness-функции (ArchUnit, NetArchTest) — и актуальный статус Hadoop (Spark, облачные хранилища).

Лекция 2. Декомпозиция и ООП

Рабочая тетрадь

Тетрадь следует за списком экзаменационных вопросов: часть *N* — практическая проработка вопроса *N*. Вопросы о моделировании (виды моделей, метафора визуализации) и о языке UML с диаграммами классов в этой лекции не раскрываются и здесь опущены — они относятся к лекции о моделировании и UML. Материал сверх текста лекции помечен [+].

1. Понятие декомпозиции. Модульность, связность, сопряжение, сложность

Определение: сложность и её виды

Сложность — неотъемлемая часть ПО: убрать её нельзя, не потеряв ценность системы (в отличие от физики, где по сложному явлению строят простую модель, у программистов сложны даже модели). Причина: почти каждая строка кода уникальна и несёт содержательную информацию.

- **Существенная (essential)** — присуща самой задаче/предметной области; убрать нельзя (код решения ДУ не проще метода его решения).
- **Случайная (accidental)** — привнесена способом решения; от неё можно и нужно избавляться (хороший API, разумное деление по классам, удобные вызовы).

Архитектор не борется со сложностью, а управляет ею; главный приём — **сокрытие**: пользователю (конечному или коллеге-программисту) дают максимально простой интерфейс.

	Существенная (essential)	Случайная (accidental)
Источник	сама задача/домен	способ решения, инструменты
Можно убрать?	нет (объективна)	да — это и есть работа над качеством
Пример	математика решаемой задачи	дублирование, кривой API, лишние зависимости

Общие свойства сложных систем (тех, о которых мы вообще способны рассуждать): иерархичность (система из подсистем); небольшое число видов компонентов, экземпляры которых сложно связаны; система — результат эволюции простой системы. Отсюда два подхода: **декомпозиция** (разбиение иерархии на отдельно-осмысляемые компоненты) и **абстрагирование** (выделение общих свойств, рассуждение о типах). Важное следствие: даже маленькую программу проектируют так, будто она вырастет в монстра на миллионы строк.

Определение: подходы к декомпозиции

Восходящее (bottom-up): сначала мелкие компоненты-«кирпичики», из них — крупные, затем система. Хорошо для исследовательских проектов и библиотек (язык FORTH: программа из слов, строящихся из слов; переключается с TDD). **Нисходящее (top-down)**: задача целиком → подзадачи → общая логика с заглушками → детализация каждой подзадачи. «Традиционный» подход; лучше направляет разработку (меньше лишнего), но до низа иерархии систему трудно тестировать.

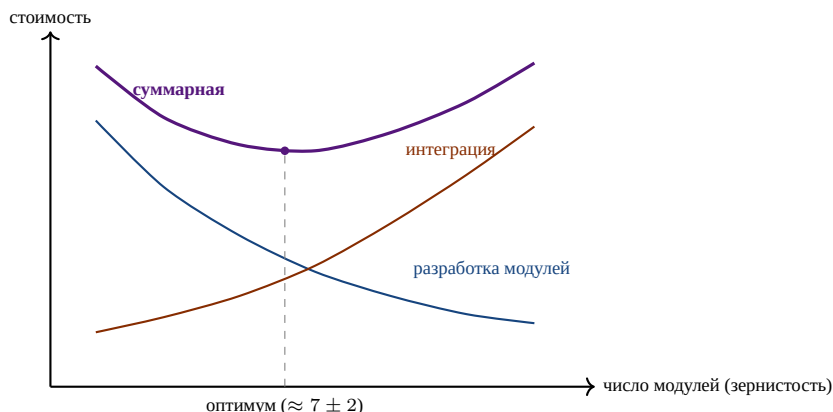
	Восходящее	Нисходящее
Когда брать	неясен итог; домен из комбинируемых мелких задач; библиотеки	есть чёткое видение результата (т.е. почти всегда)
Плюс	рано тестируется и отлаживается	предсказуемость, меньше ненужного кода
Минус	можно сделать лишнее	пока нет низа — нечего внятно тестировать

Определение: модуль

Модуль — структурная единица кода, соответствующая подзадаче. В ООП — класс/компонент (иногда целая подсистема, веб-сервис); в ФП — функция или их группировка (module в F#); в структурных — пара .h и .c в C, модуль в Pascal, пакет в Ada. Модуль характеризуется **интерфейсом** и **реализацией**; интерфейс фиксируют до работы над подзадачами (по нему общаются реализации и пишутся заглушки).

Правила проектирования модулей: чёткая декомпозиция (модуль знает своё место и задачу, понятно, как им пользоваться); минимизация интерфейса («меньше знаешь — крепче спишь»); один модуль — одна функциональность; отсутствие побочных эффектов (желательно); независимость от деталей реализации других модулей; сокрытие данных (наружу — только абстрактные типы через функции модуля). Модульность позволяет строить сколь угодно большие системы и распараллеливать работу между командами — пока все соблюдают интерфейсы (синтаксис ловит компилятор, а «сначала Connect(), потом GetData()» — нет).

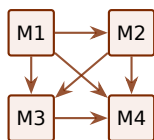
Размер модуля — баланс. Слишком крупные модули — велика сложность каждого; слишком мелкие — велики затраты на интеграцию (модулей и связей слишком много). В обоих вырожденных случаях (один огромный модуль / куча по одной функции) выгоды от модульности нет. Отправная точка — правило «**7±2**» (число сущностей, удерживаемых в голове).



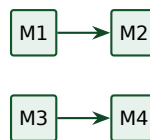
Определение: сопряжение и связность

Сопряжение (coupling) — мера взаимосвязанности *разных* модулей (как часто и как много один знает о других). **Связность (cohesion)** — мера взаимосвязанности функций *внутри* модуля (насколько похожие задачи они решают). Цель проектирования: **слабое сопряжение** (можно независимо менять компоненты) и **сильная связность** (иначе модуль надо разбить — чтобы его было проще понять и использовать). Метрики считаются автоматически или «на глаз».

	Сопряжение (coupling)	Связность (cohesion)
Про что	связи <i>между</i> модулями	связи <i>внутри</i> модуля
Цель	слабое (low)	сильная (high)
Если плохо	меняешь один — ломается всё	в модуле «свалка» разнородного — не понять, не найти



сильное сопряжение, слабая связность



слабое сопряжение, сильная связность

Осторожно: что на самом деле значит «понимание»

Все эти практики — про *понимание* системы. Не то, которое получает внимательный читатель, разбирая код по строчке, а то, которое нужно, *когда завтра релиз*, в сотне тысяч строк есть критический баг, а писала их соседняя команда из Чехии, ушедшая в отпуск всем составом, и все комментарии на чешском. Поэтому «один модуль — одна функциональность» и «утилиты для строк» + «ещё утилиты для строк» — это провал (никогда не вспомнишь, где нужный метод).

[+] Современные модульные системы и метрики

- **Системы модулей** эволюционировали: **JPMS** (Java Platform Module System, Java 9+ — `module-info.java` с явными `requires`, `exports`), ES-модули в JS, пакеты в Go, модули и крейты в Rust — все кодируют «минимизацию интерфейса» и сокрытие на уровне языка.
- **Сопряжение/связность считаются инструментами:** **SonarQube**, **NDepend**, **Structure101** меряют связность (метрика **LCOM**) и циклы зависимостей; **ArchUnit** (из лекции 1) как *fitness-функция* запрещает циклы и контролирует границы пакетов прямо в CI/CD.
- **Essential vs accidental** — по-прежнему ядро инженерии (Brooks, *No Silver Bullet*): большинство наших инструментов быют только по *случайной* сложности.

2. Понятия класса и объекта; абстракция, инкапсуляция, наследование

Определение: объект и класс

Объект (наиболее близко к курсу, «Thinking in Java») — изолированная сущность с *поведением* и, возможно, *состоянием*. Три свойства: **состояние, поведение, идентичность**. С состоянием связан **инвариант** — логические условия, истинные всё время жизни объекта (например, `length` равно числу элементов списка). Объект сам поддерживает инвариант и не даёт нарушить его извне (поэтому `public`-поля запрещены).

Поведение $\neq$ **вызов функции**: объект сам решает, как обработать запрос — правильнее считать, что объекты *шлют сообщения*. Методы полиморфны: реально вызванный код неизвестен вызывающему и определяется типом времени выполнения (а при *двойной диспетчеризации* — двух объектов; см. «Посетитель»). **Класс** — тип объекта (сущность времени компиляции); объект — сущность времени выполнения. Есть языки без явного понятия «класс» — прототипные.

[+] **Уточнение к примеру лекции.** Прототипные («бесклассовые») языки — это JavaScript (до ES6; класс там и сейчас — синтаксический сахар над прототипами), Self, Lua. А вот **Smalltalk**, упомянутый в лекции, наоборот — один из самых *класс-ориентированных* языков (в нём классы — тоже объекты); это оговорка.

Определение: абстракция

Абстракция выделяет существенные характеристики объекта с *точки зрения наблюдателя*, отличающие его от других. Один объект может иметь *несколько* абстракций сразу. Хорошая абстракция в разы упрощает работу и может иметь *взаимозаменяемые* реализации. Образец — вся математика: *моноид* — абстракция; целые числа со сложением, строки с конкатенацией, функции `int`→`int` с композицией — реализации; теорема «для любого моноида верно ...» — внешний код, пользующийся абстракцией.

Определение: инкапсуляция

Инкапсуляция разделяет *интерфейс* (контракты) абстракции и её *реализацию*. Даёт «меньше знаешь — крепче спишь» (пользователь не знает реализацию) и, главное, **защищает инварианты** абстракции от порчи извне.

Определение: наследование (is-a)

Наследование — это не «потомок получает `public/protected` члены предка» (технически так, но по сути нет). Наследование — отношение **«является» (is-a)**, способ реализации *сабтайпинга*: «объект-потомок является объектом-предком и может использоваться всюду, где используется предок». **Потомок наследует все инварианты предка и не вправе их нарушать** (даже неявные).

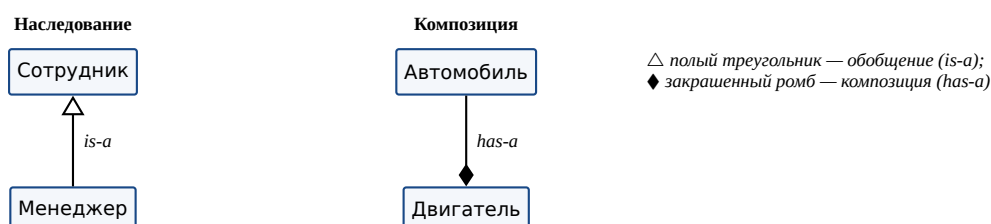
Разбор: квадрат и прямоугольник (классическая загадка)

«Квадрат — это прямоугольник с равными сторонами», поэтому кажется, что `Square` наследует `Rectangle`. Но код «удвоить площадь, удвоив длину» корректен для прямоугольника и *ломается* для квадрата (его инвариант: длина = ширина). Значит, квадрат *нельзя* использовать всюду вместо прямоугольника — наследовать нельзя. Наоборот — тоже нельзя (код, меняющий сторону квадрата, сломается на прямоугольнике). **Вывод:** прямоугольник и квадрат вообще не должны быть связаны наследованием. Мораль: наследуются все инварианты, даже неявные.

Наследование применяют только для двух целей: как средство классификации/абстрагирования (по существующим в домене системам классификации) и для обеспечения полиморфизма. Для обоих достаточно наследования *интерфейсов*; наследование конкретных классов — ныне редкость, абстрактные классы — из прагматики («поля не писать»). **Наследование ради переиспользования кода** (как мыслили во времена C++ с `ergo private`-наследованием) — устаревший приём; современная рекомендация — **композиция**.

Определение: композиция (has-a)

Композиция — отношение **«имеет» (has-a)**, обычно через `private`-поле: общий код выносят в класс и включают объектом. Позволяет создавать/менять/удалять связи *во время выполнения* и делегировать запросы — на этом стоят «Декоратор», «Прокси», «Адаптер». Технически наследование *и есть* композиция (потомок включает объект предка); они взаимозаменяемы, но композиция **предпочтительнее** — она гибче (наследование зафиксировано на этапе компиляции).



	Наследование (is-a)	Композиция (has-a)
Смысл	потомок <i>является</i> предком	объект <i>имеет</i> другой объект
Когда меняется	фиксировано в компиляции	создаётся/меняется в рантайме
Брать для	классификации, полиморфизма (хватает интерфейсов)	переиспользования кода, гибких связей
Риск	нарушить инвариант предка (LSP)	— (предпочтительнее по умолчанию)

[+] Современный ООП-инструментарий

- **Неизменяемые «записи»** (record в Java 16+, record в C#, data class в Kotlin) снимают целый класс LSP-проблем: у immutable-прямоугольника просто *нет* мутирующего «удвоить длину», ломавшего квадрат.
- **Запечатанные интерфейсы + сопоставление с образцом** (sealed в Java 17 + pattern matching в Java 21; актуальный LTS — **Java 25**) дают *алгебраические типы данных*; switch по запечатанной иерархии — современная альтернатива обходу через «Посетитель».
- «Композиция вместо наследования» усиливается интерфейсами с реализациями по умолчанию (default methods, Java 8+), трейтами (Scala, Rust) и миксинами.

3. Принципы выделения объектов

Определение: построение объектной модели домена

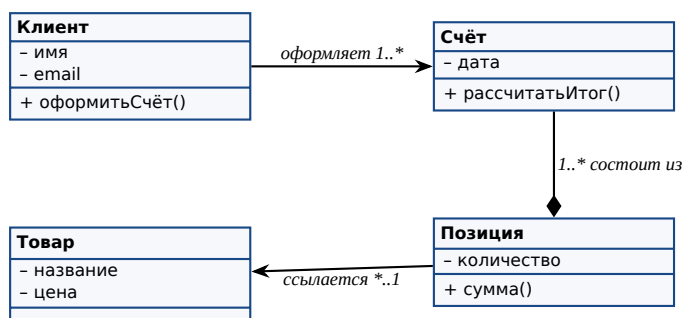
По техзаданию строят объектную модель домена за четыре шага:

1. **Объекты и атрибуты** — из общения с экспертами и здравого смысла; часто встречающиеся **существительные** → классы.
2. **Действия над объектами** (назначение ответственности) — **глаголы** → методы.
3. **Связи** — вопросами «а про это кто знает?», «а где используется?».
4. **Интерфейс каждого объекта** — фиксация и формализация сделанного.

Результат — первый грубый набросок (диаграмма классов).

Разбор: набросок доменной модели «существительные → классы, глаголы → методы»

Биллинг: *существительные* «клиент», «счёт», «позиция», «товар» становятся классами; *глаголы* «оформить счёт», «рассчитать итог» — методами. Связи: клиент *оформляет* счета (1–*); счёт *состоит из* позиций (композиция); позиция *ссылается на* товар.



Классы появляются не только из домена, но и при реализации. **Три источника абстракций:**

Изоляция сложности

сложные алгоритмы, нетривиальные структуры, целые подсистемы — за простыми фасадами; чтобы спрятать сложность и свободно менять реализацию (но интерфейс фасада сам норовит распухнуть!)

Изоляция изменений

прятать то, что вероятно поменяется, чтобы выкинуть и переписать. Источники: бизнес-правила (меняются часто!), зависимости от железа/ОС (редко, но критично — вспомните Y2K), форматы ввода-вывода, нестандартные возможности языка (переживать смену компилятора), сложные решения проектирования, третьесторонние компоненты

Изоляция служебной функциональности

фабрики, репозитории, диспетчеры, медиаторы, сервисы; бизнес-логика должна содержать только бизнес-логику, детали — отдельно

Осторожно: не переусердствовать с «гибкостью»

Хорошая архитектура позволяет себя менять и без специальной поддержки изменчивости — не надо городить точки расширения «на всякий случай». Фасады, скрывающие сложность, имеют тенденцию *сами* становиться сложными, убивая выгоду инкапсуляции. И не стоит думать «кому мой продукт будет нужен через пять лет» — ваше любимое железо снимут с выпуска, а неаккуратная зависимость от него потребует переписать систему с нуля.

[+] От «изоляции изменений» к современным шаблонам и архитектурам

- Перечисленные служебные сущности — это будущие шаблоны курса: **Фабрика, Репозиторий, Фасад, Адаптер, Медиатор**.
- **Гексагональная архитектура** (ports & adapters, Alistair Cockburn) и **луковичная/ чистая** архитектура — современное оформление принципа «изолируй домен от ввода-вывода и сторонних компонентов»: домен в центре, всё изменчивое (БД, UI, внешние API) — за портами-адаптерами.
- **DDD** (Eric Evans) формализует шаг «существительные домена → классы» через единый язык (ubiquitous language), сущности, объекты-значения и агрегаты.

4. Принципы SOLID. Закон Деметры

Принцип	Суть в одной строке
S Single Responsibility	один класс — одна ответственность (одна причина меняться)
O Open/Closed	открыт для расширения, закрыт для изменения
L Liskov Substitution	потомок заменяет предка, не нарушая его контракт и инварианты
I Interface Segregation	клиент не зависит от методов, которые не использует
D Dependency Inversion	верх и низ зависят от абстракций, не друг от друга

S — принцип единственности ответственности

Каждый класс делает *что-то одно*; о его роли должно быть можно сказать *одним предложением* («утилиты для строк», «вычисллка процентов по вкладу»). Если кратко сформулировать не выходит — абстракция плохая, разбейте её. Ответственность должна быть *полностью* инкапсулирована (двух классов «утилиты для строк» быть не должно). Касается и функций, и подсистем.

O — принцип открытости/закрытости

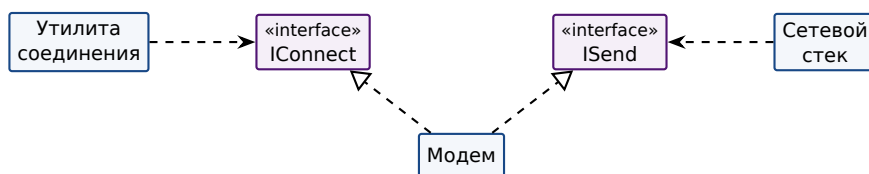
Стабилизировавшийся и уже используемый интерфейс менять *нельзя*; новую функциональность добавляют через наследование или заранее подготовленные *точки расширения*. Нарушение раздувает интерфейс, тянет за собой нарушение SRP и порождает несопровождаемого монстра (см. антипаттерны «God object», «Swiss army knife» далее в курсе).

L — принцип подстановки Барбары Лисков

То самое определение наследования: функции, работающие с базовым типом, должны работать с подтипами, *не зная* об этом. Потомок обязан реализовать интерфейс предка *синтаксически и семантически* и выполнять все его инварианты (явные и неявные). **Пример проблемы** — проверяемые исключения в Java: потомок не вправе бросать незадекларированное в предке (иначе ломает try/catch вызывающего). Предсказать все исключения потомков заранее нельзя — поэтому проверяемые исключения почти не используют, и они *не попали ни в один другой язык*.

I — принцип разделения интерфейсов

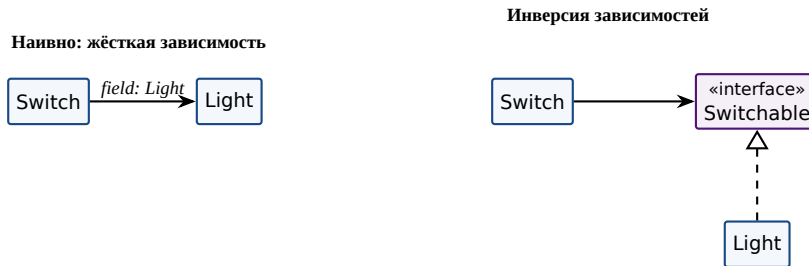
Если у абстракции есть группы методов для *разных* клиентов — сделайте отдельный интерфейс на группу. Пример: модем умеет Connect() и Send(). Единый IModem заставит пересобирать утилиту соединения при правках Send() (который ей не нужен), а сетевой стек — при правках Connect(). **Не переусердствовать:** интерфейсов не «по клиенту на каждый», а по реально кластеризующимся *по смыслу* методам.



клиент зависит только от нужного интерфейса (пунктир — зависимость, штрих-треугольник — реализация)

D — принцип инверсии зависимостей

Модули верхних уровней не должны зависеть от нижних; оба зависят от абстракций. **Пример:** наивно Switch держит поле Light и зовёт on()/off() — выключатель *знает* про лампочку, требует перекомпиляции при её смене, не тестируется через mock. Инверсия: интерфейс Switchable, лампочка его реализует, Switch держит поле Switchable и про лампочку не знает; зависимость приходит снаружи (в конструктор/сеттер) — это **Dependency Injection**.



IoC-контейнеры (Inversion of Control) управляют зависимостями и инициализацией объектов; без них не обходится ни одна современная веб-библиотека. Но и без библиотек DI — хорошая практика, резко снижающая связанность.

Закон Деметры — «не разговаривай с незнакомцами»

Объект *A* не должен напрямую доставать объект *C*, если у *A* есть доступ к *B*, а у *B* — к *C*.

```
book.pages.last.text // bad: exposes internal structure ("train wreck")
book.pages().last().text() // better, but still leaks the page collection
book.lastPageText() // ideal: ask the book directly
```

Цепочки «по точке» раскрывают внутреннюю структуру (есть коллекция страниц с last(), в ней страницы с text()) и привязывают к ней; в «Чистом коде» Р. Мартина это сравнивают с *крушением поезда* — поменяли представление в середине, и все цепочки надо переделывать.

Осторожно: Деметра ≠ запрет точек; не путать с Fluent API

Главное — не переусердствовать: book.firstPageText(), book.secondPageText(), ... хуже «паровозика». Закон лишь запрещает показывать структуру, не входящую в абстракцию; если доступ к каждой странице ожидаем — дайте pages(). И не путать нарушение с **Fluent API** (Java Stream, LINQ, шаблон «Строитель»): там цепочка возвращает *тот же* объект, не раскрывая структуры (LINQ вообще проектировался как монады на C#).

[+] Откуда SOLID и как его проверяют сегодня

- Аббревиатуру **SOLID** ввёл Michael Feathers по принципам Robert C. Martin («дядюшка Боб») в начале 2000-х. Современная формулировка SRP: «у класса одна причина меняться — один *актор*, перед которым он отвечает».
- IoC-контейнеры сегодня:** **Spring** (Java) — рефлексия в рантайме, богатая экосистема, AOP; **встроенный DI в ASP.NET Core** (IServiceProvider, дружелюбен к Native AOT, keyed-сервисы); **Dagger, Hilt** (Android, Google) и **Micronaut, Quarkus** — *compile-time* DI (генерация кода, без рефлексии, быстрый старт). Тренд — DI *во время компиляции* ради скорости и нативной компиляции.
- DIP и отсутствие циклов можно *проверять автоматически* — **ArchUnit/NetArchTest** как fitness-функции (связь с лекцией 1).

Итоговый чек-лист

- ☐ Различаю **существенную** и **случайную** сложность; помню, что архитектор сложностью *управляет* (сокрытие), а не борется.
- ☐ Знаю свойства сложных систем (иерархичность; мало видов компонентов; эволюция из простого) и два подхода — декомпозиция и абстрагирование.
- ☐ Сравниваю восходящее и нисходящее проектирование (когда какое); понимаю роль модуля (интерфейс + реализация) и правила его проектирования.
- ☐ Объясняю баланс размера модулей (U-кривая, «7±2»); определяю **сопряжение** (слабое!) и **связность** (сильную!).
- ☐ Даю три свойства объекта (состояние, поведение, идентичность); понимаю инвариант, «сообщения» вместо вызовов, класс (компиляция) vs объект (выполнение).
- ☐ Объясняю **абстракцию** (пример моноида), **инкапсуляцию** (интерфейс vs реализация, защита инвариантов).
- ☐ Понимаю **наследование как is-a** (LSP) и разбираю квадрат/прямоугольник; знаю, что **композиция (has-a)** предпочтительнее.

- ☐ Строю доменную модель: существительные → классы, глаголы → методы, связи, интерфейсы.
- ☐ Перечисляю три источника абстракций (изоляция сложности / изменений / служебной функциональности) и их подпункты.
- ☐ Формулирую **все пять SOLID** с примерами (модем — ISP, выключатель/лампочка — DIP) и **закон Деметры** (не путаю с Fluent API).
- ☐ **[+]** Знаю современный инструментарий: records/sealed/pattern matching (Java 25), JPMS, IoC-контейнеры (Spring, ASP.NET Core, Dagger/Hilt, Micronaut), гексагональная архитектура, ArchUnit.

Лекция 3. Моделирование и UML

Рабочая тетрадь

Тетрадь следует за списком экзаменационных вопросов: часть N — практическая проработка вопроса N . Диаграммы развёртывания и диаграммы случаев использования в этой лекции не рассматриваются подробно и здесь опущены. Материал сверх текста лекции помечен [+].

1. Моделирование. Визуальные модели, виды моделей, метафора визуализации

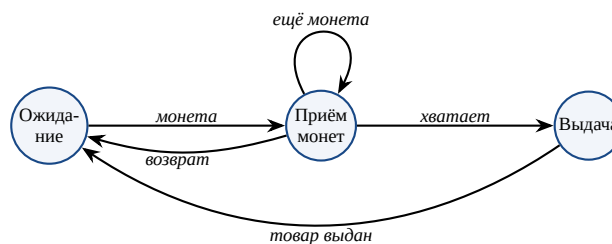
Определение: модель

Модель — упрощённое подобие объекта или явления для изучения некоторых его свойств, абстрагируясь от полной сложности. Свойства любой модели:

- *содержит меньше информации*, чем реальность (иначе изучали бы сам объект);
- *целенаправленна* — создаётся под конкретную цель, выделяет важное (рисовать диаграммы «вообще» — неправильно);
- *субъективна* — архитектор сам решает, что показать;
- *ограничена* — не может вместить бесконечную сложность; полнота не только бессмысленна, но и невозможна.

«All models are wrong, some are useful» (G. Box): ждать от модели надо не точности и полноты, а *полезности*.

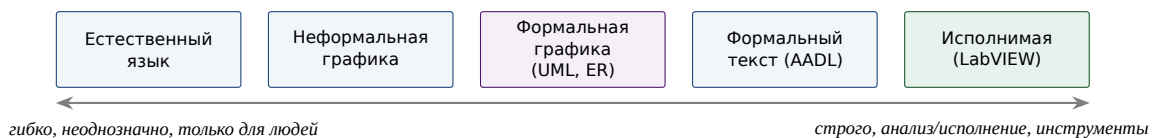
Польза моделей при проектировании ПО — прежде всего **управление сложностью**: взгляд на систему с разных точек зрения, выделяя каждый раз своё существенное; часто моделируют и *окружение* (бизнес-процесс, внешние системы, пользователей). Модель позволяет понять, проанализировать и даже *протестировать* систему до первой строки кода (пример из лекции — конечный автомат торгового автомата: можно проверить достижимость состояний и возврат в исходное).



Модель проста и неформальна, но анализируема: из любого состояния есть путь назад в «Ожидание» (важное свойство — автомат всегда возвращается в исходное состояние).

Определение: виды моделей (спектр формальности)

По нарастанию формальности (Medvidovic):



Вид	Плюсы	Минусы
Естественный язык	не нужен синтаксис; выразителен; гибок	неформален, многословен, бесполезен для автообработки
Неформальная графика (PowerPoint, Visio)	проста, гибка, красива; чаще UML встречается!	неоднозначна; <i>ловушка</i> — неспециалисты считают её строгой
Формальная графика (UML, ER, IDEF0)	стандартна → инструменты; строгий синтаксис	часто <i>нет</i> строгой семантики; трудно держать консистентность/расширять
Формальный текст (AADL)	формальный анализ свойств до кода	многословна (не на доске); тяжело учить
Исполнимая (LabVIEW, язык G)	это уже программа в графике	строго говоря, уже не модель

Определение: три ключевых понятия о визуальных моделях

- **Метафора визуализации** — договорённость, как изображать сущности языка (в UML класс — прямоугольник из трёх частей; у Буча класс — облако пунктиром, объект — облако сплошной линией). ПО незримо, поэтому метафора — лишь соглашение между программистами.
- **Точка зрения моделирования** — *какой* аспект и *для кого* моделируется (для программистов / менеджеров / заказчиков). Перед рисованием — понять, для кого и зачем; не рисовать «просто потому что можем».
- **Семантический разрыв** — принципиальная неполнота: разрыв между информацией в модели и тем, что нужно компьютеру для исполнения. Не приговор (исполнимые модели есть), но чаще модели — одноразовые, нужны лишь чтобы передать идею.

Зачем моделировать (на что надеяться): направляет проектирование (хороший язык *сам* подсказывает слабые места); коммуникация между разработчиками («one picture worth a thousand words»); наглядность для заказчика; документирование и фиксация решений; иногда — исходник для генерации кода (но мешает семантический разрыв: генерируются лишь заглушки, которые потом всё равно дописывать вручную). **Опасность** — «архитектурный паралич»: команда так детализирует архитектуру, что деньги кончаются до первой строки. Решать заранее: *что* моделировать (важное = затрагивающее многих, а не то, чем гордимся), на *каком уровне детализации*, насколько *формально*; стоимость модели не должна превышать выгоду.

[+] Виды моделей сегодня: diagrams-as-code и LLM

- Появился новый «вид» между неформальной и формальной графикой — **диаграммы как код** (diagrams-as-code): текстовый DSL рендерится в картинку. **Mermaid** (рендерится *нативно* в Markdown на GitHub/GitLab, ~85k звёзд) — лидер; **PlantUML** (полный UML, с 2009); **Structurizr** (DSL под C4-модель Simon Brown: один *model* → много видов); **D2** (2022, лучший авто-layout). Профит как у infrastructure-as-code: версионирование, ревью в pull request, нет «дрейфа» документации, рендер в CI.
- Тезис лекции «NLP пока далеко от инструментов» **устарел**: **LLM** генерируют Mermaid/PlantUML по коду или тексту и поддерживают их в актуальном состоянии в CI; диаграммы теперь нередко нужны, чтобы человек *проверил* то, что сгенерировал ИИ.
- **C4-модель** — современная замена тяжёлому UML для архитектурной коммуникации (4 уровня масштаба); хорошо ложится на diagrams-as-code (Structurizr/Mermaid).

2. Язык UML. Диаграммы классов

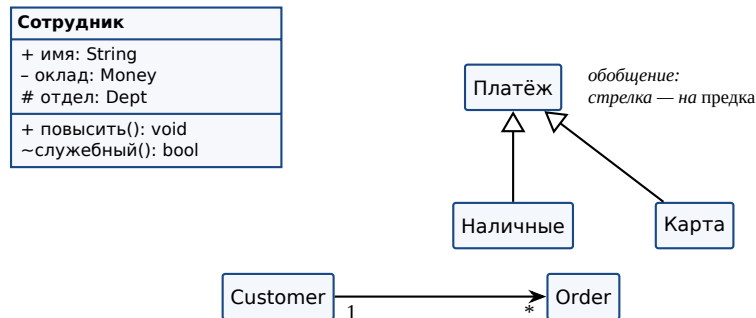
Определение: UML

UML (Unified Modeling Language) — стандарт де-факто архитектурного моделирования; не один язык, а ~14 нотаций, объединённых единым стандартом и *метамоделью*. Разрабатывается консорциумом **OMG** (он же — CORBA, BPMN). Версии 1.4.2 и 2.4.1 приняты как стандарты ISO (**ISO/IEC 19501**). Расширяется двумя способами: **профили** (лёгкое уточнение нотации под домен, широко поддерживаны) и **метамоделирование** (правка самого стандарта; так из UML вырос SysML).

Происхождение и вехи. Компания Rational наняла «трёх амиго»: **Буч** (метод Буча, соавтор Rational), **Рамбо** (OMT → диаграммы классов), **Якобсон** (OOSE → диаграммы случаев использования). Заодно создан процесс **RUP** (популярен в конце 90-х, побеждён Agile). Вехи: **1.1** (1997, рядом с первой Java) → **2.0** (2005, переработка, поддержка и аппаратных систем; автоматы и активности разнесли на два языка) → **2.5.1** (декабрь 2017) — актуальная версия **[+]** (на 2026 по-прежнему текущая; UML 3.0 не существует). **14 диаграмм** делятся на **структурные** (структура времени компиляции: классов, компонентов, пакетов, объектов, ...) и **поведенческие** (поведение во время работы: состояний, последовательностей, активностей, случаев использования, ...).

Определение: диаграмма классов — общий синтаксис

Класс — прямоугольник из трёх отделов: *имя*, *поля*, *методы*. Свойство: видимость имя: тип кратность = по_умолчанию. Видимость: + (public), - (private), # (protected), ~ (package). Кратность: 1, 0..1, *, 1..*, 2..* — пишется у *того конца* ассоциации, к которому относится. **Наследование в UML** зовётся «**обобщение**» (generalization), и стрелка указывает на **ПРЕДКА**, а не на потомка (частая путаница).

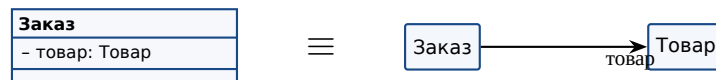


у каждого Order ровно один Customer; у Customer — * (0 и более) Order

Важно: всё можно не показывать. Если у класса не нарисованы поля/методы или у ассоциации нет кратностей/стрелок — это не значит, что их нет: автор счёл их несущественными или оставил на усмотрение реализатора (вспомните «точку зрения моделирования»).

Разбор: атрибуты и ассоциации — это одно и то же

Синтаксически **атрибут** $\equiv$ **ассоциация**, отличается лишь отображением. **Атрибут** удобен, когда связь не важна (тип — элементарный, перечисление, тип-значение или класс из сторонней библиотеки). **Ассоциация** — когда связь важна для понимания архитектуры (тип — класс самой системы). Ассоциациям тоже разрешены атрибуты (класс-ассоциация), но в реальных диаграммах редко.



Разбор: диаграмма → код (Java)

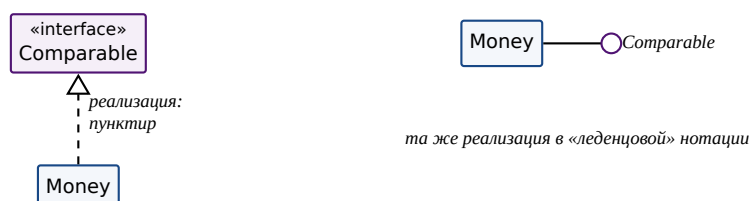
По диаграмме OrderLine (поле quantity, навигабельная ассоциация на Product, свойство price) генерируется код; навигабельная ассоциация стала *полем*, а price — *вычисляемым свойством* (такие преобразования допустимы, если реализатору удобнее).

```
public class OrderLine {
    private int quantity;           // attribute
    private Product product;       // navigable association -> field
    public int getQuantity() { return quantity; }
    public Money getPrice() {      // "price" became a computed property
        return product.getPrice().multiply(quantity);
    }
}
```

Двунаправленные ассоциации сложнее: при изменении одного конца надо поддерживать целостность другого (в сеттере удалить себя из старого владельца и добавить в новый). В C# нет friend из C++, поэтому используют метод с пакетной видимостью, доступный «только другому концу».

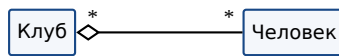
Определение: интерфейсы, зависимости

Интерфейс рисуют двумя способами: блок со стереотипом «interface» и реализация *пунктирной* стрелкой (штрих-треугольник) — когда важны методы; либо «леденцовая» (lollipop) нотация — кружок на линии, когда методы не важны или интерфейсов много. **Зависимость** — самый общий вид связи (пунктирная стрелка: «класс что-то знает о другом»); уточняется стереотипами: «call», «create», «derive», «realize», «responsibility», «refine», «trace». Слова в «ёлочках» — **стереотипы**.

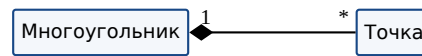


Определение: агрегация и композиция

Уточнения ассоциации по владению. **Агрегация** (открытый ромб) — без владения: участники живут независимо (человек и клуб; клуб закрылся — люди остались). **Композиция** (закрашенный ромб) — владение: время жизни связано, «хозяин» удаляет подчинённого, и у объекта один хозяин (многоугольник владеет точками; удалили многоугольник — удалились точки). **Ромб рисуется у контейнера** (оперение стрелы от контейнера к содержимому); открытый ромб — слабее закрашенного.



агрегация (◇): не владеет



композиция (◆): владеет, общее время жизни

Осторожно: агрегация/композиция чаще не нужны

Они важны прежде всего для **C++** (кто освобождает память). При сборке мусора знать владельца полезно, но в диаграммах архитекторы обычно ограничиваются *простой ассоциацией*, не специфицируя, агрегация это или композиция. И не путайте направление стрелки обобщения — на *предка*.

Шаблоны и перечисления. Параметры-типы — в пунктирном прямоугольнике в правом верхнем углу класса; инстанцирование — как класс с подстановкой или наследование со стереотипом «bind» (если экземпляр доопределяет методы/поля, как QStringList от QList<T> с join()). Перечисление — стереотип «enumeration» и список элементов.

[+] UML сегодня: меньше GUI-инструментов, больше кода

Visual Studio убрал поддержку UML в 2016 (из-за низкого спроса); большинство разработчиков рисуют *неформальные* диаграммы с элементами UML. Там, где UML используют, чаще берут diagrams-as-code: **PlantUML** и **Mermaid** умеют диаграммы классов в текстовом DSL, который живёт в репозитории и рендерится в CI/в README на GitHub. Хорошая быстрая справка по синтаксису — М. Фаулер, «UML. Основы» и сайт uml-diagrams.org.

3. Диаграммы объектов и пакетов UML

Определение: диаграмма объектов

Снимок состояния системы в момент времени: объекты в памяти и связи между ними (множество *всех* состояний задаёт диаграмма классов). По смыслу поведенческая, но считается структурной (описывает структуру времени выполнения). Объект рисуется как класс, но **имя подчёркнуто**, в форме имя:Тип, каждому атрибуту сопоставлено *значение*.

Два применения: (1) проиллюстрировать диаграмму классов (по классам не видно, что это «дерево», по объектам — сразу); (2) разобраться в реальных объектах домена до первой диаграммы классов (тогда диаграмма классов — «наведение классификации» на объекты; этот приём встретится в предметно-ориентированном проектировании).

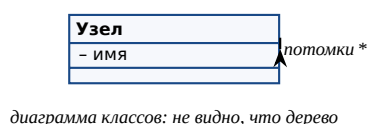


диаграмма классов: не видно, что дерево

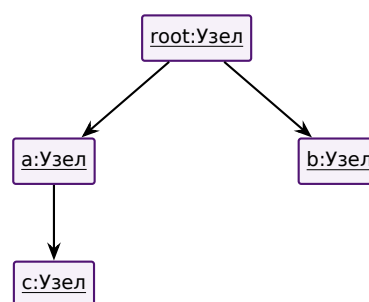
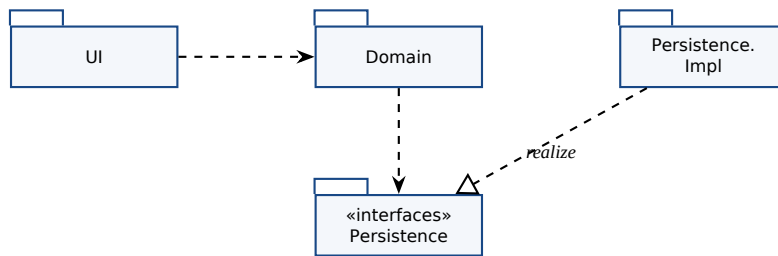


диаграмма объектов: видно дерево

Определение: диаграмма пакетов

Структурная диаграмма: разбиение кода (или модели) по **пакетам** (пакет ≈ пакет Java / пространство имён C++). На них рисуют зависимости между пакетами и содержащиеся классы/компоненты. **Зависимость** (пунктир) — пакет как-то связан с другим; стрелка **реализации** — в пакете-предке есть предок/интерфейс для класса из пакета-потомка (часто — когда смысл пакета-предка: содержать интерфейсы/абстрактные классы для реализации другими).



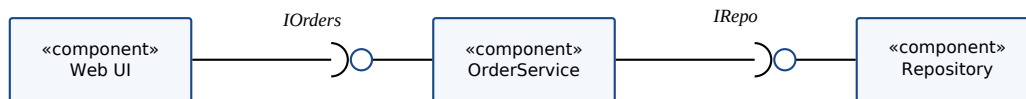
пунктир — зависимость; штрих-треугольник — реализация интерфейсов пакета

4. Диаграммы компонентов UML

Определение: диаграмма компонентов

Самая полезная для архитектора диаграмма: компоненты системы и связи между ними. **Компонент** — нечто структурно связанное и *больше класса* (пакет, сборка, .dll, .so, веб-сервис) — то, из чего состоит высокоуровневая архитектура. Компонент имеет **предоставляемые** («леденец» ◯) и **требуемые** («гнездо» ⊐) интерфейсы; их соединение — **сборочный коннектор**. Бывают вложенность, **порты** и **делегирование** (порт перенаправляет запрос вложенному компоненту).

Почему полезнее диаграмм классов: переживают большинство рефакторингов. Класс переименовали, добавили поле — диаграмму классов перерисовывать; диаграмма компонентов неизменна до *глобальной* переработки архитектуры (раз в несколько лет), поэтому остаётся актуальной. Это вид «с высоты птичьего полёта».



◯ — предоставляемый интерфейс (леденец); ⊐ — требуемый (гнездо); их стык — сборочный коннектор

Разбор: компоненты QReal (вид «с высоты птичьего полёта»)

В QReal связи — *зависимости по сборке*, каждый компонент — отдельный проект (.dll, .so): qkernel ни от кого не зависит (общие классы); qutils — полезные утилиты; qrrepo — репозиторий данных; qrgui — UI; qrmc, qrxс — компиляторы визуальных языков; плагины (editorGenerator, robotsInterpreter, ...). По одной такой диаграмме быстро понятно, «куда смотреть в исходниках».

[+] Компоненты и современная архитектура

Диаграмма компонентов прямо соответствует уровню **Container/Component** в **C4-модели**; в diagrams-as-code её удобно вести через **Structurizr** (один model → виды контекста, контейнеров и компонентов) или **Mermaid**. Для микросервисов компонент — это сервис с провайдимыми/требуемыми API (REST, gRPC), что ровно ложится на нотацию «леденец/гнездо».

Итоговый чек-лист

- ☐ Даю определение модели и её свойства (меньше информации, целенаправленна, субъективна, ограничена); понимаю «all models are wrong, some are useful».
- ☐ Перечисляю виды моделей по формальности (естественный язык → неформальная/формальная графика → формальный текст → исполнимая) с плюсами и минусами.
- ☐ Объясняю **метафору визуализации, точку зрения моделирования, семантический разрыв**; помню про «архитектурный паралич».
- ☐ Знаю, что такое UML (OMG, 14 нотаций, метамодель, профили/метамоделирование), и его вехи (1.1 / 2.0 / 2.5.1); делю диаграммы на структурные и поведенческие.
- ☐ Читаю диаграмму классов: видимости (+, -, #, ~), кратность у нужного конца, **обобщение — стрелка на предка**.
- ☐ Понимаю **атрибут** ≡ **ассоциация**; связь диаграммы с кодом; интерфейсы (две нотации), зависимости и стереотипы.
- ☐ Различаю **агрегацию** (◇, без владения) и **композицию** (◆, владение); знаю, что ромб — у контейнера и что чаще хватает простой ассоциации.
- ☐ Описываю **диаграмму объектов** (снимок, подчёркнутое имя : Тип, показывает дерево) и **диаграмму пакетов** (зависимость, реализация).

- ☐ Объясняю **диаграмму компонентов** (леденец/гнездо, порты, делегирование) и почему она переживает рефакторинги лучше диаграммы классов.
- ☐ **【+】** Знаю современный ландшафт: diagrams-as-code (Mermaid, PlantUML, Structurizr/C4, D2), генерацию диаграмм LLM, и что UML 2.5.1 — по-прежнему актуальная версия.

Лекция 4. Моделирование и анализ предметной области

Рабочая тетрадь

Тетрадь следует за списком экзаменационных вопросов: часть N — практическая проработка вопроса N (в порядке вопросов, не лекции). Диаграммы конечных автоматов UML в этой лекции лишь упоминаются (отложены на лекцию о моделировании поведения) и здесь опущены. Материал сверх текста лекции помечен [+].

1. Диаграммы развёртывания UML

Определение: диаграмма развёртывания

Диаграмма развёртывания (deployment diagram) показывает размещение логических элементов (компонентов, исполнимых файлов) на *физических/виртуальных* устройствах. **Узлы** (параллелепипеды) — устройства или похожее (компьютеры, группы машин, БД, среды выполнения); внутри них — развёрнутые **артефакты** (библиотеки, компоненты). Связи возможны *только между узлами* и показывают каналы связи с *протоколом*. Узлы и артефакты могут иметь свойства в фигурных скобках; артефакт — подробно (прямоугольник) или кратко (имя).



Почему это инструмент анализа, а не только финала. Часто железо известно заранее (заказчик уже купил два сервера и пяток терминалов), и наличная инфраструктура во многом *определяет* архитектуру — диаграмма развёртывания фиксирует, что есть и как мы это используем. Второе применение — **обратное проектирование**: у заказчика система есть, документации по коду нет, но админы знают, какую машину перезапускать; отобразив их знания, получаем первое приближение высокоуровневой архитектуры.

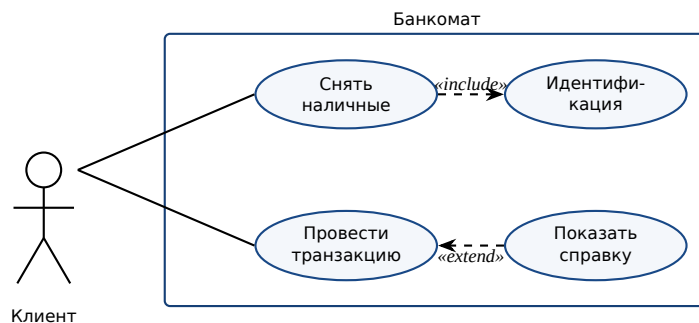
[+] Развёртывание сегодня — это код

Топологию развёртывания теперь чаще описывают *исполнимо*: **контейнеры** (Docker), **манифесты Kubernetes** (Deployment, Service, Pod), **infrastructure-as-code** (Terraform, Helm, Pulumi). Это «живая» диаграмма развёртывания, которая ещё и разворачивает систему. Концептуальная UML-диаграмма остаётся полезной как обзор; в **C4** ей соответствует *Deployment diagram*.

2. Диаграммы случаев использования UML

Определение: диаграмма случаев использования

Появились в методологии **OOSE** (Якобсон, 1992), вошли в UML почти без изменений. Состоят из трёх вещей: **акторы** (внешние сущности — роли пользователей или внешние системы, *использующие* нашу), **случаи использования** (цель использования системы актором — 1–3 слова: «Оценить сделку»), и **граница системы**. **Случай использования** ≠ **функция**: «Залогиниться» — не случай использования (это и не цель пользователя, и нефункциональное требование). Каждый случай раскрывается в один-несколько **сценариев использования** (что и в каком порядке делает пользователь; обычно текстом / user story; можно диаграммой активностей).



«include»	случай <i>всегда</i> включает другой (как вызов подпрограммы): «Снять наличные» включает «Идентификацию»
«extend»	случай <i>может</i> расширяться в <i>точке расширения</i> (опциональное поведение): «Показать справку» расширяет транзакцию. Это уточнение «include»
наследование	классификация акторов/случаев (пассажир: малолетний / инвалид / экскурсовод)

Структура сценария использования: заголовок (цель основного актора); заинтересованные лица и *основной* актор; предусловия; триггеры; основной порядок событий («всё по плану»); альтернативные пути и расширения (ошибки, точки расширения); постусловия.

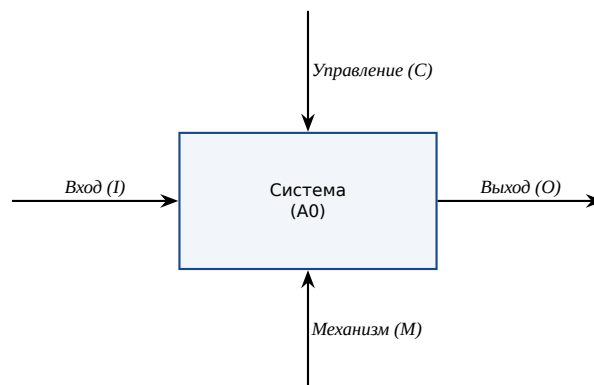
[+] Требования сегодня

Случаи использования живы (легко вписываются даже в Agile — из всего UML можно взять только их). На практике сценарии чаще пишут как **user story** («As a ... I want ... so that ...») и **Gherkin** («Given-When-Then») для BDD (Cucumber). Для формальных деревьев требований — **диаграмма требований SysML**; в 2025 OMG принял **SysML v2** (текстовая + графическая нотация на базе KerML, git-дружелюбная) — крупный пересмотр SysML 1.x из лекции.

3. Диаграммы IDEF0 (контекстные)

Определение: IDEF0 и контекстная диаграмма

IDEF0 (семейство IDEF, ВПК США, 1970-е; также **SADT**) — структурная декомпозиция системы или бизнес-процесса в набор шагов-блоков, соединённых входами/выходами. У блока — четыре стороны (**ICOM**): слева **вход** (Input — что перерабатывается), сверху **управление** (Control — что определяет, как работать: инструкции, настройки), снизу **механизм** (Mechanism — что *позволяет* работать: БД, станки), справа **выход** (Output). Блок раскрывается своей IDEF0-диаграммой, причём *каждую связь можно отследить*. В корне иерархии — **контекстная диаграмма**: один блок = вся система, стрелки извне.



Зачем. Контекстная диаграмма кажется тривиальной (один блок!), и в этом её сила: она чётко задаёт *границу системы*, её входы и выходы, управление так, что всё помещается на одном экране. IDEF0 чаще применяют к *бизнес- и производственным* процессам, чем к ПО (для ПО есть UML) — но именно модель бизнес-процесса нужна, чтобы грамотно описать требования.

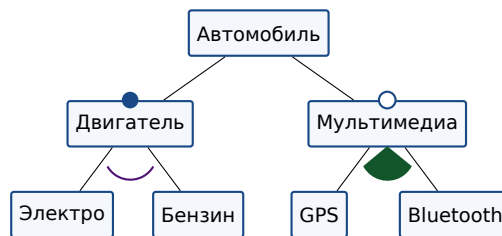
[+] Где IDEF0 сегодня

IDEF0 был федеральным стандартом США (FIPS 183, формально отозван, но всё ещё применяется в системной инженерии, оборонке, производстве). Для повседневного моделирования процессов его в основном вытеснили **BPMP** и **value stream mapping** (Lean); IDEF0 ценят за строгую трассируемость ICOM и контекстную границу.

4. Диаграммы характеристик (feature diagrams)

Определение: диаграмма характеристик

Дерево требований/характеристик. Характеристики бывают **абстрактные** (надо декомпозировать) и **конкретные** (реализуемые). Отношения: **обязательная** (заполненный кружок) / **необязательная** (пустой кружок); группы — **ИЛИ** (ог, можно несколько) и **исключающее ИЛИ** (alternative/хор, ровно одна). Главное применение — **линейки продуктов** (Professional vs Ultimate): отмечаем, какие характеристики куда входят, и при удаче *автоматически генерируем* конфигурацию или код.



● — обязательная характеристика, ○ — необязательная; открытая дуга — *xor* (ровно одна), закрашенная — *or* (можно несколько).

Минусы: громоздки, почти не поддерживаются CASE-средствами. **Пример (BRICS):** конфигурирование ПО сервисных роботов — описываются характеристики аппаратной платформы и доступного ПО, плюс отображения и зависимости между ними (нет камеры → нельзя распознавать маркеры), а потом под задачу выбираются характеристики, и система сама генерирует код развёртывания и конфигурации.

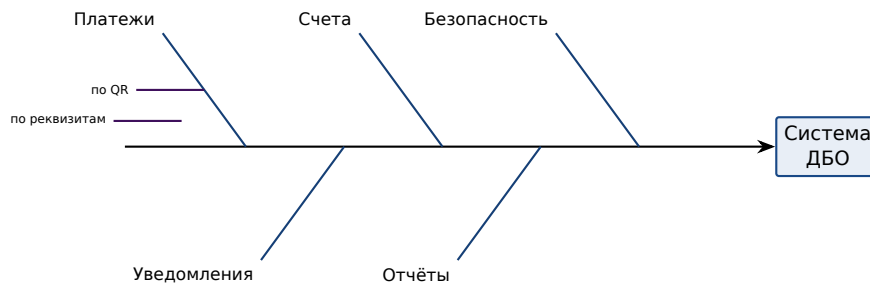
[+] Фейтур-моделирование сегодня

Истоки — **FODA** (Kang, 1990). Промышленные инструменты линеек продуктов: **pure::variants**, **Gears**, **FeatureIDE**. Родственная (но runtime-) идея — **feature flags/toggles** (LaunchDarkly, Unleash): включать характеристики уже в проде, без пересборки.

5. Feature tree (дерево характеристик)

Определение: feature tree / fishbone

Более простая и быстрая нотация, чем диаграмма характеристик, для *первоначального* анализа (brainstorm нового проекта). «Голова рыбы» — система целиком; от неё идёт «хребет», от которого ответвляются основные фиши, а от них — более мелкие. **Взаимосвязей не показывать**, но как инструмент стартового анализа очень полезна; потом уже рисуют случаи использования, диаграмму характеристик и т.д. (отсюда второе имя — *fishbone diagram*).



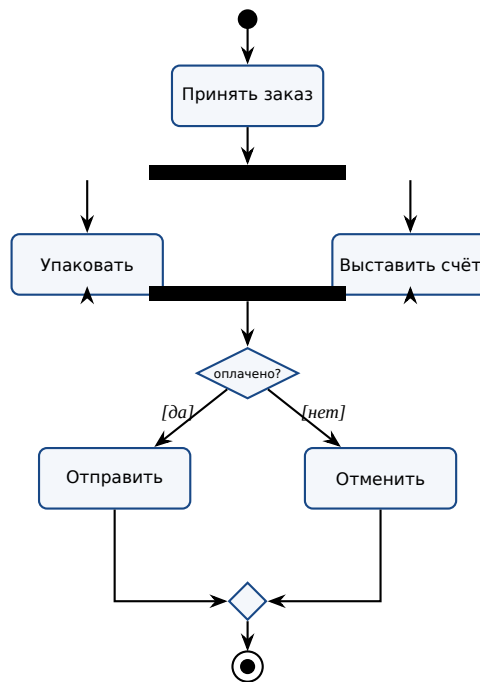
[+] Современные «лёгкие» техники старта

Те же цели сегодня закрывают **impact mapping**, **user story mapping** (Jeff Patton) и обычные mind-map-инструменты — быстрый набросок, который потом превращается в бэклог и более формальные модели.

6. Диаграммы активностей UML

Определение: диаграмма активностей

Внешне как блок-схема, но моделирует *поведение бизнес-процесса* (или алгоритм). **Одна из двух диаграмм UML с исполнимой семантикой** (на основе **сетей Петри**; вторая — автоматы). **Ветвление/объединение** = fork/join параллельных потоков (объединение *ждёт все*); потоки можно не объединять. **Решение/слияние** (decision/merge) — как switch: условия пишутся *над стрелками* (не в ромбе!) и должны быть *взаимоисключающими*. **Дорожки (swimlanes)** — разделение работ по отделам/частям системы. **Сигналы** — асинхронное взаимодействие (посылка сразу отдаёт управление; приём ждёт; таймер — отложенный сигнал себе).

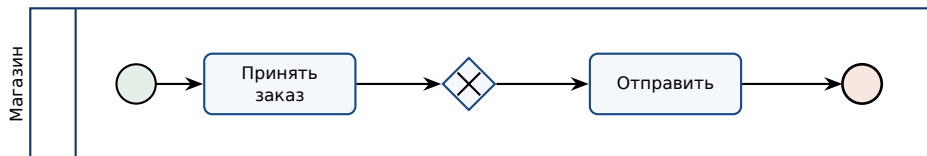


● — *старт*; чёрные полосы — *fork/join*; ромб — *решение (условия пишутся над стрелками и взаимоисключающи)*; ◎ — *конец деятельности*.

7. Язык BPMN

Определение: BPMN

BPMN (Business Process Model and Notation) — отдельный язык OMG (BPMN 1.0 — 2004, рядом с UML 2.0; задан метамоделью), *не* часть UML, но близкий родственник; набор диаграмм специально под бизнес-процессы. **Как продвинутые диаграммы активностей:** связывает *несколько* процессов настоящими стрелками, богаче ветвления и события, качественные исключения. Брать, когда процессов и участников много (сотни субподрядчиков) — тогда лучше нанять **бизнес-аналитика**. **Три диаграммы:** процессов, хореографии (кто когда с кем общается), диалогов (кто с кем *может* общаться).



○ *старт-событие*; прямоугольник — *задача*; ◇ со знаком — *шлюз (gateway)*; ● *толстый круг — конец*; всё в пуле с дорожками

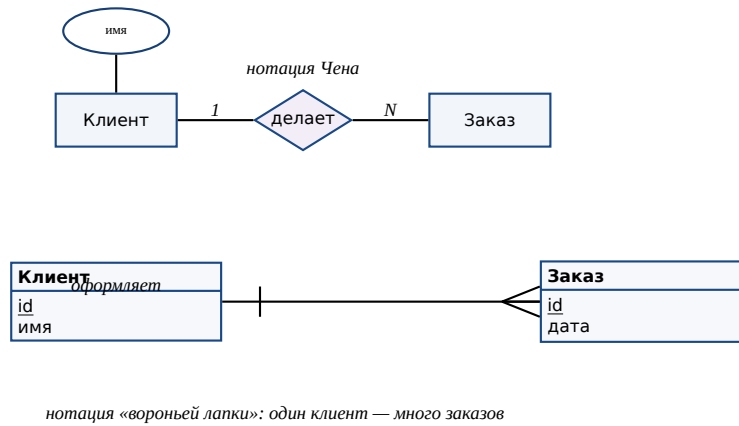
[+] Где лекция устарела: BPMN выполняется *напрямую*

Лекция говорит, что BPMN переводят в **BPEL** и исполняют BPEL-движками. Сегодня это *legacy*: **BPMN 2.0** (2011; 2.0.2 — 2013) выполняется *напрямую* современными движками без перевода в BPEL. Лидер — **Camunda 8** на движке **Zeebe** (cloud-native, распределённый, для микросервисов; Camunda 7 сворачивается); также **Flowable**, **Activiti**, **jBPM**. Решения выносят в **DMN** (таблицы решений). Альтернатива «процесс как код» — **Temporal**. Связь с лекцией: шлюзы/задачи BPMN ложатся на переходы сетей Петри → формальная проверка тупиков.

8. Диаграммы «Сущность-связь» (ER)

Определение: ER-диаграммы

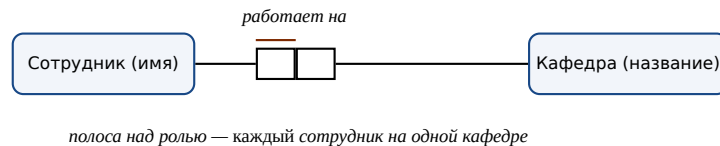
Концептуальная модель данных предметной области (затем — схема реляционной БД, куда ER ложится семантически идеально). **Нотация Чена** (1976): *прямоугольник* — *сущность*, *овал* — *свойство*, *ромб* — *связь (может иметь свойства)*; у связей — кратность (1:1, 1:N, M:N), атрибут может быть *первичным ключом*. На практике используется **нотация «вороньей лапки»** (crow's foot): атрибуты пишутся *внутри* сущности (компактнее), кратность — символами на концах связи («лапка» = «многие»).



9. Концептуальное моделирование, диаграммы ORM

Определение: Object-Role Modeling (ORM)

Ещё одна нотация концептуальной модели/схемы БД, реже ER, но *прогрессивнее* для анализа. В ней *только сущности и связи, атрибутов нет*. Сущность имеет **схему идентификации** (признак в скобках). Связи часто **п-арные** (в ER — только бинарные), играют роль таблиц; сущности играют в связи **роли** (прямоугольники на связи). Над связью — *вариант прочтения* на естественном языке («Университет присудил степень человеку»), чтобы аналитик мог проверить разумность. Полосы над ролями — *уникальность сочетаний*; роли могут быть связаны ограничениями (например, исключающее ИЛИ).



	ER	ORM
Атрибуты	есть (внутри сущности)	нет — всё через связи и сущности
Связи	бинарные	часто п-арные, с ограничениями
Размер	компактнее	крупнее (атрибут = сущность)
Устойчивость к изменениям	хуже (атрибут получил атрибуты → переделка)	лучше — просто добавить связь; удобна на ранних этапах

Осторожно: ORM — это два разных понятия

В этой теме ORM = **Object-Role Modeling**. В программировании **ORM** чаще = **Object-Relational Mapping** — библиотеки/технологии, представляющие строки реляционной БД как объекты и обратно («impedance mismatch»: в реляционной модели нет наследования и трудно выразить M:N). **[+]** типичные ORM-фреймворки: **Hibernate/JPA** (Java), **Entity Framework Core** (.NET), **SQLAlchemy** (Python), **Prisma** (TypeScript).

[+] Моделирование данных сегодня

ER по-прежнему ядро проектирования схем. Diagrams-as-code: **Mermaid** (erDiagram), **dbdiagram.io/DBML**, **PlantUML** — схема как текст в репозитории. ORM-фреймворки сами генерируют схему по моделям (code-first) и миграции.

Итоговый чек-лист

- ☐ **Развёртывание:** узлы (параллелепипеды) с артефактами, связи только между узлами с протоколом; ценно для анализа (известное железо, обратное проектирование).
- ☐ **Случаи использования:** акторы, случаи (цель $\neq$ функция, не «залогиниться»), граница; «include» (всегда) vs «extend» (может); сценарии и их структура.
- ☐ **IDEFO:** ICOM (вход/управление/механизм/выход), декомпозиция с трассируемостью; контекстная диаграмма — один блок = граница системы.
- ☐ **Диаграмма характеристик:** обязательные/необязательные, ог/хог, абстрактные/конкретные; линейки продуктов и генерация конфигурации.
- ☐ **Feature tree (fishbone):** быстрый старт-анализ; голова-хребет-фики; взаимосвязей не показывает.
- ☐ **Диаграмма активностей:** семантика на сетях Петри; fork/join, decision/merge (условия над стрелками, взаимо-исключающие), дорожки, сигналы/таймер.

- ☐ **BPMN:** продвинутые активности, OMG, метамодель; диаграммы процессов/хореографии/диалогов; **[+]** выполняется напрямую (Camunda 8/Zeebe), BPEL — legacy.
- ☐ **ER:** Чен (прямоугольник/овал/ромб) vs воронья лапка (атрибуты внутри); кратности, ключи.
- ☐ **ORM (Object-Role):** только сущности и роли, n-арные связи, схема идентификации, уникальность; устойчивее ER на ранних этапах; не путать с Object-Relational Mapping.
- ☐ **[+]** Современный контекст: развёртывание как код (K8s, Terraform), SysML v2 (2025), BPMN-движки, diagrams-as-code для ER (Mermaid/DBML), ORM-фреймворки.

Лекция 5. Моделирование поведения

Рабочая тетрадь

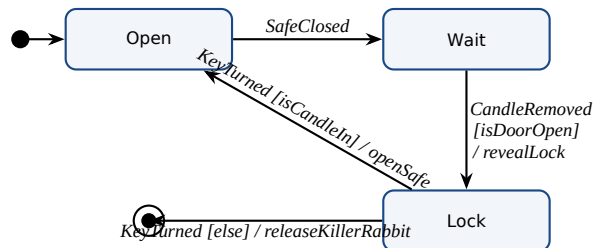
Тетрадь следует за списком экзаменационных вопросов: часть N — практическая проработка вопроса N . Материал сверх текста лекции помечен [+].

1. Диаграммы конечных автоматов UML

Определение: диаграмма конечных автоматов (состояний)

Упрощённые **диаграммы Харела** (1987); вторая диаграмма UML с исполнимой семантикой (первая — активно-стей). Моделируют **реактивные** системы — те, что находятся в чётко определённых **состояниях** и реагируют на **события**, переходя между ними и делая работу (сетевое соединение, торговый автомат, лексический анализатор). Состояние — прямоугольник со скруглением; переход — событие [стражник] / действие. **Стражники взаимноисключающе** (недетерминизм некорректен). Псевдосостояния **начала** (мгновенный переход) и **конца**.

	Активностей	Конечных автоматов
Узлы	активности (не задерживаемся)	состояния (стабильные отрезки жизни)
Полезная работа	в активностях	как правило, при переходе
Масштаб	один метод/функция	целый объект (состояние $\approx$ поля)



Внутри состояния: entry/exit (при входе/выходе по любому переходу), **do** (пока система в состоянии), **внутренний переход** (по событию, без срабатывания entry/exit — в отличие от петли-перехода в то же состояние). Синтаксис надписи на переходе тот же: триггер[, триггер]* [стражник] / действие.

Connected
entry / startSession() do / monitorLink() exit / closeSession() Disconnect [count<5] / count++ (внутренний)

Вложенные состояния содержат свой автомат (переход «cancel» из любого вложенного заменяет нестандартное псевдосостояние со звёздочкой). **Параллельные** состояния — пунктирная линия делит на независимые подавтоматы (часы: звук и время независимы). **Псевдосостояние истории** помнит последнее вложенное состояние и возвращает в него.

Разбор: генерация кода (три способа)

(1) Гигантский switch по состоянию, внутри — switch по событию, внутри — стражники, действие и смена состояния (состояние = enum в поле). Работает, но один метод на тысячи строк. **(2) Таблица состояний** + универсальный интерпретатор (популярно в лексическом/синтаксическом анализе, но тяжело отлаживать). **(3) Паттерн «Состояние»**: поле-ссылка на интерфейс «текущее состояние» с методом на каждое событие; конкретные классы-состояния возвращают следующее состояние. Это тот же switch, спрятанный в таблицу виртуальных методов — читаемо, легко добавлять состояния.

```

public void handleEvent(PanelEvent e) {           // (1) giant switch
    switch (currentState) {
        case Wait:
            if (e == CandleRemoved && isDoorOpen) {
                revealLock();
                currentState = Lock;                // guard, action, transition
            }
            break;
        case Lock:
            if (e == KeyTurned)
                currentState = isCandleIn ? Open : Final;
            break;
    }
}

```

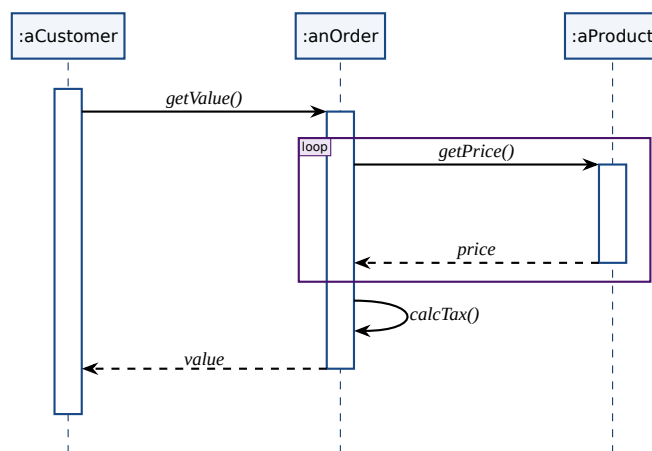
[+] Автоматы сегодня: statechart-движки и SCXML

XState (v5, JS/TS; ~1 млн загрузок/нед, ~25k звёзд) — statecharts + модель акторов для фронтенда и бэкенда; визуализация и **model-based testing** (тесты генерируются из машины). **SCXML** — стандарт W3C (2015) сериализации statechart'ов Харела, исполнимый разными рантаймами; **Spring StateMachine**, **Stateless** (.NET), **Boost.SML** (C++). Генерация кода из автоматов индустриализована — например, **автокодер JPL** генерирует бортовой C для марсохода Curiosity из statechart'ов.

2. Диаграммы последовательностей UML

Определение: диаграмма последовательностей

Визуализирует *взаимодействие объектов* (не классов!) во времени: передачу сообщений, возврат значений, время жизни. У каждого объекта — **линия жизни** (пунктир, пока объект существует) с **линиями активации** (белый прямоугольник — пока какой-то метод объекта на стеке вызовов; одновременно их может быть несколько — рекурсия). **Сообщение** (стрелка между активациями) ведёт в *начало* соответствующей активации; стрелка не выходит из неактивного объекта и не входит «в никуда». Ось X — объекты, ось Y — время (сверху вниз).



Применение: сложные протоколы взаимодействия (многопоточность, асинхронность, телеком — почти копия языка **MSC**); анализ домена; план тестирования (кто кого и в каком порядке дёргает); визуализация логов при отладке. **Создание/удаление** объекта: создание — вызов конструктора; удаление — деструктор или «самоубийство» в конце активации. **Фреймы** «loop»/«alt»/«opt» позволяют показать цикл/ветвление/опцию, но *сильно* ухудшают читаемость — для алгоритмов лучше диаграммы активностей или обзора взаимодействия.

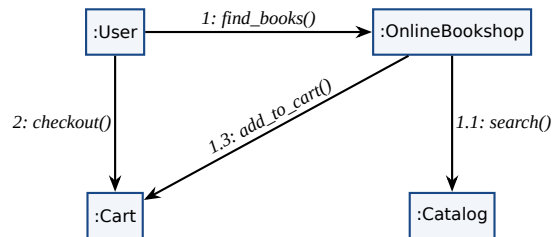
[+] Последовательности сегодня

Mermaid (sequenceDiagram) и **PlantUML** — одни из самых ходовых diagrams-as-code для API-флоу и аутентификации (живут в репозитории, рендерятся на GitHub). **Распределённая трассировка** (**OpenTelemetry**, Jaeger, Zipkin) автоматически строит sequence-подобные «водопады» вызовов между микросервисами по реальным трейсам.

3. Диаграммы коммуникации UML

Определение: коммуникационная диаграмма

Те же диаграммы последовательностей, но «с высоты птичьего полёта»: один сценарий, та же последовательность сообщений, но объекты размещены на **двумерной плоскости**, а порядок задаётся **числовыми индексами** на стрелках (1, 1.1 — первый вызов внутри 1, 1.3 — третий внутри 1). **Плюс:** компактнее последовательностей. **Минус:** порядок во времени *не очевиден*.

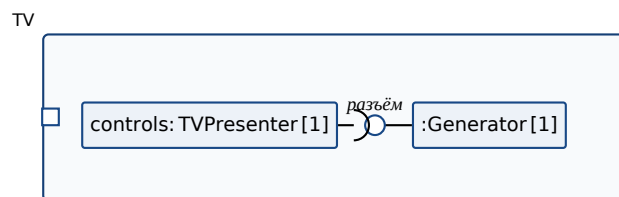


порядок — по индексам (1, 1.1, 1.3, 2), не по геометрии

4. Диаграммы составных структур UML

Определение: диаграмма составных структур

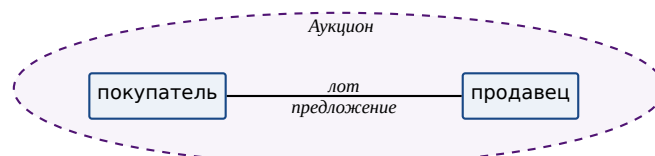
Продвинутые диаграммы компонентов (часто для аппаратуры из стандартных блоков). Отличие: внутри объемлющего компонента — не другие компоненты, а **роли**. **Роль** — это «как объект»: ролей одного *типа* может быть несколько, по-разному связанных; в работающей системе роль играет любой компонент правильного типа. [1] — кратность. Порты, интерфейсы и связи — как на диаграмме компонентов (только связи зовут **разъёмами**). Если диаграмма компонентов — структура *времени компиляции*, то здесь — структура *времени выполнения* (роли).



5. Диаграммы коопераций UML

Определение: диаграмма коопераций (collaboration)

Нечто среднее между диаграммами объектов и классов: вместо классов/объектов — **роли** (сущности, на место которых подставляется настоящий объект). Показывает взаимодействие ролей *в рамках одного сценария использования*. Два стиля: пунктирный овал-кооперация с ролями внутри, или «ближе к диаграмме классов» — класс, *исполняющий* роли. Можно дополнять диаграммой последовательностей (там пишут имя_объекта / имя_роли), чтобы добавить временной аспект.

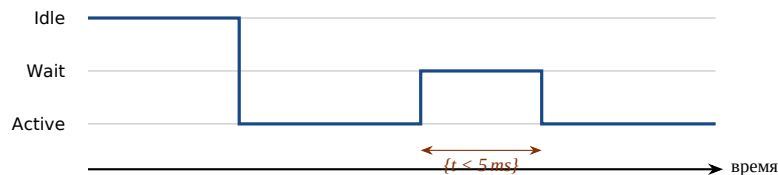


роли в сценарии «Аукцион»; место роли занимает реальный объект

6. Временные диаграммы UML

Определение: временная диаграмма (timing)

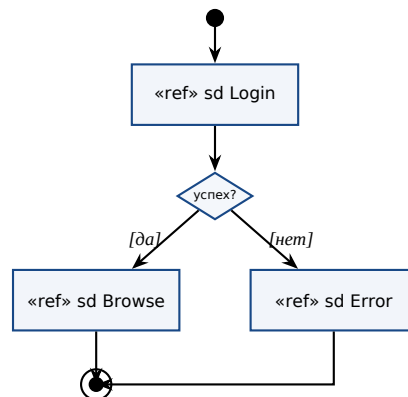
Для электронщиков и систем реального времени: последовательности событий с чёткими **временными ограничениями**. **Время идёт слева направо**; объекты расположены вертикально и находятся в состояниях. **Вариант 1** — переключение как *скачок линии* (нагляднее); **вариант 2** — *пересечение линий* с именем состояния внутри (удобнее, когда состояний много). Ограничения — в фигурных скобках, как принято в UML.



7. Диаграммы обзора взаимодействия и диаграммы потоков данных

Определение: диаграмма обзора взаимодействия

Совмещение диаграммы активностей и диаграммы последовательностей: узлы потока управления — это *фреймы* взаимодействий (ref/sd). Нужны, когда есть сложный алгоритм с ветвлениями и циклами, который надо показать как последовательность — фреймов в обычной диаграмме последовательностей было бы слишком много, и она стала бы нечитаемой.



Определение: диаграмма потоков данных (DFD)

Не входит в UML и не имеет там аналога; самая древняя и популярная. **Один** вид стрелок — поток данных; **три** вида сущностей: **процесс** (преобразует данные, *кружок*), **внешняя сущность** (поставляет/потребляет, *прямоугольник*), **хранилище** (где данные лежат, *две горизонтальные линии*). Полезна как первый набросок архитектуры или иллюстрация к готовой системе.

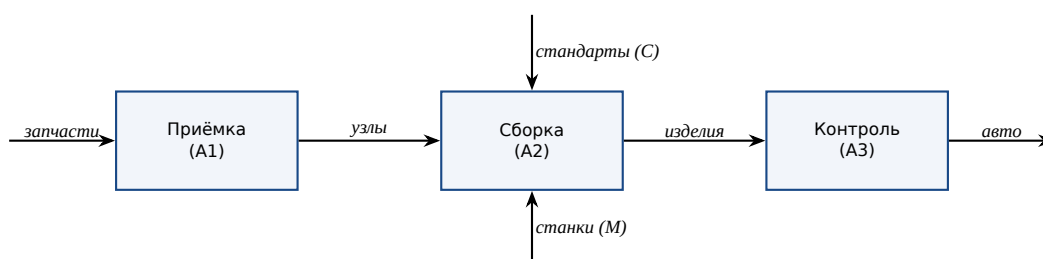


кружок — процесс; прямоугольник — внешняя сущность; две линии — хранилище

8. Диаграммы IDEF0

Определение: IDEF0 (декомпозиция)

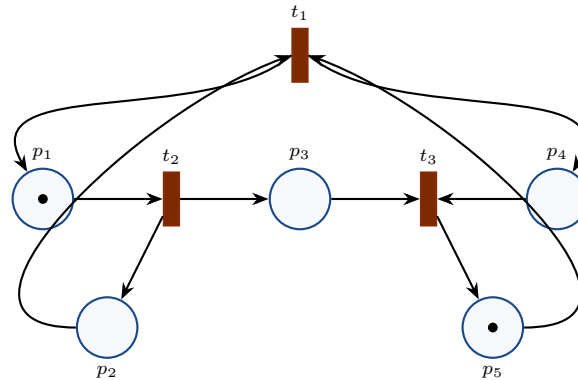
Детализация контекстной диаграммы: блоки (имя + однозначный **номер**) и стрелки. **ICOM**: слева — *материалы* (вход), справа — *результаты* (выход), сверху — *управление* (что регламентирует: стандарты, заказы), снизу — *механизмы/ресурсы* (что нужно для работы, но не перерабатывается: станки, рабочие). Стрелки могут **ветвиться** и **сходиться**; каждая входящая/исходящая стрелка должна продолжаться на диаграмме *уровнем выше* (трассируемость до контекстной). Чаще для бизнес-процессов; в ПО IDEF0 заменяют диаграммы компонентов и составных структур, но аналитики любят IDEF0, и архитектору приходится его читать.



9. Сети Петри

Определение: сеть Петри

Не просто нотация, а **математический формализм** анализа *параллельных* программ. Граф из **мест** (круги), **переходов** (закрашенные прямоугольники) и **токенов** (точки в местах). **Семантика**: переход *срабатывает*, если в каждом входном месте есть ≥ 1 токен — забирает по токenu из входных и кладёт по токenu в выходные. Формально — тройка (P, T, ϕ) с функцией потока $\phi : (P \times T) \cup (T \times P) \rightarrow \mathbb{N}$; **маркировка** $\mu : P \rightarrow \mathbb{N}$; **срабатывание** $\mu \xrightarrow{t} \mu' : \mu'(p) = \mu(p) - \phi(p, t) + \phi(t, p)$. **Конечный автомат** — частный случай (каждый переход: ровно одно входное и одно выходное место).



Производитель (p_1, p_2) кладёт данные в буфер p_3 ; потребитель (p_4, p_5) забирает из него — параллельно. Готов сработать t_2 (в p_1 есть токен); t_3 ждёт токен в буфере p_3 .

Зачем формализм: сети Петри не тьюринг-полны, поэтому многие свойства **алгоритмически разрешимы** (для машин Тьюринга — нет).

Достижимость

можно ли из одной маркировки получить другую (доказать, что программа завершится / попадёт в состояние)

Ограниченность / безопасность

токенов в месте не больше k / не больше 1 (нет переполнения буфера)

Живость (L0-L4)

степени «нет дедлоков»: L0 — мёртв; L1 — переход *может* сработать (нет мёртвого кода); L4 — может сработать *всегда*

Реверсabilidade / домашнее состояние

всегда можно вернуться в исходное / есть состояние, достижимое отовсюду

Структурные (для любой маркировки)

структурная живость, полная контролируемость, структурная ограниченность, консервативность (число токенов константно)

Способы анализа. Алгебраический (быстрый, слабый): состояние — вектор длины $|P|$, матрица переходов M ; $\mu' = \mu + M^T \cdot v$ по вектору срабатываний v . Если уравнение не решается — конечное состояние недостижимо; если решается — *ещё ничего не значит* (уравнение игнорирует последовательность и живость). **Структурный** (сифоны/ловушки) и **анализ редукцией** (склейка по правилам — упрощает сеть). **Анализ пространства состояний** (самый мощный и трудоёмкий): дерево/граф всех маркировок; дерево бесконечно, поэтому строят граф, а неограниченные места помечают ω («сколь угодно много») — граф конечен. Минус — экспоненциальный рост, но для не-тьюринг-полного формализма это успех.

[+] Где сети Петри сегодня («в IT-компаниях не встречал» — устарело)

Теория сетей Петри — фундамент **process mining**: по логам событий *автоматически* строят модель процесса (**process discovery** — часто прямо сеть Петри/BPMN) и проверяют **conformance** (где реальность расходится с моделью). Пионер — **W. van der Aalst**; инструменты **Celonis** (крупнейший; Siemens, BMW, Uber), **ProM**, **PM4Py**, Disco, Aptomore (30+ вендоров; Gartner MQ 2023). Плюс **верификация workflow**: *soundness* BPMN-процессов проверяют через сети Петри (связь с лекцией 4); инструменты TINA, CPN Tools, LoLA.

Итоговый чек-лист

- ☐ **Конечные автоматы**: реактивные системы; состояния, переходы событие [стражник] / действие; entry, exit, do, внутренний; вложенные, параллельные, история; генерация кода (switch / таблица / паттерн Состояние).
- ☐ **Последовательностей**: объекты, линии жизни и активации, сообщения и возвраты, фреймы loop/alt/opt; MSC; отладка по логам.
- ☐ **Коммуникации**: то же «с высоты птичьего полёта» — 2D-плоскость, порядок по числовым индексам (1, 1.1); компактно, но порядок неочевиден.

- ☐ **Составных структур:** продвинутые компоненты, но внутри — **роли** (структура времени *выполнения*); порты, интерфейсы, разъёмы, кратность.
- ☐ **Коопераций:** роли в рамках одного сценария использования; два стиля; дополняется последовательностью (объект / роль).
- ☐ **Временные:** время слева направо, состояния объектов, ограничения в {скобках}; два варианта (скачок линии / пересечение).
- ☐ **Обзора взаимодействия** (активности из фреймов sd) и **DFD** (процесс/внешняя сущность/ хранилище, один вид стрелок).
- ☐ **IDEFO:** декомпозиция, ICOM, ветвление/слияние стрелок, трассируемость до контекстной.
- ☐ **Сети Петри:** места/переходы/токены, семантика срабатывания, (P, T, ϕ) ; свойства (достижимость, ограниченность- /безопасность, живость L0–L4, реверсабельность); анализ (алгебраический / структурный / редукция / пространство состояний, ω); ФА — частный случай.
- ☐ **【+】** Современное: XState и SCXML (автоматы), Mermaid и трассировка (последовательности), process mining и Celonis (сети Петри), STRIDE threat modeling (DFD).

Рабочая тетрадь. Лекция 6

Структурные шаблоны проектирования

Один раздел на каждый экзаменационный вопрос (в том же порядке), углублённо: назначение, структура (участники), «когда брать», разбор на сквозном примере (текстовый редактор из «банды четырёх»), «грабли» реализации. Дополнения практиков помечены [+].

Ориентир: паттерн — это идея, а не рецепт

Паттерн — повторяемое решение типовой задачи проектирования в типовом контексте; у него есть *имя* (унификация общения), он *известен* и *переиспользуем* в разных языках/областях. Паттерн — не конкретный рецепт «вставить и станет лучше» и не языковая идиома (RAII — не паттерн). Деление на структурные/поведенческие/порождающие условно — не зубрите его.

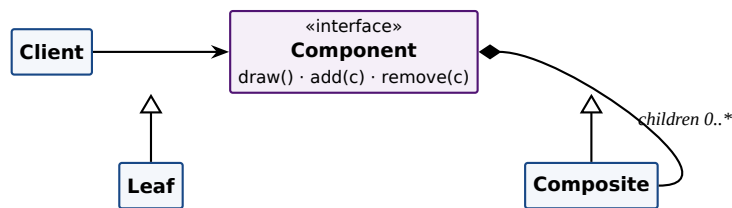
Четыре из семи паттернов лекции — «обёртки» с одинаковой структурой, но *разным назначением*; их постоянно путают:

Паттерн	Интерфейс обёртки	Зачем оборачиваем
Адаптер	другой (Target $\neq$ Adaptee)	приспособить несовместимый интерфейс
Декоратор	тот же	динамически <i>добавить</i> поведение/состояние
Заместитель	тот же	контролировать доступ / создание реального объекта
Фасад	новый упрощённый	спрятать сложность целой подсистемы

1. Паттерн «Компоновщик» (Composite)

Определение

Представляет иерархию *часть-целое* в виде дерева и позволяет работать с одиночными и составными объектами *единообразно*. Участники: **Component** (общий интерфейс/базовый класс с операциями функциональности и, обычно, управления потомками), **Leaf** (лист — без потомков), **Composite** (узел со ссылками на Component, реализует операции через потомков).



Разбор: структура документа как дерево («рекурсивная композиция»)

Всё рисуемое — наследник Glyph с операциями draw, intersects, insert. Листья — символ, изображение; составные — строка, колонка, страница. Отрисовка составного глифа = пройтись по потомкам, при нужде вызвать форматирование и нарисовать каждого. Клиент (форматирование, отрисовка) знает только Glyph и не различает букву, картинку и таблицу.

Когда брать. Везде, где данные древовидны и важна единообразная обработка: GUI-деревья, DOM в браузере, разобранный XML, AST в компиляторе. Добавить новый вид узла — просто новый наследник, клиентский код не трогаем.

Грабли: «Компоновщик» плохо уживается с ООП

Где объявить add/remove? Дилемма прозрачности vs безопасности:

- в Component — единообразно, но в Leaf нет реализации $\Rightarrow$ либо *бросать исключение* (упасть рано), либо *пустое поведение* (Leaf как узел с 0 потомков — приём **Null Object**);
- только в Composite — Leaf-клиенты довольны, остальным нужны приведение типа или getComposite() (возвращает null для листа).

Список потомков — обычно в Composite (а не Component); не обязан быть списком (хеш-таблица для разреженных детей, как подписка на события в WPF). **Ссылка на родителя** — доп. инвариант (все дети считают родителем этот узел); часто проще передавать родителя в рекурсию. **Разделяемые листья/поддеревья** (один Null Object на всех; общие подвыражения в компиляторе) экономят память, но ломают ссылки на родителя $\Rightarrow$ см. «Приспособленец». **Кеш** (bounding box для быстрого Intersects) требует *инвалидации* вверх по дереву.

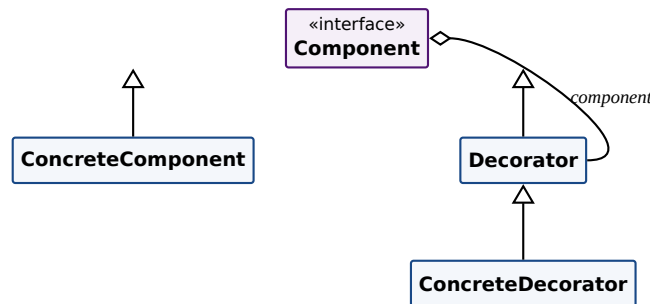
[+] Где это сегодня

Дерево компонентов React/Vue, виртуальный DOM, AST в ESLint/Babel/Roslyn, сцены в игровых движках (scene graph), файловые деревья. Дилемма «Leaf vs Composite методы» — частный случай нарушения ISP.

2. Паттерн «Декоратор» (Decorator)

Определение

Динамически навешивает на объект новые обязанности, сохраняя *тот же интерфейс*. Decorator наследуется от Component (поэтому неотличим от него для клиента) и держит *поле* типа Component: неинтересные методы пересылает внутрь, нужные — переопределяет (или добавляет новые + своё состояние).



Разбор: «прозрачное обрамление» (MonoGlyph)

Border и Scroller — глифы-обёртки с *ровно одним* потомком (вырожденный «Компоновщик»), но цель — дополнить поведение. Порядок вложения меняет результат: *рамка* → *прокрутка* скроллит содержимое рамки; *прокрутка* → *рамка* скроллит вместе с рамкой. Внешний мир работает с обёрнутым глифом как с обычным.

Почему не наследование. Наследование фиксируется при компиляции; декоратор позволяет добавлять/убирать возможности *во время выполнения* и комбинировать их. Плата — много мелких объектов и лишние полиморфные вызовы.

Грабли

Component должен быть *маленьким*: (1) по числу методов — иначе замучаешься их пересылать; (2) по состоянию — декоратор *наследует поля*, но не использует их (новичок меняет поле декоратора, а работает спрятанный объект со *своими полями*). В идеале Component — *интерфейс* без состояния. Методы должны быть *виртуальными* (в C#/C++ по умолчанию нет; `final` в Java мешает).

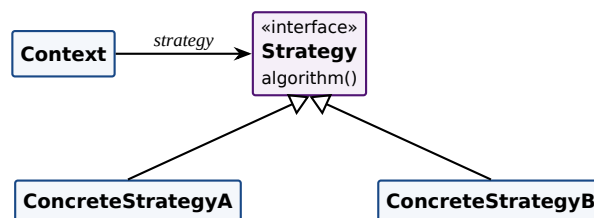
[+] Декоратор vs Стратегия vs хуки; где сегодня

Стратегия — «декоратор наоборот»: вкладывается *внутри* объекта (нужно заранее знать точки параметризации); **хуки** — лёгкие точки расширения в Component. Декоратор гибче (не требует менять Component). Сегодня: *Java I/O* (BufferedReader оборачивает Reader), *middleware* в Express / ASP.NET Core, HTTP-интерсепторы, @decorator в Python.

3. Паттерн «Стратегия» (Strategy)

Определение

Инкапсулирует *алгоритм* в объект, чтобы менять его *во время выполнения* и не тащить его сложность в остальную систему. Context держит поле по интерфейсу Strategy и делегирует ему работу; смена алгоритма = передать новый объект-стратегию.



Разбор: алгоритмы форматирования (Compositor)

Псевдоглиф Composition держит выбранный Compositor; при Insert дёргает его Compose. Быстрый алгоритм — пока пользователь печатает; медленный, но красивый (как в TeX, с учётом «цвета» текста) — при чистовой вёрстке. Алгоритм можно сменить на лету.

Когда брать. (1) Несколько взаимозаменяемых алгоритмов; (2) близкие классы, отличающиеся только поведением (вынести поведение в стратегии, классы склеить в Context); (3) один класс, но сложный многоэтапный алгоритм со вспомогательными данными; (4) много условных операторов (заменить полиморфизмом стратегии).

Грабли

Проектирование интерфейса — главное решение: данные *параметрами метода* (стратегия не знает про Context, но получает лишнее) или *передать сам Context* (гибко, но круговая зависимость и течёт инкапсуляция). **Стратегия по умолчанию** — иначе Context упадёт, если её забыли выставить. Стратегия часто *разделяема* (одна на много контекстов) ⇒ без собственного состояния (или «Приспособленец»). В C++ стратегия как *параметр шаблона* убирает оверхед vtable (так передают аллокаторы в STL), но только если её не менять в рантайме.

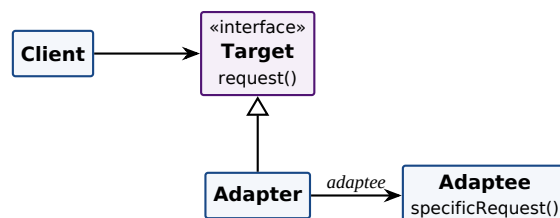
[+] Лямбды почти вытеснили «Стратегию»

Для стратегии с одним методом — лямбда (Comparator, sort(cmp), политика, колбэк); замыкания дают ей состояние. Поэтому в современных языках полноценный класс-стратегия нужен реже — лекция прямо отмечает, что лямбды делают паттерн «избыточным».

4. Паттерн «Адаптер» (Adapter)

Определение

Делает *несовместимый* интерфейс Adaptee пригодным для клиента, ожидающего Target. Adapter реализует Target и перенаправляет (возможно, нетривиально преобразуя) вызовы в Adaptee. *Отличие от декоратора*: там интерфейс совпадает, тут — *меняется*.



Разбор: чужая библиотека текста в графическом редакторе

Редактор ждёт Shape; сторонний TextView его не реализует. TextShape — адаптер: реализует Shape, делегирует всё TextView, плюс может добавить своё (CreateManipulator — ручка для перемещения фигуры).

Два вида:

Адаптер объекта (композиция)	держит Adaptee полем; гибче, предпочтителен
Адаптер класса (наследование)	наследуется от Adaptee + реализует Target; короче, но негибко; при Target-абстрактном классе нужно множественное наследование

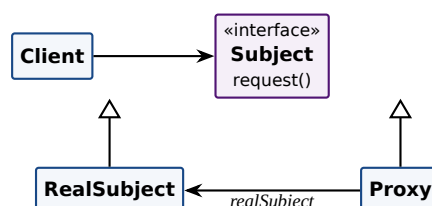
[+] Детали и где сегодня

Единственное осмысленное применение *private-наследования* C++ — адаптер класса (скрыть методы Adaptee). Сегодня адаптеры — это «порты и адаптеры» (гексагональная архитектура): домен зависит от *порта*-интерфейса, адаптеры подключают БД/очереди/HTTP; а также обёртки чужих SDK, Arrays.asList, потоковые адаптеры.

5. Паттерн «Заместитель» (Proxy)

Определение

Подставляет вместо реального объекта *суррогат* с *тем же интерфейсом*, чтобы контролировать доступ к нему. Proxy умеет добраться до реального объекта (или *создать* его при нужде) и сам обслуживает что может, остальное перенаправляет. *Отличие от декоратора*: прокси ссылается на *конкретный класс* и сам управляет его созданием (прокси на прокси не надевают).



Разбор: ленивая загрузка изображений (виртуальный прокси)

ImageProxy реализует интерфейс Graphic, хранит имя файла и *размер* (считан заранее), а ссылку на Image — изначально null. Форматирование получает размер сразу (можно сверстать страницы), а тяжёлая загрузка происходит только когда пользователь долистал до картинки. Большинство изображений в большом документе не грузятся никогда.

Виды: удалённый (объект на другой машине), виртуальный (ленивое создание тяжёлых), защитный (контроль доступа), «умный указатель» (подсчёт ссылок, блокировки, сору-on-write).

Грабли

Прокси обязан реализовать *весь* интерфейс (часто громоздкий). Если методы только перенаправляют — помогает => `realSubject.do()` (C#/F#) и *кодогенерация*. Если реального объекта ещё нет в памяти — наивное => `realSubject.do()` даст `NullReferenceException`; проверяйте наличие. В C++ `operator->` *цепляется* рекурсивно — на этом сделаны умные указатели.

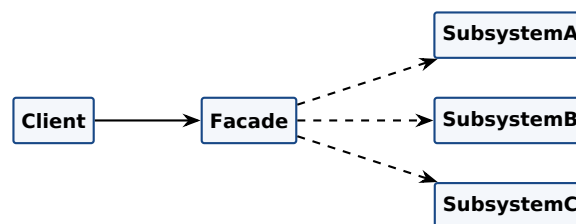
[+] Прокси — повсюду

Ленивые прокси ORM (Hibernate/JPA, EF) для @OneToMany; *динамические прокси* для AOP (Spring AOP: JDK Proxy/CGLIB) — транзакции, кеш, безопасность «поверх» бинов; клиенты gRPC/OpenAPI(Swagger), генерируемые из описания сервиса (WCF из лекции — *legacy*, на смену пришли gRPC и ASP.NET Core); Future/Promise/Task; shared_ptr/unique_ptr; sidecar service mesh (Envoy) — сетевой прокси на инфраструктурном уровне.

6. Паттерн «Фасад» (Facade)

Определение

Даёт *простой* интерфейс к большой и сложной подсистеме, скрывая её внутренности. Может и *ограничивать* доступ к внутренним классам, становясь *границей* подсистемы и её контрактом.



Разбор: фасад компилятора

Подсистема компиляции — лексер, парсер, узлы AST, анализаторы типов, оптимизаторы, генераторы кода, всё гибко конфигурируемо. Фасад `Compiler` с единственным `Compile()` сам собирает и запускает их с параметрами по умолчанию. Кому надо тоньше — лезет внутрь напрямую (фасад не запрещает).

Когда брать. Упростить переиспользование подсистемы; *отделить* подсистемы друг от друга (в многоуровневой архитектуре каждый уровень общается с соседним только через фасад). *Не* брать, если в фасаде получаются десятки *несвязных* методов.

Грабли: фасад как единственная точка входа

Сделать фасад *единственным* доступом сложно даже в современных языках. Приёмы: фасад — единственный `public`-класс пакета, остальные `internal` (но мешает, если подсистему саму делят на пакеты); соглашения об именах (`details` в C++, `internal` в Java) — *компилятор их не проверяет*, отсюда поломки сборки при импорте «не того» пакета.

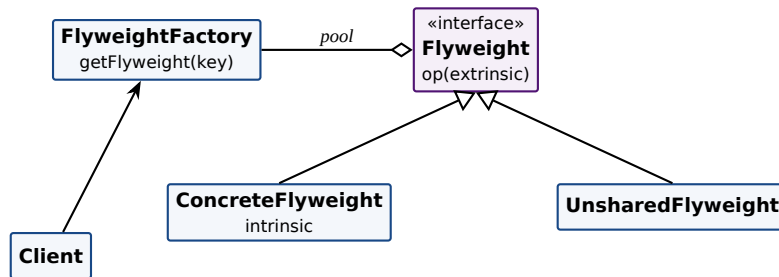
[+] Где сегодня

API Gateway и BFF (backend-for-frontend) перед микросервисами; SDK-клиенты библиотек; сервисный слой над репозиториями. **Абстрактный фасад** (интерфейс + фабрика) даёт замену подсистемы без знания конкретных классов.

7. Паттерн «Приспособленец» (Flyweight)

Определение

Позволяет вместить *много мелких объектов*, разделяя их: состояние делится на **внутреннее** (intrinsic — общее, хранится в приспособленце, немутабельно) и **внешнее** (extrinsic — зависит от контекста, хранится у клиента и *передаётся* в операции). Клиент берёт приспособленцев только из `FlyweightFactory` (пул), напрямую не создаёт.



Разбор: буквы как приспособленцы

Вместо объекта на каждую букву — пул по одному объекту на символ; в тексте — ссылки на пул. Шрифт/цвет (внешнее состояние) хранятся не у буквы, а у *промежутка* (run): обычно соседние буквы форматированы одинаково, форматирование выносится в style, а run держит лишь ссылку на стиль. При отрисовке внешнее состояние буквы быстро вычисляется по её run.

Когда брать (все условия сразу): много мелких объектов; есть осмысленное деление на внутреннее/ внешнее состояние; внешнее — вычислимо/компактно; идентичность не важна (нет сравнений по ссылке); семантика типа-значения; и *измеримый* выигрыш (паттерн сильно усложняет код).

Грабли

Внутреннее состояние — *немутабельно* (включая отсутствие ссылок на мутабельные объекты), иначе правка одним клиентом видна всем. Нужна *семантика значения*: переопределили equals — обязательно и hashCode (равные объекты ⇒ равные хеши). Классический сюрприз — кеш Integer в Java (−128..127): Integer.valueOf(10)==Integer.valueOf(10), но valueOf(150) != valueOf(150) — ровно из-за разделения приспособленцев и ссылочного ==.

[+] Где сегодня

Интернирование строк (Java/.NET), кеш Integer, пулы немутабельных value-объектов (record в Java/C#, value types). Паттерн редок — применять только при доказанном выигрыше по памяти.

Итоговый чек-лист.

- ☐ Не путаю обёртки по *назначению*: Адаптер (сменить интерфейс) / Декоратор (добавить поведение) / Заместитель (контроль доступа) / Фасад (упростить подсистему).
- ☐ «Компоновщик»: решил дилемму методов потомков (исключение / Null Object / getComposite); продумал кеш и его инвалидацию.
- ☐ Декоратор/Заместитель: Component мал и методы виртуальны; помню про null реального объекта в прокси.
- ☐ Стратегия с одним методом — рассмотрел лямбду; задал стратегию/поведение по умолчанию.
- ☐ Приспособленец — только при измеримом выигрыше; внутреннее состояние немутабельно; equals + hashCode; осторожно со ссылочным ==.

Рабочая тетрадь. Лекция 7

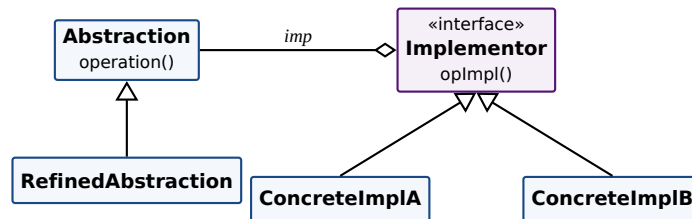
Порождающие шаблоны проектирования

Один раздел на каждый экзаменационный вопрос (в том же порядке), углублённо: назначение, структура, «когда брать», разбор примера, «грабли». Дополнения практиков помечены [+]. «Мост» — оставшийся структурный паттерн; остальные пять — порождающие (отвечают на вопрос «как гибко создавать объекты»).

1. Паттерн «Мост» (Bridge)

Определение

Разделяет две независимые иерархии — **абстракцию** (высокоуровневое, с чем работает клиент) и **реализацию** (Implementor — низкоуровневое, через что абстракция работает) — связывая их *композицией*. Фраза «отделение абстракции от реализации» осмысленна только вместе с диаграммой. Обе стороны — иерархии классов, и обе расширяемы.



Разбор: интерпретатор роботов

Слева — иерархия *логических сенсоров* («ИК-датчик расстояния», фильтрация/нормализация), справа — иерархия *связи* (Bluetooth/Wi-Fi/USB: упаковка пакета, передача, распаковка). Между ними мост — композиция. Раньше число классов было *произведением* (сенсоры × связь), стало *суммой*. Плата: чуть меньше гибкости (нет места под запрос, специфичный для пары сенсор-связь).

Когда брать — и чем отличается от соседей:

«Стратегия»	абстракция одна, реализации не образуют сложной иерархии
«Заместитель»	не надо повышать абстракцию, просто менять реализацию; интерфейсы <i>совпадают</i>
«Мост»	и абстракция, и реализация расширяются новыми подклассами

Как внедряется Implementor: абстракция создаёт сама (по параметрам/конфигу); создаёт реализацию по умолчанию + сеттер для подмены; передаётся в конструктор клиентом или IoC-контейнером; создаётся *фабрикой* (полностью отвязывает от реализации — её может вообще не быть при компиляции, как платформозависимых реализаций оконной системы).

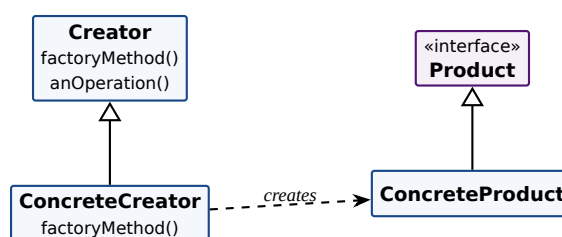
[+] PImpl и где «Мост» сегодня

PImpl (Pointer to Implementation) — C++-идиома, вырожденный «Мост»: вся private-часть выносится в класс в .cpp, в заголовке остаётся *один указатель*. Эффект: правка реализации перекомпилирует *один* файл (а не пол-проекта), сохраняется *бинарная совместимость* (размер класса не меняется). Так устроена почти вся Qt. *Архитектурные мосты*: JDBC (абстракция) над драйверами БД (реализации), SLF4J над бэкендами логирования, платформенные слои за DI.

2. Паттерн «Фабричный метод» (Factory Method)

Определение

Creator делает много содержательной работы, но *не знает* точный тип создаваемого Product: он объявляет виртуальный `factoryMethod()`, а подклассы (ConcreteCreator) переопределяют его, возвращая нужный ConcreteProduct (обычно в одну строку — вызов конструктора).



Разбор: эволюция «виртуального конструктора»

1. «Виртуальный конструктор» (Warrior с полем-прокси `p` и `if`-цепочкой в конструкторе) — *плохо*: нарушен SRP, лишняя косвенность, два набора полей, нельзя сделать абстрактным. 2. Статический `createWarrior(id)` — *приемлемо*: Warrior снова абстрактный. 3. Виртуальный `factoryMethod` вместо `if`-цепочки — *настоящий паттерн*. (GoF-пример: `Application.CreateDocument()` — фреймворк делает всё, ты переопределяешь создание своего `Document`.)

Когда брать. `Creator` делает основную работу, но часть делегирует продукту, который может делать её по-разному (FM может создавать стратегию, где `Creator = Context`). Удобный способ параметризовать объект другим объектом, не передавая его явно извне.

Грабли

`Creator` не обязан быть абстрактным (реализация по умолчанию как точка расширения — но пользователь может забыть переопределить и получить не ошибку, а сюрприз). **Нельзя** вызывать `factoryMethod` (и вообще любой виртуальный метод) в конструкторе `Creator`: подкласс ещё не проинициализирован $\Rightarrow$ труднонаходимые ошибки. Альтернативы: параметр шаблона (C#/C++, не Java — там нельзя создать объект типа-параметра); прототип (JS).

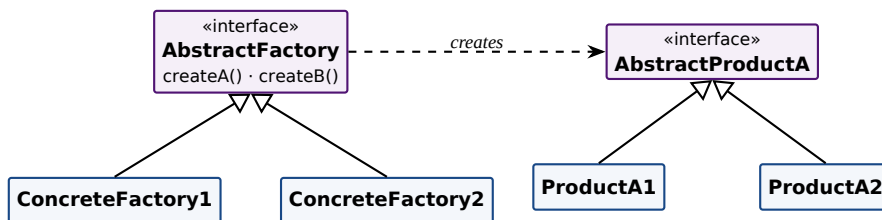
[+] Лямбда — лучший современный способ

В книге его нет (лямбд тогда не было): передать `Creator`-у функцию создания — `Supplier<Product>` (Java), `Func<Product>` (C#), просто `() => new ...`. Не нужен подкласс `Creator` на каждый продукт.

3. Паттерн «Абстрактная фабрика» (Abstract Factory)

Определение

Создаёт семейства связанных продуктов. `AbstractFactory` объявляет создание абстрактных продуктов; `ConcreteFactory` (по одной на семейство) создаёт конкретные. Клиент знает только абстрактные продукты и абстрактную фабрику; конкретных классов может не видеть вовсе (хоть подключаешь плагином). Сменить семейство = передать другую фабрику.



Разбор: стили оформления редактора

Элементы (`ScrollBar`, `Button`...) реализованы под каждый стиль (Windows/Linux). Вместо `new RoundedScrollBar` пишут `guiFactory->createScrollBar()`. Один `switch` при старте выбирает фабрику, дальше её раздают всем. Каждая фабрика — набор однострочных методов; гарантируется, что все элементы одного стиля (не перепутать).

Грабли асимметрии расширения

Добавить новый вид семейства (`Product3`) — легко (новая фабрика). Добавить новый тип продукта (`AbstractProductC`) — больно: метод в каждую фабрику + правка клиента. Сблэзн сделать один `createWidget(type)` вместо `createButton/createScrollBar` ломает типобезопасность — клиент не знает статический тип результата.

Фабричный метод vs Абстрактная фабрика: FM — метод внутри класса, который и так делает работу (создание — побочно). Абстрактная фабрика — выделенный класс, чья единственная цель — создавать (по сути набор фабричных методов). **[+]** DI-контейнеры (регистрация семейств реализаций), системы плагинов и тем; кросс-платформенные UI-тулкиты (look-and-feel в Java AWT/Swing).

4. Паттерн «Одиночка» (Singleton)

Определение

Гарантирует единственный экземпляр и глобальный доступ к нему: `private`-конструктор + `public-static instance()`, лениво создающий и возвращающий статическое поле. «Multiton» — реестр одиночек по ключу.



Реализация (любимый вопрос собеседований). Наивная ленивая — *гонка* (два потока создадут два объекта). Варианты: *eager* static-инициализация (потокобезопасно средой, но теряем ленивость); `synchronized getInstance` (безопасно, но синхронизация на *каждый* вызов); правильно — **double-checked locking** + `volatile`:

```
private static volatile Singleton instance;    // volatile is essential
public static Singleton getInstance() {
    Singleton local = instance;               // one volatile read
    if (local == null) synchronized (Singleton.class) {
        local = instance;
        if (local == null) instance = local = new Singleton();
    }
    return local;
}
```

Критика: «Одиночка» ближе к антипаттерну

Это *хитрая глобальная переменная*: мутабельный — запутывает поток данных; порядок инициализации скрыт (перестановка строк ломает систему). *Мешает тестам* — нельзя подменить mock-ом и внедрить зависимости. Нарушает SRP (жизненный цикл + работа). *Имеет привычку размножаться* — «единый» репозиторий вдруг требует второго экземпляра (для сравнения версий), и приходится больно рефакторить.

[+] Чем заменяют

IoC-контейнер со `scope = singleton` сам управляет жизненным циклом (ленивость, потокобезопасность), а перестаёт быть одиночкой — одной строкой конфигурации. **enum-одиночка** (J. Bloch, *Effective Java*) — потокобезопасен и устойчив к сериализации «из коробки»; либо *holder-идиома* (ленивый статический вложенный класс).

5. Паттерны «Ленивая инициализация» и «Пул объектов»

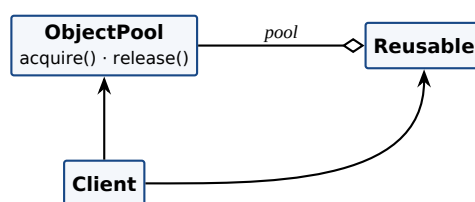
Ленивая инициализация (Lazy Initialization)

Создаёт объект *по требованию* (структурно как «Одиночка», но без цели уникальности и глобальности). Обёртка хранит способ создания (часто лямбду) и метод вроде `instance()`, который создаёт объект при первом обращении и запоминает. Те же проблемы многопоточности, что у «Одиночки» (`double-checked locking`).

Зачем. Не тратить ресурсы на то, что может не понадобиться; а кое-что без ленивости *невозможно* — бесконечные структуры данных. **[+]** `Lazy<T>` (.NET), `lazy` (F#), `by lazy` (Kotlin); в Haskell ленивость — основа модели вычислений; JIT-компиляция — тоже подвид ленивости.

Пул объектов (Object Pool)

Поддерживает набор *переиспользуемых* объектов, дорогих в создании, но нужных ненадолго. Класс-пул создаёт объекты при нужде, выдаёт по запросу (или сам использует, как `ThreadPool`) и переиспользует.



Разбор: пул потоков

Создание ОС-потока дорого (мегабайты под стек, структуры ядра, планировщик), и потоков нет смысла иметь больше, чем ядер. `ThreadPool` держит N потоков в цикле «взять задачу из очереди — выполнить — ждать»; `QueueUserWorkItem(task)` ставит задачу, возвращая `Task` для результата. $N \approx$ числу ядер.

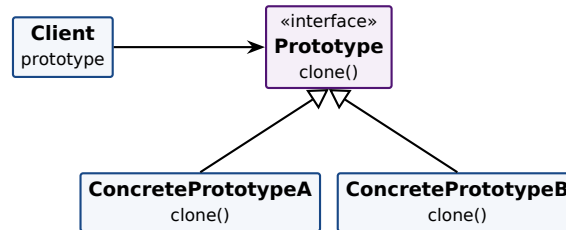
Грабли пула

Со *сборкой мусора* пул объектов часто *вредит*: выделение памяти и так почти бесплатно (сдвиг указателя), а пул держит ссылки $\Rightarrow$ нужен подсчёт использующих клиентов и синхронизация. Пул должен быть *маленьким* (десятки потоков — да, пул из 50000 соединений — нет). **[+]** `ExecutorService/ForkJoinPool` (Java), `Task/TPL` (.NET), пулы соединений (`HikariCP`). *Java 21: виртуальные потоки* (Project Loom) делают пул потоков под блокирующий ввод-вывод во многом ненужным.

6. Паттерн «Прототип» (Prototype)

Определение

Объявляет метод `Clone()` и предлагает порождать объекты копированием существующих. `Clone()` виртуальный $\Rightarrow$ создаёт копию того класса, у которого вызван — клиент про тип не знает (ещё один «виртуальный конструктор»).



Разбор: палитра нотного редактора

В палитре — *прототипы* нот разной длительности. По клику `GraphicTool` вызывает `Clone()` выбранного прототипа и тащит копию на нотный стан. Новый вид ноты — просто новый прототип в палитре, без `switch` и без фабрик на каждый тип.

Грабли: реализация `Clone()`

Глубокое vs мелкое копирование: глубокое (весь граф по ссылкам) — то, что ждут люди; но при *круговых ссылках* нужен честный обход с запоминанием скопированного, и оно медленное. Мелкое годится, только если структура немутабельна. *Хак* — сериализовать и десериализовать (просто, но медленно; следить за совпадающими `id` у оригинала и копии). `Clone()` без параметров — у каждого прототипа свой набор параметров инициализации; объединять их в базовом классе хрупко.

[+] Где сегодня

Реестр прототипов (часто «Одиночка»); прототипы — основа модели объектов JavaScript. Современные аналоги копирования: `with`-выражения у `record` в C#, `copy()` у `data`-классов Kotlin, копирующие конструкторы, `structured clone`.

Итоговый чек-лист.

- ☐ «Мост» — когда обе иерархии (абстракция и реализация) растут; одна реализация и без иерархии $\Rightarrow$ «Стратегия».
- ☐ Фабричный метод — `Creator` делает работу и делегирует создание; не зову `factoryMethod` из конструктора; рассмотрел лямбду.
- ☐ Абстрактная фабрика — выделенный класс для *семейств* продуктов; помню про дорогое добавление нового *типа* продукта.
- ☐ «Одиночка» — применяю с осторожностью (тесты, SRP, скрытое глобальное состояние); по возможности — `scope` контейнера, `enum`-одиночка.
- ☐ `Lazy` — создание по требованию; `Object Pool` — только для дорогих и переиспользуемых, маленький, помню про вред пула при GC.
- ☐ Прототип — по умолчанию *глубокое* копирование; `Clone()` без параметров.

Рабочая тетрадь. Лекция 8

Поведенческие шаблоны проектирования

Один раздел на каждый экзаменационный вопрос (в том же порядке), углублённо: назначение, структура, «когда брать», разбор примера, «грабли». Дополнения практиков помечены [+]. «Строитель» — оставшийся порождающий паттерн; остальные десять — поведенческие.

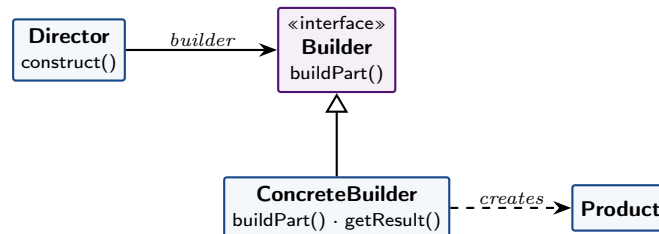
Ориентир: какие поведенческие паттерны путают

Делегируют действие объекту: «Стратегия» (сменный алгоритм — меняет *клиент*); «Состояние» (само переключается); «Команда» (действие как объект — очередь/отмена); «Шаблонный метод» (скелет в предке, шаги в потомках — *наследование*, а не композиция). **Развязывают коммуникацию:** «Наблюдатель» (субъект → многим напрямую); «Посредник» (всё через центральный хаб). **Работают по дереву:** «Посетитель» (много операций — *вне узлов*); «Интерпретатор» (мало простых операций — *в узлах*).

1. Паттерн «Строитель» (Builder)

Определение

Director (распорядитель) обходит исходные данные и *по частям* скармливает их **Builder**-у, который пошагово строит **Product**. **getResult()** не входит в интерфейс **Builder** (разные строители делают несвязанные продукты; **Director** про продукт не знает — результат забирает клиент, создавший конкретный **Builder**).



Разбор: конвертер форматов и StringBuilder

RTFReader (**Director**) разбирает .rtf и шлёт элементы конвертеру (**Builder**): **ASCIIConverter** добавляет символы, теги форматирования игнорирует. **StringBuilder** — тот же паттерн: наивная конкатенация в цикле *квадратична* (каждый + копирует обе строки), а билдер копит строки и делает *одно* выделение памяти в конце.

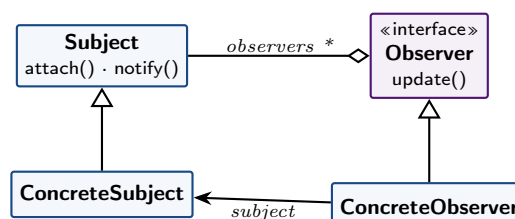
[+] Главное применение — «продвинутый конструктор»

Чаще всего **Builder** используют не для долгой стройки, а как конструктор с *именованными* параметрами, *значениями по умолчанию*, валидацией и *выбором типа в рантайме*. Пример — `Guava NetworkBuilder.directed().allowsParallelEdges(true)....build()`: по параметрам выбирается реализация графа, клиент про неё не знает. Сегодня: *fluent-билдеры*, **@Builder** (Lombok). *Правило*: путаетесь в нескольких конструкторах класса — заведите **Builder**.

2. Паттерн «Наблюдатель» (Observer)

Определение

Subject генерирует события: хранит список наблюдателей, даёт подписаться/отписаться и нотифицирует их. **Observer** — простой интерфейс (обычно один метод). Вся реакция — в **ConcreteObserver**, который может знать свой **Subject** и запрашивать его состояние. Разворачивает управление: генератор событий никого явно не зовёт, просто сообщает, что событие произошло.



Разбор: данные и несколько представлений

Данные в **Subject**; представления (таблица, столбчатая/круговая диаграмма) — наблюдатели. Кто-то меняет данные → **Subject** обходит наблюдателей с «обновись» → каждый пересчитывает свою часть. Связность низкая: **Subject** не знает деталей видов.

Габли

Утечки памяти: субъект держит ссылки на наблюдателей — забыл отписаться ⇒ утечка (в C++ — ещё и segfault). **Нотифицировать только после полного обновления** состояния — при наследовании легко нарушить (потомок дёрнул `Notify` в середине). **Кто иницирует:** при серии мелких правок лучше публичный `notify()`, вызываемый *после* пакета. **push vs pull:** слать данные с событием или дать наблюдателю самому спросить (приём `EventArgs` — новые данные = новый класс-наследник).

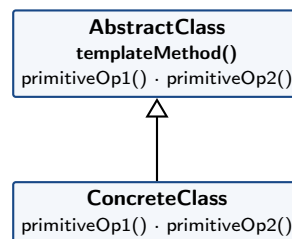
[+] Чаще всего уже встроено

`event` в C#, сигналы/слоты в Qt, `listener`-интерфейсы в Java. Идентификация отправителя: первый аргумент `sender` (.NET), `sender()` (Qt). Развитие — реактивные потоки: Rx.NET / RxJava (события как бесконечная коллекция с `Map/Filter`). Тяжёлая маршрутизация (RabbitMQ, Kafka) — уже не паттерн, а событийный архитектурный стиль.

3. Паттерн «Шаблонный метод» (Template Method)

Определение

AbstractClass задаёт *невиртуальный* `TemplateMethod()` — скелет алгоритма в терминах абстрактных примитивных операций; потомки переопределяют операции, но *не могут* изменить скелет. По сути развитие фабричного метода (там абстрактной была только операция создания).



Разбор: открытие документа

`OpenDocument()` (шаблонный метод): `CanOpenDocument` → `DoCreateDocument` (это фабричный метод!) → `AboutToOpenDocument` (hook по умолчанию пуст) → `Open/DoRead`. Скелет в предке, конкретика — в потомках.

Габли

Виртуальные операции имеют пустую реализацию по умолчанию (потомок может забыть переопределить); *чисто виртуальными* делают то, что переопределить *обязательно*. Примитивы — **protected** (или **private-virtual** в C++); сам шаблонный метод обычно неvirtуальный (давать переопределять его целиком — «два способа решить одну задачу», плохо). Единый префикс имён (**Do/On/Handle**); чем меньше чисто виртуальных, тем понятнее.

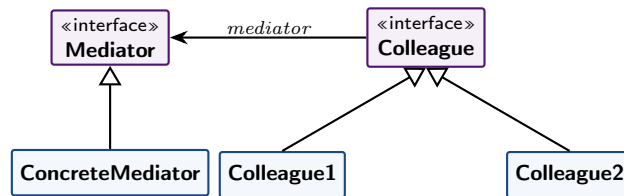
[+] Принцип Голливуда и vs Стратегия

«Don't call us, we'll call you»: фреймворк зовёт ваши хуки (lifecycle: `setUp/tearDown` в JUnit, методы жизненного цикла React, сервлеты, `OnPaint/WndProc` в Windows Forms). Vs Стратегия: Шаблонный метод варьирует шаги наследованием (compile-time), Стратегия — композицией (runtime).

4. Паттерн «Посредник» (Mediator)

Определение

Выделенный объект, отвечающий за *коммуникацию* между «коллегами»: коллеги знают только его и шлют события только ему, он рассылает дальше. Превращает *квадратичную* зависимость связей (все знают всех) в *линейную*. Абстрактный медиатор чаще не нужен — коллеги общаются с конкретным.



Разбор: окно выбора шрифта

Превью зависит от всех; для гарнитуры доступны не все начертания; поле поиска и список синхронизированы. Вместо «все дёргают всех» — `FontDialogDirector`: коллега сообщил об изменении ⇒ медиатор понял, кто изменился, и сообщил *только тем, кому это интересно*.

Грабли: God Object

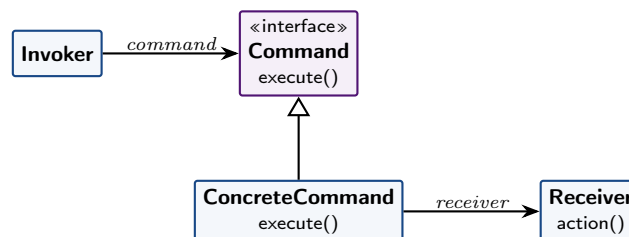
Медиатор знает про все взаимодействия — велик соблазн навесить на него диспетчеризацию, конвертацию, бизнес-логику ⇒ несопровождаемый God Object. *Только* организация общения, *никакой* бизнес-логики.

Vs Наблюдатель. Наблюдатель — субъект напрямую рассылает многим; Посредник — *центральный хаб* между равноправными коллегами. Обмен можно строить на «Наблюдателе» (коллеги дают события, медиатор подписан — тогда коллеги могут быть библиотечными) или наоборот (медиатор даёт события). **[+]** Шина событий, библиотеки in-process медиатора (например, MediatR в .NET).

5. Паттерн «Команда» (Command)

Определение

Заворачивает действие в объект с методом `Execute()`. `Client` создаёт `Receiver` (бизнес-логика), `Invoker` (инициатор) и `Command` (даёт ей `Receiver`). Один объект команды раздаётся меню, кнопке, хоткею — инициаторы не знают, что команда делает (видят интерфейс с одним методом). По сути «Стратегия» для `Invoker`-а.



Разбор: «умность» команды и составные команды

`PasteCommand` *глупая* — зовёт `Document.Paste()`. `OpenDocumentCommand` *оркеструет* (спрашивает имя файла, создаёт, регистрирует, открывает), но бизнес-логику не реализует. *Составная* команда («Компоновщик») — макрос: `Enter` в IDE = перевод строки + отступ, отменяются по отдельности. «Глупость-умность» — непрерывная шкала.

Undo/Redo

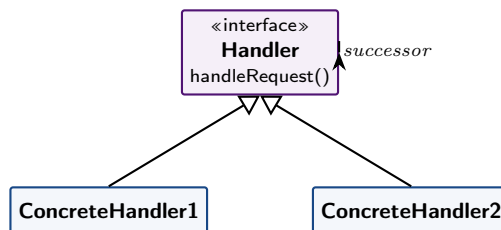
Два стека. Первое исполнение: `Execute()` + команда на `Undo`-стек. Отмена: снять с `Undo`, `Undo()`, переложить на `Redo`. Повтор: снять с `Redo`, `Execute()`, вернуть на `Undo`. Новая команда *очищает* `Redo`. Стек держит `Controller` (он же исполняет). `Undo()` хранит либо всё состояние, либо *дельту*. Мутабельную команду перед исполнением *копируют* («Прототип»); хорошо сочетается с «Хранителем» (команда — `Caretaker`).

[+] `QAction` (глупая команда в Qt); команды в CQRS; очереди задач/джобов; единое место для логирования и валидации всего пользовательского ввода.

6. Паттерн «Цепочка ответственности» (Chain of Responsibility)

Определение

Запрос идёт по *цепочке* обработчиков, пока кто-то его не обработает. Базовый класс хранит ссылку на преемника и по умолчанию *передаёт дальше*; потомки переопределяют `HandleRequest()`: могут — обрабатывают, нет — зовут предка. Развязывает отправителя и получателя; цепочку можно менять динамически (в т.ч. плагинами).



Разбор: контекстная справка

Клик по элементу: есть справка — показали; нет — спросили у объемлющего элемента, и так вверх по дереву. Ни у кого нет — никто ничего не показал (это нормально).

Габри

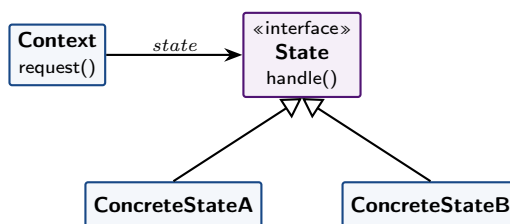
Цепочка обычно уже есть (дерево контролов, список обработчиков) — ссылку «преемник» делать не надо. **Несколько видов запросов** — не плодить *Handle-это/Handle-то* (новый запрос ломает всю иерархию), а представить запрос *объектом* `BaseRequest` с наследниками (как `EventArgs` в .NET): обработчик кастует и решает. *Нет гарантии*, что запрос вообще обработают.

[+] Распространение исключений (`throw` — клиент, `catch` — обработчики); всплытие событий (Qt, DOM); *middleware*-конвейеры (ASP.NET Core, Express), сервлет-фильтры.

7. Паттерн «Состояние» (State)

Определение

`Context` делегирует работу объекту `State`; поведение меняется со сменой состояния, *незаметно* для клиента. Замена *code smell* «тэг типа» (переменная-состояние + `switch`) на иерархию состояний. Имитирует «смену класса в рантайме».



Разбор: TCP-соединение

`TcpConnection` с состояниями `Established/Listen/Closed` делегирует `Open/Close/Acknowledge` объекту-состоянию; `Open()` на открытом соединении ничего не делает — потому что это метод состояния `Established`.

Vs Стратегия. Внешне одинаково (контекст делегирует объекту), но «Состояние» *само* переключается (паттерн определяет, как), а «Стратегию» меняет *клиент*. То есть Состояние = специализированная Стратегия + автопереключение.

Габри

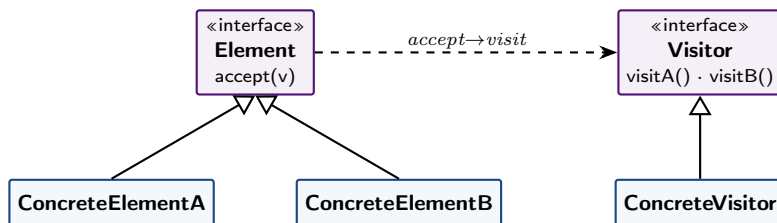
Переходы: в `Context` (центрально, можно таблицей переходов, но трудно действие при переходе) или в самих `State` (гибче, но логика размазана). **Создание/удаление** состояний: мало — создать раз и хранить; много — создавать при переходе (осторожно с `delete this`). Разделяемые состояния без данных — «Приспособленец»/«Пул».

[+] Идеален для ручной реализации конечных автоматов (ср. Лекцию 5: `switch` / таблица / паттерн «Состояние»); библиотеки `XState`, `Stateless`. Для скорости (лексеры) чаще *таблицы переходов* — нет оверхеда виртуальных вызовов.

8. Паттерн «Посетитель» (Visitor)

Определение

Выносит *операции* из структуры объектов в отдельные классы. В узлах — одна операция `Accept(visitor)`; в `Visitor` — по методу `Visit` на каждый тип узла; операция реализуется в конкретных посетителях. Механизм — **двойная диспетчеризация**: `Ассепт` (по типу узла) зовёт `Visit` (по типу посетителя).



Разбор: операции над AST компилятора

Типов узлов — десятки-сотни (по числу продукций грамматики). Реализовывать в каждом узле проверку типов, печать, кодогенерацию и обновлять всю иерархию на каждую новую операцию — мучительно. Вместо этого: один **Accept** в узлах + посетители **TypeChecker**, **Printer**, **CodeGen**.

Габли: проблема выражения

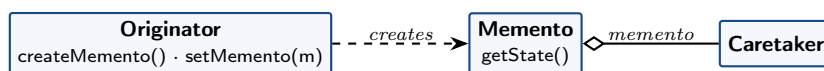
Новую *операцию* добавить легко (новый **Visitor**), новый *тип узла* — трудно (править все посетители). **Visitor** *нарушает инкапсуляцию* (узлы открывают данные). **VisitClassA()** (имя с типом) *читабельнее* перегруженного **Visit()**. Обходом управляет структура, элемент или сам посетитель (последнее — когда обход зависит от результата, напр. не идти в правый операнд **&&**).

[+] Компиляторы; современная альтернатива — *сопоставление с образцом* по запечатанным типам (**sealed** + **switch** в Java 21, C#, Kotlin, Scala). **Visitor** может копить состояние — работает как **map/fold** (преобразование деревьев).

9. Паттерн «Хранитель» (Memento)

Определение

Сохраняет и восстанавливает состояние объекта, *не раскрывая* его деталей посторонним. **Originator** умеет сделать слепок и восстановиться; **Memento** — сам слепок; **Caretaker** хранит слепок, но *внутри не лезет*. **Memento** имеет «широкий» интерфейс для **Originator** и «узкий» для остальных.



Разбор: отмена в векторном редакторе

Откат перемещения фигуры наивно ломает связи (связь решила, что фигуру просто подвинули). **Memento** прячет внутри координаты и фигуры, и связей: команда (**Caretaker**) получает слепок, а при **Undo()** отдаёт его фигуру/сцене — те восстанавливаются сами, команде знать детали не надо.

Габли

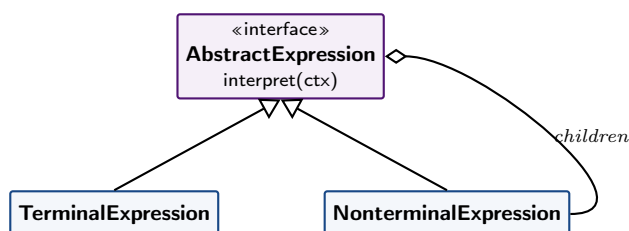
Нужна *языковая поддержка* узкого/широкого интерфейса: **friend** (C++), вложенный класс (C#), статический вложенный (Java); часто **CreateMemento()** возвращает абстрактный **Object**, а **SetMemento()** кастует. Слепок может быть *дельтой* (как системы контроля версий) — экономно, но хлопотно.

[+] Связка с «Командой» для **undo/redo**; сериализация для слепка; **event sourcing**; немутабельные снимки состояния.

10. Паттерн «Интерпретатор» (Interpreter)

Определение

Лёгкая замена полноценному интерпретатору/компилятору: *абстрактное синтаксическое дерево* (по сути «Компоновщик») с операцией **interpret(context)**, которую каждый узел реализует сам.



Разбор: регулярное выражение

Разобранное регулярное выражение хранится деревом; `interpret()` проверяет строку на соответствие. Дерево строит либо сам программист в коде (`Add(salary.Data(), bonus.Data())`), либо Builder.

Когда брать. Грамматика *простая*, операций мало и они несложные, скорость не критична — маленькие встроенные DSL. *10-е правило Гринспена*: сложные программы обрастают своим кривым мини-языком — потребность в индустрии высока. «Спецификация» — это «Интерпретатор» для булевых предикатов.

Vs Посетитель — главное

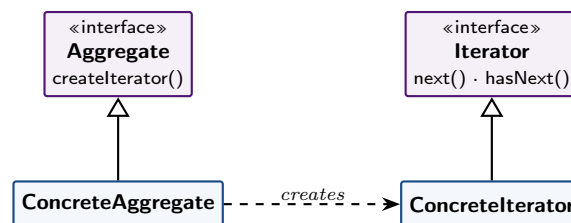
Если операций *много* или они *сложные* (даже одна) — берите «Посетитель». «Интерпретатор» — только для простой грамматики и немногих простых операций. Листья — кандидаты в «Приспособленцы», но если дошло до этого, дела плохи.

[+] Внутренние vs внешние DSL; деревья выражений (`LINQ Expression<T>`); парсер-комбинаторы.

11. Паттерн «Итератор» (Iterator)

Определение

Инкапсулирует *способ обхода* однородной коллекции, скрывая порядок и метод. Клиент просит у `Aggregate` итератор и работает через интерфейс `Iterator`; одну коллекцию можно обходить многими итераторами одновременно и разными способами (BFS/DFS/preorder...), и даже генерировать данные «на ходу» (бесконечные последовательности).



Два стиля интерфейса:

```
// Java style
interface Iterator<E> {
    boolean hasNext();
    E next();
    void remove();
}

// .NET / C++ style
interface IEnumerator<T> {
    bool MoveNext();
    T Current { get; }
    void Reset();
}
```

Внешний vs внутренний. *Внешний* — клиент сам решает, когда шагнуть (гибко: согласованный обход двух коллекций в merge-sort). *Внутренний* — коллекции передают действие, она сама себя обходит (`ForEach(t => ...)`); проще, когда надо просто посетить всех.

Габли

Курсор — проще итератора (логика обхода в коллекции), но надо хранить ссылку на коллекцию. **Устойчивые vs неустойчивые:** большинство — *неустойчивые* (fail-fast): при изменении коллекции бросают исключение. Реализуется *версией* коллекции: правка ++version; итератор сверяет версию (напр. .NET бросает на `MoveNext`, но `Current` гарантирует всегда). Устойчивые требуют нотификации об изменениях («Наблюдатель») и потому редки.

[+] Прячется за синтаксическим сахаром: `foreach/for-each`, генераторы (`yield`), потоки и LINQ; интерфейс «у чего есть итератор» — `Iterable / IEnumerable<T>`.

Итоговый чек-лист.

- ☐ Не путаю семейства: делегирование действия (Стратегия/Состояние/Команда/Шаблонный метод), развязка коммуникации (Наблюдатель/Посредник), обход дерева (Посетитель/Интерпретатор).
- ☐ Наблюдатель — отписываюсь (утечки), нотифицирую *после* полного обновления; Посредник — без бизнес-логики (God Object).
- ☐ Команда — undo/redo на двух стеках, мутабельную копирую; Состояние само переключается (в отличие от Стратегии).
- ☐ Посетитель — легко новые операции, трудно новые типы узлов (проблема выражения); современно — pattern matching по sealed-типам.

- Интерпретатор — только простая грамматика и мало операций, иначе Посетитель; Итератор — fail-fast по версии коллекции.

Лекция 9. Архитектурные стили

Рабочая тетрадь

Лекция целиком «архитектурная»: от понятия *стиля* и трёх классических шаблонов (*State-Logic-Display*, *MVC*, *Sense-Compute-Control*) через семейство *уровневых стилей* (*слоистый* → *гексагональный* → *луковый* → *чистый*) к *стилям потока данных*, *доске (Blackboard)*, *событийным схемам* и *peer-to-peer*. Все вопросы билета раскрыты лекцией. Метка **[+]** помечает то, чего в лекции нет (современные инструменты и уточнения).

1. Понятие архитектурного стиля, трёхзвенная архитектура

Определение: архитектурный стиль

Архитектурный стиль — набор проектных решений, которые: (1) применимы в выбранном контексте разработки; (2) задают ограничения на архитектурные решения, специфичные для систем этого контекста; (3) приводят к желаемым *положительным* качествам системы. Стиль действует на уровне *подсистем* или *всей системы*, а не отдельных классов, и предпочтителен в своей предметной области.

Стиль фиксируется тремя соображениями: **набор элементов** (компоненты, соединители, элементы данных), **правила соединения** (топологические ограничения — кто с кем может связываться; это и есть *суть* стиля) и **семантика** элементов (что им позволено — например, фильтру можно запретить иметь состояние, а каналу — преобразовывать данные).

Где стиль стоит на лестнице абстракций. Главная путаница лекции — различать четыре уровня. Ориентир:

Уровень	Масштаб / привязка к предметной области	Что диктует	Примеры
Паттерн проектирования	Класс / маленькая подсистема; вне предметной области	Тактическое решение	Наблюдатель, Команда, Мост
Архитектурный шаблон	Подсистема; слабая привязка к области	Типовое решение типовой задачи	MVC, Sense-Compute-Control
Архитектурный стиль	Подсистема / вся система; предпочтительная область	Топологические ограничения структуры	Слоистый, Pipes & Filters, P2P
Reference-архитектура	Вся система; жёсткая привязка к области (часто стандарт)	Готовый каркас целой области	SAVOIR (ESA), CVRIA

Граница между уровнями условна: MVC иногда зовут паттерном, трёхзвенку — то стилем, то шаблоном. Важно не название, а где решение действует и насколько оно привязано к области.

Трёхзвенная архитектура (State-Logic-Display)

Определение: трёхзвенная архитектура

State-Logic-Display («три звена») разделяет приложение на три части: **Display** — представление и взаимодействие с пользователем (и больше ничего); **Logic** — бизнес-логика; **State** — слой хранения данных. Чёткое разделение делает каждое звено управляемым и переиспользуемым.



Главные ограничения: клиент ничего не знает о БД и не лезет в неё напрямую; логика не хранит данные и не занимается UI; БД не делает с данными ничего нетривиального.

Пример: веб-приложение и многопользовательская игра

Подавляющее большинство веб-приложений и информационных систем — трёхзвёнки. В сетевой игре клиент может быть очень сложным (рендеринг, ввод), но *не вправе* реализовывать игровую логику и хранить что-либо, кроме данных взаимодействия с пользователем (логин, настройки). Масштабирование почти бесплатно: много клиентов

→ один (или несколько) серверов логики → одна (возможно распределённая) БД.

Делать	Не делать
Менять UI (десктоп→моб.), не трогая логику	Дублировать бизнес-правила в клиенте
Держать логику без знаний о конкретном UI	Позволять клиенту читать/писать БД напрямую
Масштабировать звенья независимо	Класть нетривиальную обработку в БД

Осторожно

«Толстый» клиент — не повод нарушать слои: рендеринг и ввод можно, бизнес-логику и хранение — нельзя. Иначе теряется главная выгода — независимая замена и масштабирование звеньев.

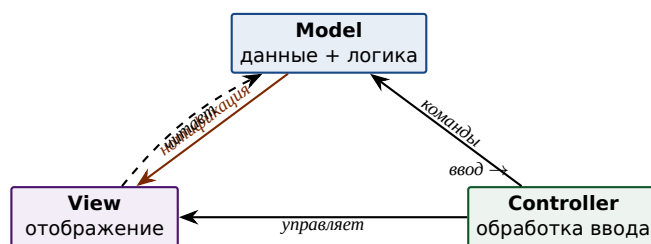
Приём: трёхзвёнка сегодня [+]

Современная веб-трёхзвёнка — это SPA-фронтенд (React/Vue) + API-бэкенд (логика) + управляемая БД; вариант **BFF** (Backend-for-Frontend) даёт по выделенному тонкому бэкенду на каждый тип клиента, не размывая ядро логики. В serverless звено логики дробится на функции, но ограничения остаются те же.

2. Шаблоны Model-View-Controller, Sense-Compute-Control

Определение: Model-View-Controller

MVC — архитектурный шаблон с тремя ролями. **View** только отображает данные. **Controller** принимает пользовательский ввод и обеспечивает логику взаимодействия, при необходимости управляя View. **Model** хранит данные (и обычно бизнес-логику), выполняет команды и сама *рассылает нотификации* об изменениях.



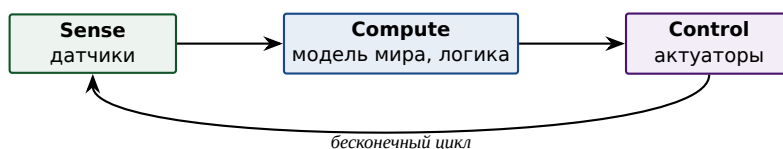
Ограничения: View только читает из Model; команды Model отдаёт только Controller; сигнал об обновлении Model рассылает сама ⇒ несколько View синхронно обновляются. Controller — единственная точка прохождения команд пользователя.

Пример: Undo/Redo через «Команду»

Controller — естественное место для Undo/Redo: каждое действие пользователя оборачивается объектом-командой (паттерн «Команда») и кладётся в стек. «Отменить» — вызвать `undo()` верхней команды. Внутри MVC обычно живут «Наблюдатель» (нотификации Model→View) и «Команда» (Controller). Поэтому MVC — *больше*, чем паттерн.

Определение: Sense-Compute-Control

Sense-Compute-Control — шаблон для робототехники и «умных домов»: система работает в бесконечном цикле из трёх фаз — **Sense** (снять показания датчиков), **Compute** (вычислить управляющее воздействие), **Control** (послать воздействие на актуаторы).



Простой робот (едет вдоль стенки по датчику расстояния) больше ничего не содержит; в общем случае Compute может включать построение и обновление модели мира (например, SLAM) и сложную логику поведения. SCC задаёт самый внешний каркас; для сложных систем это *только* каркас — потому это шаблон, а не стиль.

	MVC	Трёхзвёнка (State-Logic-Display)
Хранение данных	Не специфицирует (применим и без хранения)	Выделенное звено State (БД)
Обработка ввода	Требует выделенного Controller	Делает Display вместе с выводом
Типичное место	Десктоп с развитым UI (часто лишь вершина айсберга)	Веб/ИС, многопольз. игры

Осторожно

MVC — обычно лишь «вершина айсберга», отвечающая за общение с пользователем; настоящая архитектура начинается под Model. Не превращайте View в место бизнес-логики и не делайте «умную» View, которая сама командует Model в обход Controller.

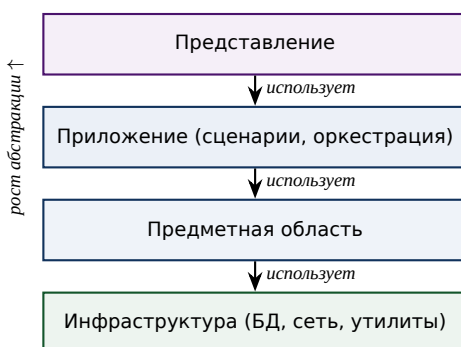
Приём: наследники и современное железо [+]

Семейство MVC: **MVP** (Presenter полностью управляет пассивной View) и **MVVM** (ViewModel + двусторонний биндинг) — основа SwiftUI/WPF/Android. Для Sense-Compute-Control де-факто стандарт сегодня — **ROS 2**: ROS 1 снят с поддержки в мае 2025, ROS 2 использует децентрализованное обнаружение узлов и pub-sub поверх **DDS** (без единого «мастера»), LTS-выпуск Jazzy поддерживается до 2029. Узлы ROS общаются по топикам именно по схеме «издатель-подписчик».

3. Слоистый стиль, «Клиент-сервер»

Определение: слоистый стиль

Система делится на **слои**: каждый пользуется слоями ниже и предоставляет интерфейс слоям выше, ничего о них не зная. Это «многоуровневый клиент-сервер»: слой — клиент нижних и сервер верхних. Компоненты — сами слои (внутри сколь угодно сложные), соединители — протоколы/программные интерфейсы. В *строгом* варианте слой общается только с непосредственно нижним.



Плюсы: постепенный рост абстракции (программируешь в терминах слоя, не зная реализации ниже — сокеты, OpenGL / Vulkan под движком); лёгкое сопровождение (изменение влияет только на слои выше, легко отследить); взаимозаменяемость реализаций по общему интерфейсу (Ethernet ↔ WiFi, драйверы).

Пример: сетевой стек и слои DDD

Сетевые стеки (OSI, TCP/IP) — слои протоколов; ОС, многие бизнес-приложения — тоже. Классический пример — **Domain-Driven Design**: уровни *представления* (UI, без логики), *приложения* (сценарии, координация), *предметной области* (модель и логика, переиспользуемая), *инфраструктурный* (хранение, сеть, библиотеки; формы UI могут наследоваться от классов инфраструктуры — «наследование снизу вверх»). Внутри домена тоже бывают под-уровни: принятия решений, операционный, ресурсный.

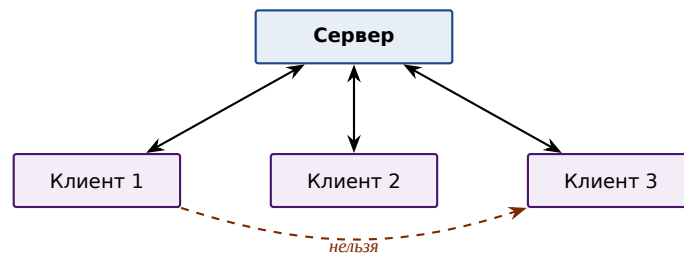
Осторожно: когда слои не годятся

Если взаимодействие не упорядочивается по уровням — не натягивайте стиль. Пример из первой лекции (ПО осциллографа): данные шли *вверх*, команды — *вниз*, и все части всё равно знали друг о друге. Второй риск — *производительность*: «прокидывающие» методы и лишние уровни дают оверхед на вызовы.

«Клиент-сервер»

Определение: клиент-сервер

Вырожденный случай уровневой архитектуры — *всего два уровня*. Компоненты — клиенты и сервер, соединители — (обычно) сетевые протоколы. Ограничения: клиенты не общаются друг с другом и только с сервером; сервер ничего не знает о клиентах (даже их количество), пока те не начнут взаимодействие.



Серверов может быть много (масштабируемость), ценой синхронизации их состояния. Стиль типичен для несложных веб- и мобильных приложений и, неожиданно, для ОС: графика Linux — клиент-серверная (оконный сервер + приложения, шлюющие запросы на отрисовку).

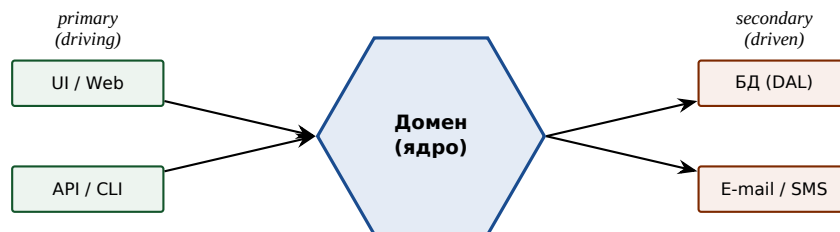
Приём: уровневость «по умолчанию» [+]

Уровневая архитектура считается стандартом (профстандарт «архитектор» подразумевает её как архитектуру по умолчанию). Дальнейшие уточнения слоистого стиля — гексагональная, луковая и чистая архитектуры (вопросы 4–5): они выносят инфраструктуру во «внешний мир» и инвертируют зависимости. Современная enterprise-практика — DDD поверх уровней.

4. Гексагональная архитектура, луковая архитектура

Определение: гексагональная архитектура (порты и адаптеры)

Развитие уровней: инфраструктуру тоже считают внешним миром. В центре — **уровень предметной области** (ядро); вокруг — **порты** (предоставляемые и потребляемые интерфейсы) и **адаптеры**, преобразующие данные внешнего мира в чистую доменную модель. Реализуется через Dependency Inversion + Dependency Injection.



Primary-порты (driving) — инициируют действие (запросы пользователя, админа, API); **secondary-порты** (driven) — служебная функциональность (нотификации, поиск, хранение), ими управляет ядро. Ядро не знает, куда уходят нотификации: один и тот же интерфейс может реализовать адаптер почты, SMS или Slack.

Пример: бэкэнд веб-приложения

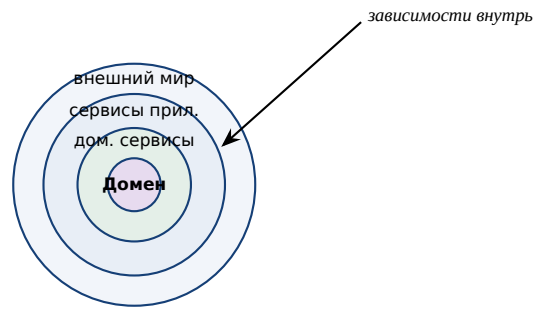
Primary-порты соответствуют случаям использования (пользователь, админ, API); за ними — «средства доставки» (HTTP, консоль), о которых порты ничего не знают. Secondary-порты: интерфейс Notifier, реализованный адаптером, который дёргает библиотеку отправки писем. Класс домена получает реализацию в конструктор (DI), а наружу отдаёт себя или адаптер, реализующий обещанный интерфейс.

Плюсы	Минусы
Изоляция ядра от «механизмов доставки» (как любая уровневая)	Часто «из пушки по воробьям» для одного простого приложения
Изоляция вспомогательных механизмов (чего уровневая не умеет)	Размыта граница «платформа vs внешняя зависимость» (Qt, boost, Guava?)
Лёгкое тестирование: заглушки во все внешние зависимости	Слишком много свободы: стиль мало что специфицирует
Адаптеры валидируют/конвертируют данные ⇒ ядро доверяет входу	

Луковая архитектура

Определение: луковая архитектура (Onion)

Уточнение гексагональной с *внутренними* уровнями ядра. Центр — **доменная модель** (сущности и логика); над ней — **доменные сервисы** (работа с несколькими сущностями); над ними — **сервисы приложений** (оркестрация, случаи использования); снаружи — внешний мир (UI, инфраструктура, хранение). Правило: внутренние слои ничего не знают о внешних; уровневость *нестрогая* (можно обращаться к любому слою ниже).



Структура слоёв примерно соответствует DDD; отличия: DDD разделяет инфраструктуру и приложение (в «луке» они слиты), но доменные сервисы в DDD — полноценные жители домена, а тут вынесены в отдельный слой. *Репозитории* логичнее держать снаружи (как в гексагональной), хотя в оригинале они были частью доменной модели.

Осторожно

Не путайте «есть ядро, есть адаптеры» с готовой архитектурой: и гексагональная, и луковая оставляют *слишком много свободы*. И не платите за изоляцию там, где технологии менять не будете («это всегда не планируется...») и предметная область проста.

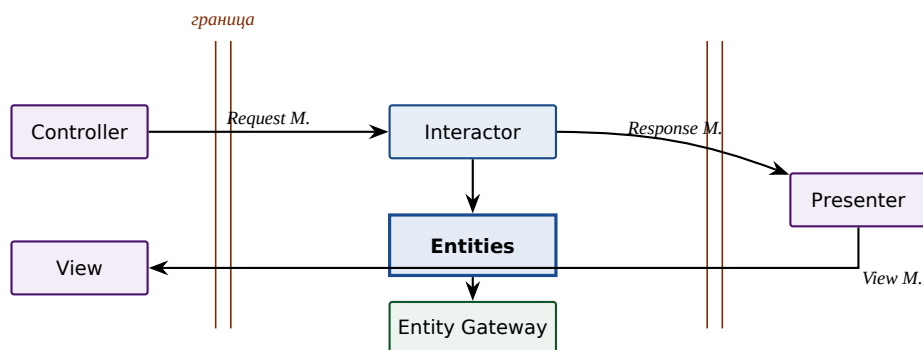
Приём: что дальше [+]

И гексагональная, и луковая — ступени к **Чистой архитектуре** (вопрос 5), которая доспецифицирует поток управления. Естественные спутники: **CQRS** (разделение моделей чтения и записи) и **BFF**. Цель всех трёх — держать доменное ядро максимально простым и независимым.

5. Чистая архитектура

Определение: чистая архитектура (Clean)

Дальнейшее уточнение луковой и гексагональной. Слоистый стиль, самый внутренний слой — доменная модель; зависимости направлены строго *снаружи внутрь* («правило зависимостей»); используется DI. Дополнительно специфицируется поток управления и конкретные способы взаимодействия с пользователем (Controller, Interactor, Presenter, Request/Response/View Model).



Пример: поток управления в рантайме

1. Запрос (из сети или GUI) попадает в **Controller**.
2. Controller валидирует/парсит и конвертирует запрос в **Request Model** (вид, удобный ядру).
3. Controller дёргает **Interactor** (тот передан ему по интерфейсу Boundary через DI).
4. Interactor через **Entity Gateway** (репозиторий, известный только по интерфейсу) находит сущности.
5. Репозиторий грузит/создаёт/возвращает сущности — это вне границ системы (а сами сущности — в центре).
6. Interactor координирует сущности и формирует **Response Model**.
7. Response Model уходит **Presenter** (известен Interactor только по интерфейсу).
8. Presenter строит **View Model** и отдаёт клиенту.
9. **View** отображает обновлённые данные.

Стиль	Что добавляет к предыдущему
Гексагональная	Ядро + порты + адаптеры; инфраструктура — внешний мир
Луковая	+ внутренние слои ядра (модель / дом. сервисы / сервисы приложений)
Чистая	+ поток управления и роли (Controller/Interactor/Presenter, Request/Response/View Model)

Осторожно

По сути нового мало — ценность в *дисциплине* разделения, а не в магии. Не путайте чёткие роли с самоцельной абстракцией: лишние Boundary там, где приложение одно и простое, — тот же перерасход, что и в гексагональной.

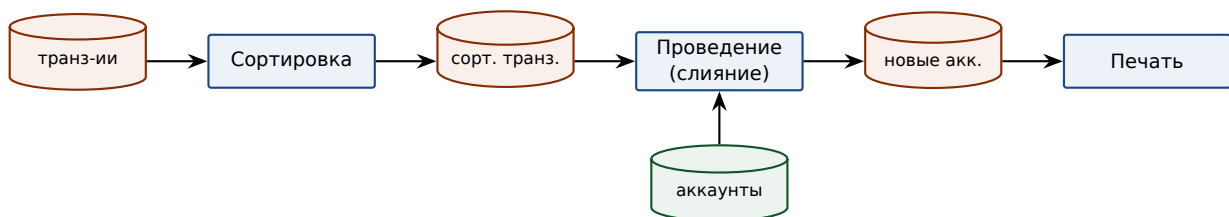
Приём: VIPER и iOS сегодня [+]

VIPER (View-Interactor-Presenter-Entity-Routing) — буквальная реализация «Чистой архитектуры» для iOS. Сегодня VIPER — в основном территория UIKit; в эпоху SwiftUI (iOS 17+) по умолчанию берут **MVVM** на макросе @Observable, а для сложного состояния — **TCA** (The Composable Architecture, однонаправленный поток в духе Redux). «Правило зависимостей» — то же «Dependency Rule» Роберта Мартина. Альтернативные современные взгляды — **Screaming Architecture** и **вертикальные срезы** (vertical slices).

6. Пакетная обработка, каналы и фильтры

Определение: пакетная обработка

«Прадедушка стилей». Система — набор *отдельных программ*, выполняемых последовательно и обменивающихся данными средствами ОС (когда-то ленты, теперь файлы или pipes). Между процессами в явном виде передаётся всё нужное: данные, конфигурация, команды.



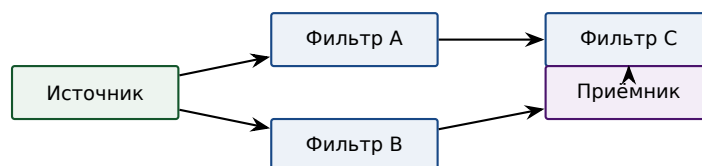
Стиль древний, но живой: **Linux way** — много маленьких утилит, соединённых пайпами в конвейеры — это и есть пакетная обработка (скрипты, работа DevOps/админов). **Плюс** — независимость программ (любой язык/технология, переиспользование готовых). **Минус** — неторопливость: запуск процессов и межпроцессное общение тяжёлы.

```
# Unix way == batch: small tools chained by pipes
cat access.log | grep 404 | awk '{print_$7}' | sort | uniq -c | sort -rn
```

Каналы и фильтры (Pipes and Filters)

Определение: каналы и фильтры

Программа — набор **фильтров**, преобразующих данные в **каналах**. Фильтры независимы: нет разделяемого состояния, фильтр видит только свои входные каналы и ничего не знает о соседях. Фильтры — единственный тип элементов, каналы — единственный тип соединителей.



В отличие от пакетной обработки, фильтр не обязан быть отдельной программой (нет проблем с производительностью), а сети фильтров бывают сложными (не только линейные). Варианты: **конвейеры** (линейная цепочка); **ограниченные каналы** (очередь конечной длины, блокирует источник при переполнении — полезно *всегда*, чтобы не было «пробок»); **типизированные каналы** (фильтр подключается только к каналу нужного типа — выбор в примере про осциллограф).

Плюсы	Минусы
Поведение = последовательное применение фильтров; легко понять/сопровождать	Последовательное исполнение: при редких данных фильтры простаивают
Легко добавлять/менять/переиспользовать фильтры (интеграция «сама собой»)	Интерактивность затруднена (данные идут в одну сторону; нужны «обратные» каналы)
Анализ как графа: пропускная способность, задержки, дедлоки	Пропускная способность = самый «узкий» фильтр
Параллелизм: каждый фильтр — кандидат в поток/процесс/микросервис	

Осторожно

«Параллелизм» и «последовательное исполнение» — две стороны одной медали: пока первые фильтры не отработали, следующие ждут. Узкое место — самый медленный фильтр; масштабировать его бывает технически непросто. Интерактивность ломает стройность стиля.

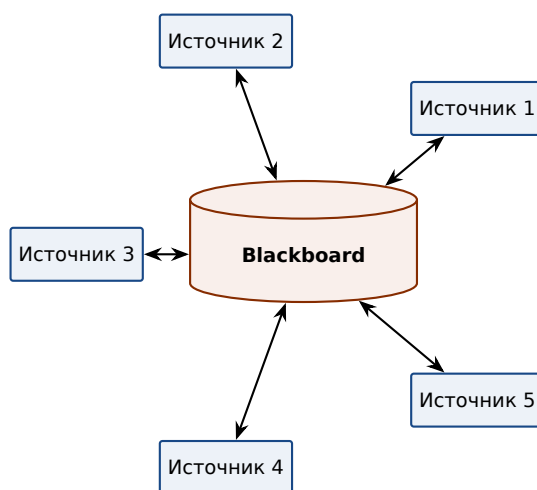
Приём: поток данных сегодня [+]

Современные потоковые/пакетные движки: **Apache Spark** и **Flink**, **Apache Beam** (единый API для batch и stream), **Airflow** (оркестрация пакетных DAG), **GStreamer** (медиаконвейеры). Визуальные DSL поверх «каналов и фильтров» — **LabVIEW**, **Matlab/Simulink**. «Фильтр → микросервис» — естественный мост к потоковой обработке событий (вопрос 8).

7. Стиль Blackboard

Определение: Blackboard («доска»)

Стиль с *общей памятью*. Есть центральное хранилище — **Blackboard** — и компоненты (источники знаний), которые знают только про доску, не имеют своего состояния и делают полезную работу. Каждый компонент смотрит на доску, (возможно) находит данные, которые умеет преобразовать, преобразует и пишет обратно — в надежде, что их подхватит кто-то ещё.



Метафора: эксперты у доски, каждый шарит в своём; выходит и пишет то, что соответствует его экспертизе и, возможно, поможет другим. Чтобы упорядочить действия, на доске держат *токен исполнения* и обновляют его по ходу.

Пример: ИИ и переписывание

Стиль применяется прежде всего в системах ИИ (распознавание речи), где задача и так может быть нерешаемой. Родственники — системы на *правилах переписывания* (машины Маркова, язык Рефал, графовые грамматики): центральная структура данных + набор правил, независимо и постепенно её меняющих.

Плюсы

Компоненты независимы, работают параллельно, разрабатываются отдельно
Лёгкое масштабирование и расширение (добавил компонент)
Не нужно знать алгоритм: «бросаешь компоненты, пока не решится»

Минусы

Задача легко может *не решиться*
Blackboard — узкое место (синхронизация доступа)
Управление порядком — только через токен на доске

Осторожно

Главный риск — *неконтролируемость*: нет гарантии сходимости. И доска синхронизирует доступ независимых компонентов ⇒ почти всегда становится узким местом по производительности.

Приём: общая память сегодня [+]

Идея «доски» живёт в **пространствах кортежей** (Linda, JavaSpaces) и общих хранилищах (**Redis** как разделяемое состояние). Логика правил дают **rule engine** (Drools). Современный всплеск — мульти-агентные LLM-системы с общим «scratchpad»: несколько агентов читают и дописывают общий контекст, что структурно и есть Blackboard.

8. Событийно-ориентированные стили, Publish-Subscribe, событийная шина

Определение: событийный стиль (неявный вызов)

Общее название стилей, где вместо явных вызовов используются **оповещения**: есть «слушатели», подписывающиеся на события; при наступлении события система сама вызывает зарегистрированных слушателей. У компонентов два вида интерфейсов — обычные методы и события; соединители — прямые вызовы либо *неявные* вызовы слушателей.

Инварианты: источник событий не знает, кто и как отреагирует (обычно даже не вправе узнать число подписчиков); не делается предположений, будет ли событие обработано вообще. **Плюсы:** низкая связность $\Rightarrow$ переиспользуемость; лёгкость конфигурирования (меняешь подписки — меняешь поведение).

Минусы событийных стилей

Неинтуитивная структура: подписки превращаются в хаотичный клубок нотификаций

Компоненты не управляют последовательностью вычислений

Неизвестно, кто и в каком порядке ответит $\Rightarrow$ системы принципиально асинхронны

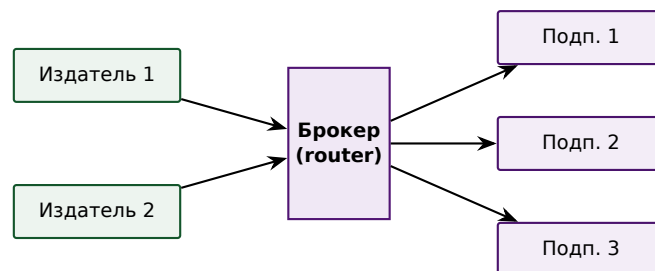
Тяжёлая отладка (нет простого «step into»; прыжки управления по коду)

Гонки даже в одном потоке (результат зависит от порядка вызова обработчиков); опасен антипаттерн «Sequential coupling»

Издатель-подписчик (Publish-Subscribe)

Определение: Publish-Subscribe

Простейший и популярный подвид. **Издатели** публикуют сообщения (синхронно/асинхронно), **подписчики** подписываются и получают события, **маршрутизаторы** — посредники, которые фильтруют и маршрутизируют (например, поочерёдная отправка $\rightarrow$ балансировка нагрузки). Соединители — сетевые протоколы, *очереди сообщений* или (на одной машине) «Наблюдатель».

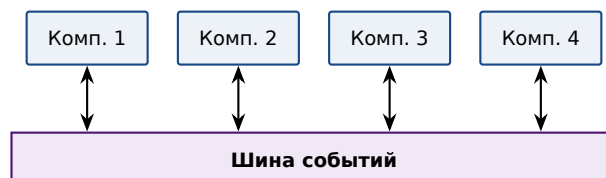


Ограничения: издатели ничего не знают о подписчиках; подписчики обычно не знают друг о друге. Простая топология $\Rightarrow$ высокая эффективность распространения информации.

Событийная шина

Определение: событийная шина

Компоненты *не* взаимодействуют напрямую — только через **шину**: подписаться можно лишь на события в шине, посылать — только шине. Два режима: **push** (шина уведомляет подписчиков сама) и **pull** (компоненты опрашивают шину). Шин может быть несколько именованных (разные типы данных).



Шина даёт ещё большую независимость, лёгкое масштабирование (не успеваешь — подключи копию) и добавление функциональности (написал компонент, подключил — остальное не трогаешь). Особенно хороша для распределённых приложений. Приём **Event Sourcing**: вместо явного состояния хранится append-only лог изменений, по которому состояние однозначно восстанавливается — удобно в распределённых системах (теорема **CAP**: из трёх — консистентность, доступность, устойчивость к разделению — доступны два; событиям проще дать «eventual consistency»). Бонусы: лог «кто виноват» и откат к любому состоянию; плата — скорость, частично компенсируется снапшотами. Уточнение для интеграции — **Enterprise Service Bus** (умная шина с преобразованием данных под каждый компонент, обычно сторонние приложения).

	Соединитель	Особенность
«Наблюдатель»	Прямой вызов в процессе	Внутри одной машины, простейший неявный вызов
Publish-Subscribe	Очереди сообщений / сеть / брокер	Издатели → (router) → подписчики
Событийная шина	Шина (push/pull)	Только через шину; Event Sourcing, ESB

Осторожно

Гонки бывают и в одном потоке: результат зависит от порядка вызова обработчиков. Порядок подписывания — слабый ориентир (в распределёнке «кто первый получил, тот и обработал»). В таких местах особенно опасен «Sequential coupling».

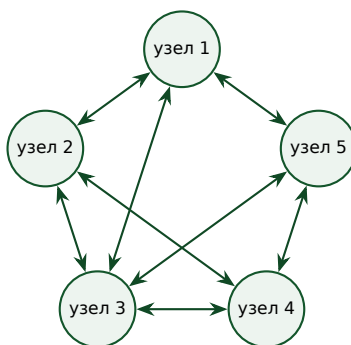
Приём: брокеры и потоки сегодня [+]

Современные брокеры/шины: **Apache Kafka** (распределённый лог; в 4.x появились нативные очереди), **RabbitMQ**, **NATS**, облачные **Pub/Sub** и **Azure Service Bus**. **Event Sourcing** обычно идёт в паре с **CQRS**. Классический **ESB** (MuleSoft, IBM Integration Bus, TIBCO) сегодня всё чаще считают тяжёлым централизованным наследием: интеграцию микросервисов строят на потоковой обработке (Kafka) и API-gateway — «умные конечные точки, тупые трубы». В P2P pub-sub роль шины играет **GossipSub** (libp2p).

9. Peer-to-peer

Определение: Peer-to-peer (одноранговая сеть)

Система из множества приложений, каждое из которых в принципе самодостаточно и может решать все задачи, но чем больше узлов работают совместно, тем шире возможности. Компоненты — отдельные приложения со своим состоянием и потоком управления; соединители — сетевые протоколы; данные — сообщения по сети.



Топология без ограничений (даже приветствуются избыточные связи) и динамична: узлы выходят и появляются на ходу. **Главный плюс — отказоустойчивость:** можно физически уничтожить часть системы, остальное продолжит работать (менее эффективно). Легко масштабировать динамическим подключением узлов ⇒ идеален для распределённых вычислений, особенно когда участник «никому ничего не должен» и работает по возможности.

Пример: BitTorrent и стаи

BitTorrent: каждый узел — файловый сервер; больше узлов — больше данных и быстрее скачивание. Менее известные применения — сенсорные сети и стаи беспилотников (военные, природоохранные): вышел из строя один — стая переконфигурируется и продолжает задание.

Осторожно

Отсутствие топологических ограничений оборачивается *сложностью координации и согласования состояния*: явное состояние корректно синхронизировать трудно (та же CAP). Подробности — в части курса про распределённые приложения.

Приём: P2P сегодня [+]

Современный фундамент P2P — **libp2p** (выделился из IPFS): модульный сетевой стек поверх TCP/QUIC/WebRTC с обнаружением через **DHT** (Kademlia), NAT-обходом и pub-sub (GossipSub); на нём построены Ethereum, IPFS, Filecoin, Celestia. Крупнейшие живые примеры P2P — блокчейны (**Bitcoin**, **Ethereum**) и **IPFS** (контентная адресация: файл ищется по хешу содержимого, а не по адресу сервера). Браузерный P2P даёт **WebRTC**; синхронизация файлов — **Syncthing**; федеративные мессенджеры — **Matrix**.

Итоговый чек-лист

- ☐ Различаю *паттерн / архитектурный шаблон / стиль / reference-архитектуру* по масштабу и привязке к предметной области.

- ☐ Знаю три соображения, задающие стиль: *элементы, правила соединения* (суть стиля), *семантика*.
- ☐ Трёхзвёнка: Display/Logic/State; клиент не лезет в БД, логика без UI и без хранения.
- ☐ MVC: View читает Model, команды — только через Controller, Model сама шлёт нотификации; Undo/Redo — в Controller («Команда»).
- ☐ Sense-Compute-Control: бесконечный цикл из трёх фаз; для сложных систем — лишь внешний каркас.
- ☐ Слоистый: строгий vs нестрогий; плюс — абстракция и сопровождение, минус — неприменимость и оверхед.
- ☐ Клиент-сервер — двухуровневый частный случай; клиенты не общаются напрямую.
- ☐ Гексагональная: ядро + порты (primary/driving, secondary/driven) + адаптеры; DIP + DI.
- ☐ Луковая: внутренние слои ядра (модель / дом. сервисы / сервисы прил.); нестрогая уровневость; репозитории снаружи.
- ☐ Чистая: правило зависимостей внутри *плюс* поток управления (Controller → Interactor → Presenter; модели Request, Response, View).
- ☐ Пакетная обработка: отдельные программы, ОС-посредник; Linux way. Каналы и фильтры: независимые фильтры, анализ как графа, узкое место = медленный фильтр.
- ☐ Blackboard: общая доска + источники знаний; параллелизм и расширяемость ценой несходимости и узкого места доски.
- ☐ Событийные стили: низкая связность ценой неинтуитивности, асинхронности, гонок и «Sequential coupling». Pub-Sub vs шина (push/pull) vs Event Sourcing/ESB.
- ☐ Peer-to-peer: самодостаточные узлы, динамичная топология; плюс — отказоустойчивость, минус — координация/согласование (CAP).
- ☐ Помню связки [+]: WCF → gRPC / ASP.NET Core; ROS 1 → ROS 2 (DDS); ESB → Kafka / потоки; VIPER → MVVM (@Observable) / TCA; P2P → libp2p / IPFS / блокчейн.

Лекция 10. Предметно-ориентированное проектирование

Рабочая тетрадь (DDD)

Лекция — сжатый пересказ первых частей книги Эрика Эванса «Предметно-ориентированное проектирование» («blue book», 2003). Сначала идея проектирования вокруг «смыслового ядра» и единый язык, затем изоляция домена через уровневую архитектуру (и антипаттерн «Умный GUI»), потом структурные кирпичики модели (сущности, значения, службы, модули) и идентичность, далее паттерны жизненного цикла (Агрегат, Фабрика, Репозиторий) и, наконец, Спецификация. Все вопросы билета раскрыты лекцией. Метка [+] помечает то, чего в лекции нет (современные инструменты, стратегический DDD, уточнения).

1. Понятие предметно-ориентированного проектирования, единый язык

Определение: DDD

Domain-Driven Design (предметно-ориентированное проектирование) — методология, при которой архитектуру строят не вокруг сиюминутных нужд реализации, а вокруг «смыслового ядра» — **модели предметной области**, отражающей основные сущности реального мира. Модель выражается *прежде всего в коде* (сущности области → классы), а также в диаграммах и — что важно — в *устной речи*.

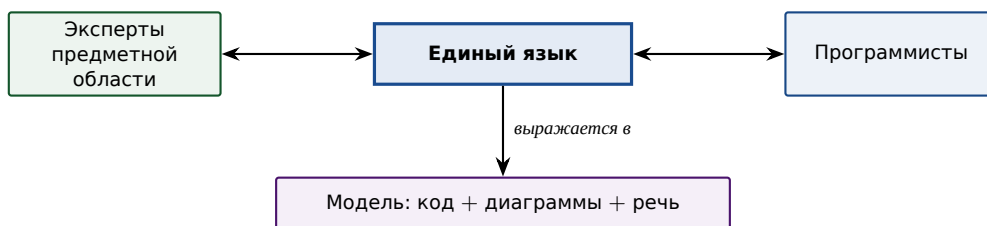
Три кита, на которых держится создание модели:

- **Переработка знаний** (knowledge crunching). Программисты редко владеют предметной областью, поэтому строят модель, *одновременно* активно её изучая: узнали новое → внесли в модель → нарисовали диаграмму → обсудили с экспертом → поправили. Цель — понять *принципы* работы области, а лишь потом писать программу.
- **Единый язык** (ubiquitous language) — общий язык команды и экспертов (см. ниже).
- **Устное общение**. Неуклюжие имена классов и описания взаимодействий *слышны* в разговоре. Естественный язык сотни лет оттачивался ровно для передачи сути понятий, поэтому он сам подсказывает архитектурные решения. Архитектура — про понимание программы *человеком*.

Определение: единый язык

Единый язык — один язык для всей команды и экспертов, используемый *повсюду*: от ТЗ до имён классов, методов, переменных и тестов. Если термин закреплён как имя класса, его синонимы запрещены везде — в коде, документации, диалогах.

Что входит в единый язык: термины области (становятся *классами*); действия и отношения (*методы*); имена паттернов проектирования, даже если в области их нет («репозиторий», «спецификация»); высокоуровневые понятия архитектуры, не выражимые прямо в коде (уровни, ограничения взаимодействия — например, «канал» в стиле «каналы и фильтры»).



Эксперты и программисты общаются только на едином языке.

Язык *эволюционирует* вместе с пониманием области, моделью и кодом. За изменениями, которые естественно возникают в разговорах, надо следить и рефакторить код: если все стали говорить «связь» вместо «ассоциация», класс переименовывают.

Языков в проекте может быть несколько. Если система состоит из обособленных частей, у каждой может быть свой язык (и своя модель). Пример: система грузоперевозок зовёт связь «перевозкой», а встроенная библиотека графов — «ребром». Это допустимо, *если* есть чёткая граница и определены правила преобразования (фасады, адаптеры). Если границы нет, модели проникают друг в друга ⇒ путаница и провал.

Разбор: сила единого языка (пример из книги)

Обсуждение смены пункта растаможивания на *низком* уровне абстракции звучит так: «удаляем все строки в таблице отправки с данным id груза, передаём пункт отправки, назначения и новый пункт в Маршрутизатор, он заполняет

таблицу заново; в Груз добавляем флаг, есть ли данные». На *хорошей* модели — так: «при изменении любого атрибута **Спецификации маршрута** удаляем старый **Маршрут** и просим **Маршрутизатор** построить новый по новой Спецификации». Короче, точнее и полнее — и само появление понятия «Спецификация маршрута» родилось из попытки выговорить фразу без «и всё такое».

Приём: как строят язык и контексты сегодня [+]

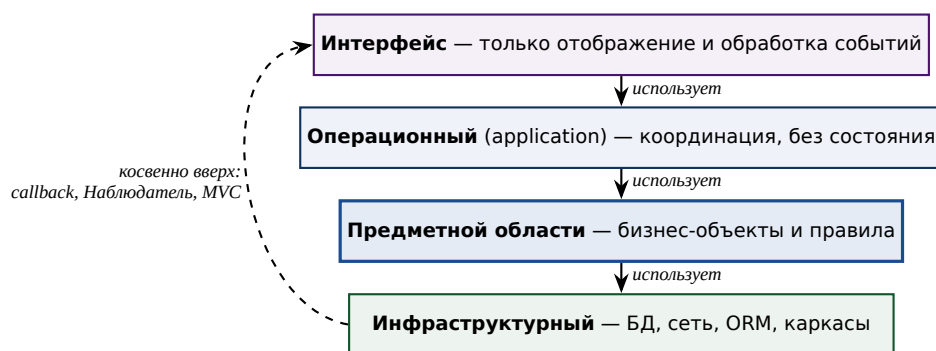
Лекционные «несколько языков с границей и правилами перевода» — это стратегический DDD, который Эванс развил во второй части книги: **Ограниченный контекст** (Bounded Context) — зона, где язык однозначен; **Карта контекстов** (Context Map) — схема их отношений; **Антикоррупционный слой** (adapter на стыке) защищает модель от чужой. Каждый контекст сегодня часто = отдельный **микросервис** (или модуль «модульного монолита»). Сам язык и границы ищут на воркшопах **Event Storming** (Альберто Брандолини, с 2013: стикеры с доменными событиями на таймлайне) и **Domain Storytelling** — лёгкая замена громоздкому UML.

2. Изоляция предметной области, антипаттерн «Умный GUI»

Определение: изоляция домена

Чтобы код модели — средоточие сути приложения — был *маленьким, простым и содержал только существенное*, всё остальное (UI, сеть, БД, логирование, обработка ошибок, тесты) архитектурно отделяют от модели. Прочие модули просто *используют* классы доменной модели.

Самый популярный способ — **уровневая архитектура**: уровень взаимодействует напрямую только с уровнями ниже. DDD не диктует конкретное разбиение (трёхзвенка, OSI, TCP/IP подойдут), требует лишь, чтобы *был* уровень предметной области, где сосредоточены все классы модели.



Ответственность уровней. *Интерфейс* — никакой содержательной логики. *Операционный* — тонкий, инициализирует систему и по сигналу интерфейса запускает нужные операции; ему *запрещено состояние* (кроме связанного с общением с пользователем, типа прогресса). *Домен* — бизнес-объекты, делающие *только* бизнес-правила; их реализации уделяют максимум внимания, ведь именно домен даёт конкурентное преимущество. *Инфраструктура* поддерживает всех выше (каркасы вроде Java Beans или ROS, ORM, работа с библиотеками).

- **Домен переиспользуем** в семействе приложений; операционный уровень специфичен для каждого. Бизнес-регламенты (последовательность шагов процесса) живут в домене, а не в операционном уровне, — они объективно присущи области.
- **Деревья наследования растут вверх:** базовые классы для верхних уровней могут быть в инфраструктуре. Это нормально — предок «знает» о потомке меньше, чем потомок о предке.
- **Вверх — только косвенно.** Уровень снизу иницирует действия наверху через callback-и (виртуальные hook-и), паттерн «Наблюдатель», MVC. Чуть труднее отлаживать, зато уровни разрабатываются независимо.

Осторожно: антипаттерн «Умный GUI»

Вся бизнес-логика (и даже работа с БД) свалена прямо в обработчики событий GUI. Доменных классов нет; есть методы-обработчики, где перемешаны интерфейс, бизнес-правила и инфраструктура.

Разбор: когда «Умный GUI» — не зло, а когда зло

Допустимо (прототип, утилита < 3000 строк): куча RAD-тулзов генерит большую часть приложения; быстро (рабочая программа за пару часов, без оверхеда на классы и уровни); фичу добавить легко (кинул контрол + обработчик → в продакшн); код каждого куска прост, легко переписать. **Становится антипаттерном** (> 3000 строк): нет проектирования по модели (классов области просто нет); правило, нужное двум фичам, реализуется *дважды*; сложное поведение не декомпозировать без уровней; интеграция почти невозможна — всё завязано на GUI, отдельный API не сделать.

Приём: изоляция домена сегодня [+]

Современный способ изоляции — **гексагональная / луковая / чистая** архитектура (вопрос про стили): домен в центре, инфраструктура снаружи через порты и адаптеры. DDD + «порты и адаптеры» — стандарт де-факто. Границы уровней можно *проверять автоматически*: **jMolecules** размечает пакеты как @DomainLayer / @ApplicationLayer / @InfrastructureLayer, а **ArchUnit** падает на сборке, если кто-то нарушил направление зависимостей. «Умный GUI» сегодня — это low-code / no-code платформы: там он уместен ровно по тем же причинам и ломается на тех же порогах сложности.

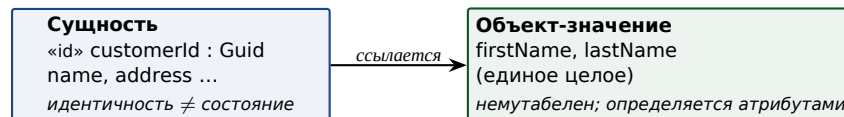
3. Основные структурные элементы модели, идентичность объекта

Модель собирается из «элементарных» кирпичиков; выбор кирпичика для каждого понятия — нетривиальное проектное решение.

Ассоциации. Если одно направление приоритетнее («страна → президент»), делаем ассоциацию *односторонней*; двусторонняя мешает понять объект в отрыве от другого. Минимизируем *множественность* (квалификатор: «страна → год → президент») и *число* ассоциаций. Меньше связей — проще анализ и сопровождение.

Определение: сущность и объект-значение

Сущность (Entity) имеет собственную *идентичность*, не зависящую от состояния. **Объект-значение** (Value Object) определяется *только значениями атрибутов*. Похоже на ссылочные типы и типы-значения, но в жизни сложнее: ссылочная идентичность не переживает сериализации.



Идентичность в информационных системах

Естественной идентичности почти не бывает: паспорт меняют, ИНН и СНИЛС есть не у всех, тёзки путаются. Идентичность должна *переживать* сохранение, загрузку, передачу в другую систему и иной формат — значит, нужна явная **операция идентификации** (ссылочное равенство не годится). Это важное архитектурное решение.

Разбор: входящий звонок

Звонит клиент — кто это? (1) старый клиент со *знакомого* номера; (2) старый клиент, сменивший номер (надо догадаться, что уже имели дело); (3) *новый* клиент, которому достался номер старого (надо догадаться, что новый). Номер телефона как идентификатор обманчив.

- **Суррогатный идентификатор** (GUID при создании): иногда виден пользователю (трек-номер посылки), иногда нет (id документа в Google Docs — в одной папке два файла с одним именем).
- **Естественный атрибут** (номер кресла в кинотеатре): лучше суррогатного, ведь сам решает проблему «дубликатов» той же сущности из другой системы — *но* осторожно: мнимо-уникальное свойство часто таким не оказывается.
- **Превращайте всё возможное в значения.** Иначе сложные правила идентификации придётся применять к *каждому* объекту (боль и неэффективность). Значения — единое концептуальное целое («имя + фамилия» — один VO, не два атрибута), желательно немутабельные; могут быть ссылочными типами (для разделяемости). Разделяемость экономит память, но в распределённой системе — сетевой запрос на каждое обращение.

Осторожно: сущность или значение — решение проектирования

Это *не врождённое* свойство объекта. Билет с *указанием места* ⇒ место — сущность (нельзя продать два билета на одно). Билет *без места* ⇒ важно лишь количество, место — значение (считать его сущностью — ошибка, лишнее усложнение).

Службы (Services). В DDD разрешены классы *без состояния* (по сути набор статических методов). В службу выносят операцию над объектами *нескольких равноправных* классов, которой нет естественного места ни в одном из них (диспетчеры, менеджеры, Маршрутизатор). Состояния нет, поэтому методы зовут откуда угодно и в любом порядке.

Модули (Modules). Группировать классы надо *семантически* (по единому языку), а не механически. Модуль повышает «объясняющую способность» модели; имена модулей — из единого языка. Высокая связность (cohesion) и низкая зависимость (coupling) при этом получаются *сами собой*; если нет — рефакторить модель. Главное — не метрики, а объясняющая способность.

Приём: кирпичики сегодня [+]

К четырём кирпичикам лекции (сущности, значения, службы, модули) современный DDD добавляет **доменные события** (Domain Events) — факт «что-то произошло», основа связи между агрегатами и контекстами. Немута-

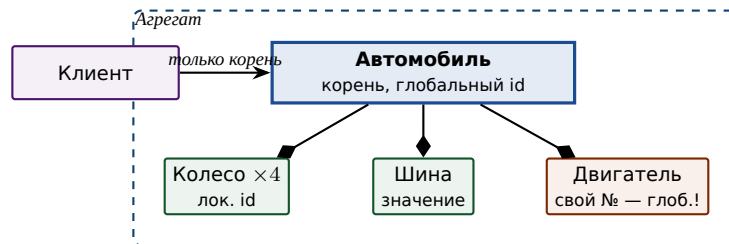
белые значения теперь пишут как **records** (Java record, C# record, Kotlin data class). Кирпичики можно *разметить* в коде: jMolecules даёт @Entity, @ValueObject, @Identity, @Service, @Module, а инструменты проверяют правила и рисуют диаграммы.

4. Паттерн «Агрегат»

Объекты в ИС живут сложнее, чем «создал → попользовался → удалил»: добавляются проблемы идентичности, целостности и управления состоянием. Их решают три паттерна жизненного цикла; первый — **Агрегат**.

Определение: агрегат

Агрегат — изолированный кусок модели с **корнем** и **границей**. Доступ ко всем элементам — *только через корень*. Корень — сущность с *глобальной* идентичностью (в рамках всего приложения); внутренние объекты держат идентичность лишь *локально* (в рамках агрегата). Это единица инкапсуляции: пишем агрегат — думаем о связях *внутри*; пользуемся им — думаем только о *корне*.



Зачем агрегат: упрощает и декомпозирует модель; технически — транзакции, затрагивающие *разные* агрегаты, гарантированно не мешают друг другу, что в разы облегчает обновления в БД.

Инварианты и граница. У агрегата (как у класса) есть инварианты, за их поддержание отвечает корень. Содержимое *может быть видно* снаружи, ссылки на внутренние объекты отдавать не возбраняется — но *хранить* их извне нельзя. Поэтому отдаваемые наружу объекты должны быть *немутабельны* или быть копиями.

Разбор: автомобиль как агрегат

Машина — корень. Колёса и шины внутри: пользователю важно, *на каком колесе какая шина*, но глобально идентифицировать шину не нужно — ей хватает локальной идентичности. Двигатель — тоже часть машины, но имеет *собственный номер*, уникальный глобально (его проверяют и отдельно от номера авто). Управление шинами наружу даёт сам Автомобиль, поддерживая свои инварианты.

Приём: правила агрегатов сегодня [+]

Вон Вернон («red book», 2013) сформулировал практические **правила проектирования агрегатов**: держать агрегаты *маленькими*; ссылаться на другие агрегаты *только по идентификатору* (не по объектной ссылке); *одна транзакция — один агрегат*, а согласованность между агрегатами — *отложенная* (eventual), через доменные события. Граница агрегата = граница транзакции = граница согласованности (а часто и граница микросервиса). **Spring Modulith** прямо *проверяет* эти правила на сборке.

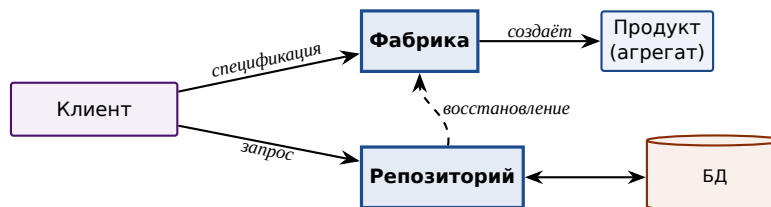
5. Паттерны «Фабрика», «Репозиторий»

Определение: фабрика

Фабрика — создаёт и инициализирует агрегат (или объект) так, чтобы выполнялись его инварианты. Инкапсулирует сложное создание, не раскрывая внутреннюю структуру, и избавляет агрегат от обязанности уметь создавать себя. Создание *атомарно*: если что-то пошло не так — клиент не получает результата вовсе. Может реализовываться через «Фабричный метод», «Абстрактную фабрику», «Builder».

Определение: репозиторий

Репозиторий (Хранилище) — отвечает за хранение, сохранение и загрузку объектов, давая, как правило, *глобальный доступ*. Обычно прослойка между приложением и БД; может держать объекты в памяти. За интерфейсом запросов прячет реальный способ получения (ORM + реляционная БД, объектная БД, фабрики). Задача — чтобы клиентскому коду не надо было обо всём этом заботиться.



Фабрика подробнее. Клиент передаёт параметры (часто — «Спецификацию», см. далее), получает готовый продукт. Фабрика *вправе* знать структуру агрегата и лезть в его внутренности, а внешний код — нет, поэтому без фабрики агрегатом часто не воспользоваться. Фабрики почти никогда не присутствуют в области *явно*, но всё равно — часть модели и единого языка наравне с «настоящими» объектами. Фабрика умеет и *восстанавливать* объект из хранилища (например, разбор XML), при этом *не* присваивая новый идентификатор и не портя идентичность.

Репозиторий подробнее. Зачем он: чтобы не искать объект длинной цепочкой переходов по ссылкам от корневого объекта (это блокировало бы половину БД и убивало параллельную работу). Часто лучше хранить *id* нужного объекта и брать его из репозитория по запросу, чем держать прямую ссылку. Репозиторий может иметь развитый **язык запросов** (по суррогатному *id* или выборка по атрибутам и отношениям; снова пригодится «Спецификация»). Репозитории отлично дружат с агрегатами: дают корень по глобальному *id* и собирают остальной агрегат при необходимости.

Осторожно: фабрика ≠ репозиторий

Фабрика делает *новый* объект; репозиторий находит *уже существующий* (часто — через фабрику для восстановления). Не путать создание с воссозданием: при восстановлении нельзя менять идентичность.

Приём: фабрики и репозитории сегодня [+]

Репозиторий — самый «выживший» паттерн DDD: **Spring Data** (JpaRepository, производные запросы из имён методов) и **EF Core** (DbContext — это репозиторий + Unit of Work) дают репозиторий почти даром, объявляя лишь *интерфейс*. jMolecules помечает их @Repository. Фабрики сегодня — это билдеры и DI-контейнеры; «восстановление из XML» — гидрация объектов средствами ORM. При **CQRS** читающий и пишущий репозитории разделяют.

6. Паттерн «Спецификация»

Отдельная важная задача — моделирование **ограничений** на состояние системы. Ограничения умеют «внезапно» порождать в модели новые объекты, которых будто не было в области.

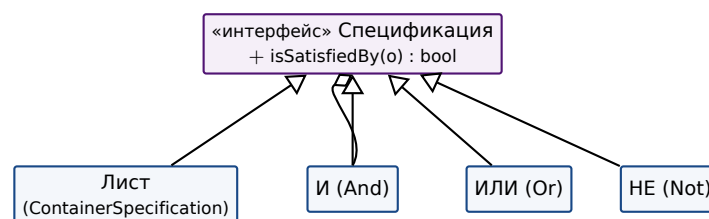
Лестница абстракций ограничения (пример: ведро, в которое нельзя налить больше ёмкости): смешать проверку с полезным кодом → вынести в *метод с говорящим именем* (constrainedToCapacity) → для сложных ограничений завести *отдельный класс*.

```

// step 2: constraint extracted into a named method -- impossible to lose
class Bucket {
    private float capacity, contents;
    public void pourIn(float addedVolume) {
        float volumePresent = contents + addedVolume;
        contents = constrainedToCapacity(volumePresent);
    }
    private float constrainedToCapacity(float v) {
        return (v > capacity) ? capacity : v;
    }
}
  
```

Определение: спецификация

Спецификация — класс, чья работа — проверять объект на соответствие ограничению. Содержит булев метод *isSatisfiedBy(o)*: «удовлетворяет объект или нет».



Композиция (Компоновщик). Сложные спецификации собирают из простых через «И», «ИЛИ», «НЕ». **Три применения** спецификации: (1) *проверка* ограничения (большой assert); (2) *выборка* объекта из репозитория (по сути упрощённое дерево разбора SQL); (3) *построение* объекта в фабрике («хочу однонаправленный граф, где вершин примерно столько же, сколько рёбер» — и фабрика выберет подходящую реализацию).

Разбор: склад химикатов

Химикаты (обычные, испаряющиеся, взрывоопасные) хранят в контейнерах (обычный, с вентиляцией, бронированный). Каждый контейнер даёт набор *характеристик*; требования химиката к хранению — это **спецификация**, которой контейнер может удовлетворять. У химиката есть ссылка на спецификацию, спецификация проверяет наличие нужной характеристики у контейнера.

```
public class ContainerSpecification {
    private ContainerFeature requiredFeature;
    public ContainerSpecification(ContainerFeature required) {
        requiredFeature = required;
    }
    boolean isSatisfiedBy(Container c) {
        return c.getFeatures().contains(requiredFeature);
    }
}
// the Container uses the spec to check it is packed safely:
boolean isSafelyPacked() {
    for (Drum drum : contents)
        if (!drum.containerSpecification().isSatisfiedBy(this))
            return false;
    return true;
}
```

Приём: спецификация сегодня [+]

Спецификация жива во фреймворках доступа к данным: **Spring Data Specification<T>** (типобезопасные критерии запроса, комбинируются and/or), **LINQ** и предикаты **EF Core**. По сути это «объект-запрос» (query object) и функциональный Predicate<T>. Та же идея — в движках правил (rule engines), где бизнес-правило отделено от кода, который его применяет.

Итоговый чек-лист

- ☐ DDD строит систему вокруг *смыслового ядра* — модели области, выраженной в коде; держится на переработке знаний, едином языке и устном общении.
- ☐ Единый язык — везде (ТЗ, классы, методы, тесты); синонимы запрещены; язык эволюционирует → рефакторим код. Языков может быть несколько при чёткой границе. [+] это Bounded Context + Context Map + антикоррупционный слой; границы ищут Event Storming.
- ☐ Изоляция домена — уровневая архитектура (Интерфейс / Операционный / Домен / Инфраструктура); вниз — напрямую, вверх — косвенно (callback, Наблюдатель, MVC). Домен переиспользуем, операционный тонкий и без состояния.
- ☐ «Умный GUI» — логика в обработчиках GUI; ок до ~3000 строк (RAD, быстро), дальше антипаттерн (нет переиспользования правил, нет интеграции).
- ☐ Ассоциации — однонаправленные, минимум множественности и числа. Сущность (идентичность ≠ состояние) vs значение (атрибуты, немутабельно). Идентичность переживает save/load/transfer; surrogate (GUID) vs естественный атрибут. Сущность-или-значение — решение проектирования.
- ☐ Службы — классы без состояния (операции над равноправными объектами). Модули — по семантике (единый язык), а не по метрикам.
- ☐ Агрегат — корень + граница; доступ только через корень; глоб. id у корня, лок. внутри; инварианты держит корень; транзакция = агрегат. [+] правила Вернона: маленькие, ссылка по id, eventual consistency.
- ☐ Фабрика — атомарное создание агрегата с инвариантами (и восстановление без смены id). Репозиторий — хранение + глобальный доступ, прячет ORM; дружит с агрегатами. [+] Spring Data / EF Core.
- ☐ Спецификация — класс с isSatisfiedBy; Компоновщик (И/ИЛИ/НЕ); три применения — проверка, выборка, построение. [+] Spring Data Specification<T>, LINQ.

Лекция 11. DDD: стратегические аспекты

Рабочая тетрадь

Лекция — пересказ части IV книги Эванса: как применять DDD, когда над проектом работают несколько команд и система большая. Сначала целостность модели (ограниченный контекст, непрерывная интеграция, карта контекстов) и семь паттернов интеграции контекстов; затем выделение смыслового ядра (дистилляция, абстрактное ядро); наконец, крупномасштабная структура (метафора системы, осмысленные уровни, типичные уровни производственных и финансовых систем) и стили «Уровень знаний» и «Подключаемые компоненты». Все вопросы билета раскрыты лекцией. Метка [+] помечает то, чего в лекции нет (современные практики и уточнения).

1. Ограниченный контекст, непрерывная интеграция, карта контекстов

Единая модель и единый язык хороши, пока команда мала. Над большим проектом работают много команд — у них разное видение продукта. Две крайности одинаково плохи:

- **держат модель цельной** — неподъёмные затраты (в идеале все команды общаются со всеми, каждое изменение утверждают остальные), а модель выйдет слишком общей, чтобы быть полезной;
- **разрешить модели фрагментироваться** — затруднит переиспользование кода и интеграцию системы.

Разбор: баг с классом «Платёж»

Команде понадобилась абстракция платежа клиента; в коде уже был класс «Платёж» другой команды (для оплаты субподрядчику) — его переиспользовали («пары полей не хватало, одно не так названо — да какая разница»). Через пару дней система упала на генерации налоговых отчётов: у платежей клиентов не заполнено поле «необлагаемый процент» (для них оно бессмысленно), но они *тоже* попадали в налоговый расчёт. Корень — системный: две команды вкладывали в один термин разный смысл без механизма выявить расхождение. Решение: две абстракции — «Платёж поставщику» и «Платёж клиента».

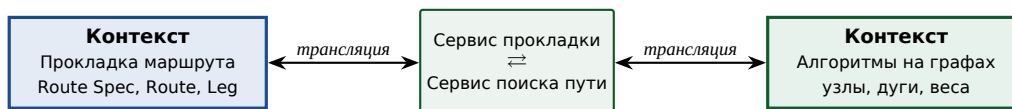
Определение: ограниченный контекст

Ограниченный контекст (Bounded Context) — кусок предметной области и реализующего её кода, где применяется одна модель (и единый язык). Внутри всё следует этой модели; снаружи никто не вправе делать предположений о ней или переиспользовать её без спроса. Это «атомарная» область проекта, над которой работает одна команда. Разбиение на контексты обычно следует организационной структуре (а та — высокоуровневой структуре системы).

Непрерывная интеграция (внутри контекста, в классическом смысле): слияние в основную ветку раз в несколько часов, сборка и юнит-тесты после каждого коммита, высокое покрытие, постоянное общение. 3–4 человека легко договариваются, но дробить систему на контексты по 3–4 человека непрактично — CI и держит единство модели внутри более крупного контекста.

Определение: карта контекстов

Карта контекстов (Context Map) — фиксирует, как понятие одной модели транслируется в понятие другой. Обычно описывается на естественном языке (в терминах единых языков), иногда — диаграммами.



Leg и Arc выражают одно понятие — дубликация осмысленна: команды работают параллельно.

Пример (любимый Эванса). Команда бизнес-логики оперирует понятиями «Спецификация маршрута», «Маршрут», «Перевозка» и зовёт чужой код через «Сервис прокладки маршрута». Вторая команда знает только графы: узлы и дуги с весами, узлы привязаны к координатам, в узлах — погрузка абстрактных ТС по коду. Это даёт ей переиспользовать готовые библиотеки графов. Цена — трансляция в «Сервисе» туда-обратно, о которой команды чётко договариваются; зато за сервисами всё разрабатывается независимо.

Приём: контексты, команды и карты сегодня [+]

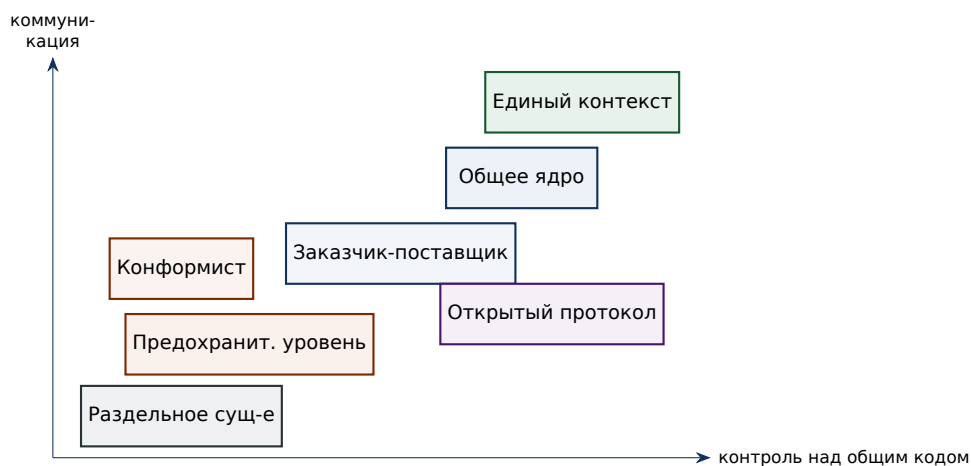
Связь «контекст следует за орг-структурой» — это **закон Конвея** (1968: система повторяет структуру коммуникаций команд). Современное развитие — **Team Topologies** (Скелтон, Пайс, 2019): ограниченный контекст ↔ *stream-aligned*-команда; *обратный манёвр Конвея* (строим команды так, чтобы получить нужную архитектуру); «Team API» ≈ Open Host Service; ключевой критерий — *когнитивная нагрузка* команды. Карты рисуют DSL-ем **Context Mapper**; стыки проверяют контрактными тестами (**Pact**, Spring Cloud Contract). «Непрерывная интеграция» сегодня — это весь конвейер CI/CD.

2. Подходы к интеграции контекстов

Степень интеграции моделей зависит от обстоятельств, обычно не зависящих от команды. Стремиться к высокой интеграции *не всегда* стоит — один из официальных паттернов есть «не интегрироваться вовсе». Эванс выделяет семь подходов (своего рода паттерны интеграции).

Паттерн	Когда применять	Что требует / даёт
Общее ядро (Shared Kernel)	Команды могут <i>прямо</i> общаться, на одном уровне подчинения	Совместная реализация общей части модели; менять ядро — только по согласованию обеих
Заказчик-поставщик (Customer-Supplier)	Общение затруднено или команды в отношении <i>зависимости</i> ; разные циклы релизов	Явно зафиксировать роли; заказчик даёт задачи и приёмочные тесты; одна иерархия управления
Конформист (Conformist)	Полная зависимость от компонента, на который <i>не повлиять</i> (legacy, платформа, навязано сверху)	Принять чужую модель как есть; борется с синдромом «not invented here»; часто — мудрое решение
Предохранительный уровень (Anticorruption Layer)	Конформизм нельзя — чужая модель противна, но компонент нужен	Прослойка-переводчик понятий (Фасад, Адаптер, службы, трансляторы, кеши); много кода
Раздельное существование (Separate Ways)	Выгода от интеграции < затрат на неё	<i>Никакой</i> интеграции; игнорируем чужой проект, ничего не переиспользуем
Служба с открытым протоколом (Open Host Service)	Поставщик с <i>очень многими</i> заказчиками	Строим модель сами, публикуем интерфейс + документацию; заказчики подстраиваются
Общедоступный язык (Published Language)	Область популярна: много заказчиков и поставщиков	Совместно разработать, стандартизовать и навязать общую модель (макс. простую, выразительную)

Открытый протокол vs реализация: внешняя (опубликованная) модель часто *проще* внутренней — снаружи стараются упростить, внутри можно «разгуляться». Примеры «Открытого протокола» — публичные REST-сервисы (VK API, Google Drive API). Примеры «Общедоступного языка» — языки программирования (C++, UML), отраслевые стандарты (медицинские данные, авионика, автомобильное ПО); IEEE 1016-2009 из первой лекции тоже описывает предметную область архитектурных видов через UML.



Справа вверху — максимум интеграции; слева внизу — паттерны «когда всё плохо».

Разбор: унификация слона (шесть слепых мудрецов)

Шесть команд в шести контекстах «ощупывают слона» (кто стена, кто змея, кто верёвка). Постепенная интеграция: **Раздельное существование** (каждый при своём, системы несовместимы) → **Предохранительный уровень** (договорились о правилах трансляции свойств: «расположение стены» → «точка привязки верёвки») → **Общее ядро** (поняли, что говорят об одном; общая сущность «Слон», класс «Стена» стал предком — но в единый язык «слон есть стена» не вошло: язык важнее техники) → **единый контекст** (полное согласие). Так команды, в отличие от мудрецов, собирают из фрагментов единую модель.

Приём: интеграция контекстов сегодня [+]

Паттерны Эванса — основа современного «context mapping» (микросервисы). **Общедоступный язык** — это версионизируемые контракты: схемы **OpenAPI** / **AsyncAPI** / protobuf / GraphQL. **Открытый протокол** — публичный API за шлюзом («Team API»). **Предохранительный уровень** — штатный приём переезда с legacy: **Strangler Fig** (Фаулер) + ACL, у Microsoft и AWS это отдельные «architecture patterns». **Конформист** — потребление чужого API «как есть». К списку добавляют **Партнёрство** (Partnership — две команды успевают или проваливаются вместе) и анти-паттерн **Big Ball of Mud**.

3. Смысловое ядро, приёмы дистилляции, абстрактное ядро

Определение: смысловое ядро

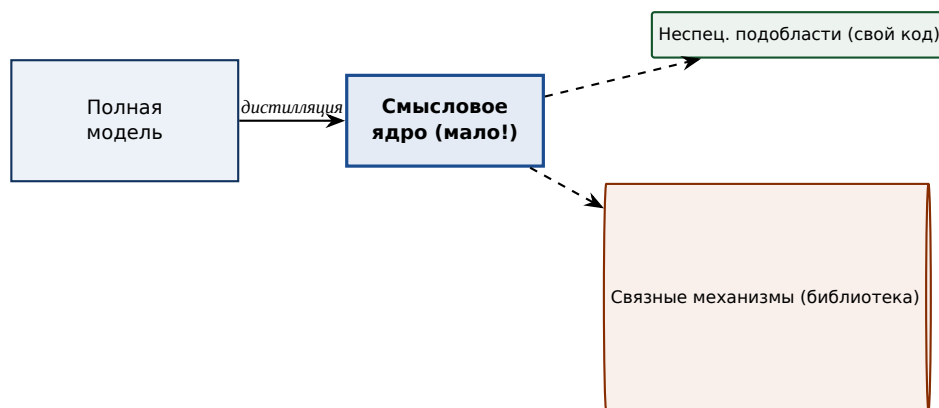
Смысловое ядро (Core Domain) — самая содержательная часть области: то, что делает систему ценной, *know-how*, за что платят заказчики и чем продукт отличается от конкурентов. Всё прочее (БД, сеть, UI) можно смело брать или выкладывать в open source. Ядро должно быть *самым проработанным, самым маленьким и чётко отделённым*.

Осторожно: ядром не любят заниматься

Ирония: опытные программисты избегают ядра — оно требует глубокого погружения в область (приятнее писать БД и UI и собрать длинный список технологий в резюме). Но к уровню senior резюме-стек обесценивается (junior-ы знают новые технологии лучше), а *специализация в предметной области* делает вас редким и дорогим специалистом, которого зовут сами.

Дистилляция — процесс выделения ядра и вынесения из него лишнего. Приёмы:

- **Domain Vision Statement** — короткий (~1 страница) документ: что ядро умеет и зачем (не архитектура!). Делается в начале проекта (родственник «устава проекта»).
- **Выделенное ядро** (Highlighted Core) — обозначить ядро на готовой модели. Два вида: *дистилляционный документ* (3–7 страниц про состав ядра и взаимодействие элементов — развёрнутый Vision Statement) и *Flagged Core* (элементы ядра помечены стереотипом или просто цветом — дёшево и эффективно).
- **Вынести неспецифичное** в отдельные модули: **Неспециализированные подобласти** (Generic Subdomains — важны для области, но не специфичны для *этой* системы; возможно, придётся писать самим) и **Связные механизмы** (Cohesive Mechanism — неспецифичны для области вообще, например работа с графами; точно реализованы 1000 раз — берём готовую библиотеку).



Разбор: дистилляция системы грузоперевозок

Ядро логистической компании — классы про *доставку*: Cargo, RouteSpecification, Itinerary, HandlingStep, BillOfLading, CustomerAgreement. *Не ядро*: Customer и контакты (это умеют CRM лучше нас); Перевозка, расписание, даже прокладка маршрута (есть библиотеки) — важно, но не «то, за что нас любят клиенты». Сложность ядра упала вдвое. Дальше: разбить Shipping на Customer + Logistics; Invoice и PricingModel → в Customer; Money и Currency → отдельный модуль-*неспециализированная подобласть*.

Абстрактное ядро. Если ядро всё ещё велико, оставляют в нём *только абстрактные классы и интерфейсы*, а реализации выносят в отдельные модули. Сложность это радикально не снизит (это *essential complexity*, она неустранима), но избавит ядро от подробностей реализации.

Приём: ядро и подобласти сегодня [+]

Современная классификация подобластей (Вернон): **Core** / **Supporting** (свои, но не ключевые) / **Generic** — и стратегия **buy-vs-build**: ядро *пишем сами*, обобщённое *покупаем или берём из OSS* (платежи — Stripe, аутентификация — Auth0 / Keycloak, CRM — SaaS). Domain Vision Statement ≈ сегодняшний «product one-pager»; Flagged Core размечают стереотипами jMolecules (@Module) и проверяют ArchUnit; связный механизм — сигнал «здесь нужна готовая библиотека».

4. Крупномасштабная структура, метафора системы, разбиение по уровням

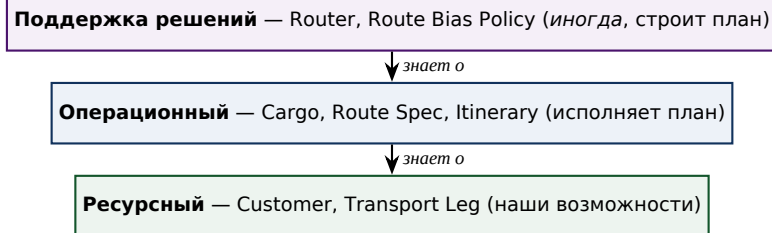
Определение: крупномасштабная структура

Крупномасштабная структура (Large-Scale Structure) — самая «архитектурная» часть архитектуры: набор основных правил, которым следует вся система. Обычно это *уточнённый под систему архитектурный стиль* плюс специфичные ограничения (перечень основных компонентов и их ответственностей, правила добавления новых).

- **Эволюционирует** вместе с пониманием области (переработка знаний), моделью и кодом. Не фиксировать навсегда, не делать слишком жёсткой, не насаждать административно.
- **Антипаттерн «Архитектор в башне из слоновой кости»:** архитектор проектирует, но не пишет код и не слушает программистов — особенно вредно в DDD (мешает переработке знаний).
- Маленькие проекты живут без явной структуры (пять классов — их связи и есть архитектура); «архитектурить» скрипт — саботаж сроков. Но большие проекты начинались малыми → зафиксировать хоть какую-то структуру и постепенно уточнять.
- Носитель структуры — не обязательно UML-фолиант, вполне может быть **единый язык** и устная традиция («нет, мы так не делаем»). Автогенерация диаграмм по коду — *не* архитектура.

Метафора системы — аналогия для понимания системы в целом (рабочий стол, файервол, дерево). ПО незримо, поэтому метафоры нужны. *Опасность:* метафора тащит лишний смысл — файервол (как противопожарная стена) заставляет забыть про *локальный* трафик; рабочий стол не бесконечен. Хорошая метафора (особенно из предметной области) сильно помогает.

Осмысленные уровни. Нельзя делить систему на уровни механической топологической сортировкой графа зависимостей. Разбиение должно *нести смысл из предметной области* и объяснять происходящее — качество архитектуры (как научной теории) определяется её *объясняющей способностью*. Каждому уровню придают значение из области и связанные ограничения.



Топологическое правило: нижний уровень не знает о верхнем (связи назад — через репозиторий).

Разбор: уровни грузоперевозок и опасные грузы

Систему делят на **Ресурсный** (то, чем мы не управляем: перевозчики, клиент) и **Операционный** (как используем возможности), затем выделяют **Поддержку решений** (Router строит Расписание *иногда*). Чтобы нижний Customer не знал о Cargo — паттерн «Репозиторий» (репозитории глобальны, ссылок не хранят, в уровни не вписываются). Новое требование — опасные грузы («HazMat code»). *Неверно:* сервис, где Груз запрашивает спецификацию-ограничение у уровня выше — ломает топологию. *Верно:* спецификацию по коду опасности получает сам *Маршрутизатор* (и сразу видно, что так лучше даже без опасных грузов — *переработка знаний*). **Мораль:** крупномасштабная структура *приоритетнее* сиюминутных требований; ради неё рефакторят, иначе — эрозия, Lava Flow, смерть проекта.

Приём: фиксация структуры сегодня [+]

Крупномасштабную структуру сегодня документируют **C4-моделью**, **ADR** (Architecture Decision Records) и шаблоном arc42, а соблюдение проверяют **фитнес-функциями** / ArchUnit («архитектура как код»). Тезис «автогенерация ≠ архитектура» жив. Метафора системы и осмысленные границы переключаются с «fracture planes» из Team Topologies.

5. Типичные уровни в производственных и финансовых системах

Полученное в примере деление не случайно — оно повторяется в реальных системах. Для систем **автоматизации производства** типична четырёхуровневая структура; для **финансовых** — тоже четырёхуровневая, но иная (нет «потенциального» уровня — особым оборудованием они не управляют — зато есть уровень «обязательств»).



Производство (снизу вверх):

- **Потенциальный** — наши возможности: оборудование, площади, контракты с поставщиками. Меняется *редко*.
- **Операционный** — оперативное управление: что на складе, статус производства, оперативный план, рабочие на местах. Обновляется *постоянно*.
- **Регламентный** — ограничения управления и планирования: техрегламенты, законодательство, локальные акты, стратегия. Меняется *раз в годы*.
- **Принятия решений** — стратегическое управление: анализ и оптимизация цикла. Своего состояния, как правило, *не имеет* — пользуется данными уровней ниже.

Разный темп изменения (потенциальный — редко, операционный — постоянно, регламентный — раз в годы, решений — без состояния) полезен для проектирования схемы БД и оптимизации запросов.

Финансы: «Потенциального» уровня нет; самый нижний — **Операционный** (финансовые операции). Добавлен **Обязательств** — то, чем организация реально торгует (кредит выглядит как «продажа денег», а на деле — *обмен обязательствами*): договора, сделки. **Регламентный** и **Принятия решений** — как в производстве.

Приём: уровни как reference-архитектуры [+]

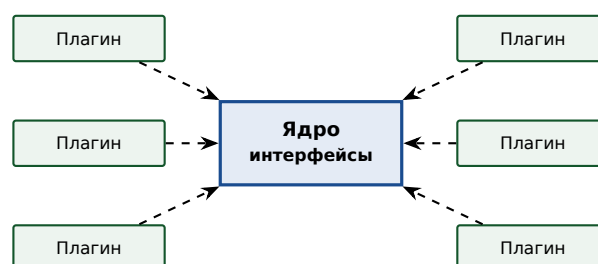
Это предметные **reference-архитектуры** (вопрос про стили). Сегодня: *регламентный* уровень часто реализуют движком правил (Drools) или **DMN** (Decision Model and Notation); *уровень решений* — аналитика, BI, ML. Принцип «разный темп изменения» жив как «rate-of-change»-разделение (конфигурация vs данные), а сам регламентный уровень нередко строят как «Уровень знаний» (следующий вопрос).

6. Стили «Уровень знаний», «Подключаемые компоненты»

Кроме уровневой, бывают и другие высокоуровневые структуры; «тысячи их». Две важные:

Определение: подключаемые компоненты

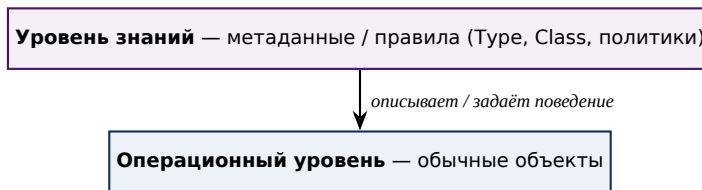
Подключаемые компоненты (Pluggable Component Framework) — есть **ядро** системы и плагиновая система, динамически подключающая плагины через чётко определённые интерфейсы; плагины реализуют большую часть полезной функциональности.



Примеры: VS Code, отчасти IDEA; браузеры (ограниченно); даже Linux — простое ядро + модули через чёткие интерфейсы.

Определение: уровень знаний

Уровень знаний (Knowledge Level) — система содержит *данные* операционного уровня и *метаданные* (данные о данных) на уровне знаний. Объекты уровня знаний определяют, *что и как* можно делать с объектами операционного уровня.



Аналогия — рефлексия: на уровне знаний объекты Type / Class, на операционном — обычные объекты. Уровень знаний можно *редактировать в рантайме* — и операционный начинает вести себя иначе (визуальное метамоделирование; финансовые продукты — хитрые правила начисления процентов меняет даже финансист, не умеющий программировать). *Минус*: логика уровня знаний *не проверяется компилятором* — чрезмерное увлечение (вся система как правила в XML) превращает поддержку в кошмар.

Важно: крупномасштабная структура у системы одна, но стили не исключают друг друга — например, уровневая структура для системы в целом, «Уровень знаний» для одного уровня и плагины для другого.

Приём: эти стили сегодня [+]

Подключаемые компоненты — это *микроядро* / *плагиновая архитектура*: расширения VS Code, OSGi, Eclipse RCP, плагины Backstage. **Уровень знаний** — движки правил (Drools), **BPMN / DMN**, *feature flags* (LaunchDarkly), low-code / no-code платформы, конфигурация-как-данные. «Финансист меняет правила» — ровно сегодняшний no-code; минус тот же — нет проверки типов, ошибки в рантайме.

Итоговый чек-лист

- ☐ Много команд ⇒ разные видения. Крайности (цельная модель / фрагментация) плохи. Баг «Платёж»: один термин — два смысла.
- ☐ **Ограниченный контекст** — одна модель, один язык, одна команда; следует орг-структуре. **Непрерывная интеграция** внутри (слияние/часы, тесты/коммит). **Карта контекстов** — трансляция понятий. [+] закон Конвея, Team Topologies, Context Mapper.
- ☐ 7 паттернов интеграции: Общее ядро, Заказчик-поставщик, Конформист, Предохранительный уровень, Раздельное существование, Открытый протокол, Общедоступный язык. Справа-вверху интеграция, слева-внизу «всё плохо». [+] OpenAPI / AsyncAPI, Strangler Fig + ACL.
- ☐ **Смысловое ядро** — know-how, маленькое и проработанное. **Дистилляция**: Domain Vision Statement, Выделенное ядро (документ / Flagged Core), вынос Generic Subdomains и Cohesive Mechanism. **Абстрактное ядро** — только абстракции. [+] Core / Supporting / Generic, buy-vs-build.
- ☐ **Крупномасштабная структура** — стиль + ограничения; эволюционирует; не «башня из слоновой кости»; носитель — единый язык. **Метафора системы** помогает, но тащит лишний смысл. Уровни — осмысленные, не топосортировка.
- ☐ Типичные уровни. *Производство*: Потенциальный / Операционный / Регламентный / Принятия решений (разный темп изменения → схема БД). *Финансы*: без потенциального, + Обязательств («обмен обязательствами»).
- ☐ **Подключаемые компоненты** — ядро + плагины (VS Code, Linux). **Уровень знаний** — метаданные задают поведение операционного уровня (рефлексия, правила в рантайме); минус — нет проверки компилятором. Структура одна, но стили комбинируются.

Лекция 12. Распределённые приложения: технические вопросы

Рабочая тетрадь

Лекция — технический фундамент распределённых систем: от природы распределённости и классических заблуждений через механизмы удалённого вызова (RPC / RMI, gRPC поверх protobuf) и веб-сервисы (SOAP + WCF, REST + ASP.NET Web APIs) к очередям сообщений (RabbitMQ). Все вопросы билета раскрыты лекцией. Комментарии в листингах — по-английски. Метка [+] помечает то, чего в лекции нет (актуальные версии и инструменты на 2026 год).

1. Понятие распределённой системы, заблуждения при проектировании

Определение: распределённая система

Распределённая система — приложение, компоненты которого находятся в компьютерной сети и взаимодействуют через обмен сообщениями, работая с общими ресурсами. Нужна для доступа к общим ресурсам нескольким пользователям (или одному с разных устройств) и для эффективной распределённой обработки и хранения.

Чем принципиально отличается от «обычного» приложения:

- **Принципиально параллельна** — причём *многопроцессно*: нет общей памяти, а расходы на коммуникацию велики.
- **Независимые отказы**: каждый компонент — отдельный процесс, падает независимо; плюс сеть ненадёжна (временные отказы).
- **Нет единого времени**: единую последовательность событий установить нельзя. Пакеты приходят в разном порядке, в разное время или не приходят вовсе — и это *норма*, не ошибка. Разные компоненты видят действия в разном порядке ⇒ гонки большого масштаба, чем в многопоточности.

Осторожно: 8 заблуждений распределённых вычислений (L. P. Deutsch)

Заблуждение	Реальность
Сеть надёжна	Теряет соединение даже в норме — надо восстанавливаться после сбоев
Задержка нулевая	Нельзя пилить монолит на сервисы и слать запрос «в цикле до миллиона»
Пропускная способность бесконечна	Гонять гигабайтные файлы между сервисами — безумие
Сеть защищена	Навредит даже забытый червь в локалке, без злого умысла
Топология не меняется	...пока уборщица не выдернет кабель из маршрутизатора
Администратор один	Права и открытые порты у всех разные (школьный ПК без прав на запись)
Передача данных бесплатна	Оборудование, трафик, аренда инфраструктуры — в смету
Сеть однородна	У кого-то мобильный или спутниковый интернет с иными свойствами

Способы организовать взаимодействие компонентов: **межпроцессное** средствами ОС (пайпы — если на одной машине, мало распределённый сценарий); **удалённый вызов** по сети («запрос-ответ», RPC, RMI); **неявное взаимодействие** (очереди сообщений, шины — вызывающий не знает вызываемого). Размещение: обычно сервисы по нескольким серверам (клиент → точки входа → десятки-сотни сервисов; плюс балансировщики, автомасштабирование, мониторинг). Альтернативы — **мобильный код** (клиент получает код на исполнение, как Google Docs) и **мобильный агент** (скачиваемая программа; P2P-вычисления, IoT — «код к данным» проще, чем данные к коду). Активно используется **кеширование** (Redis).

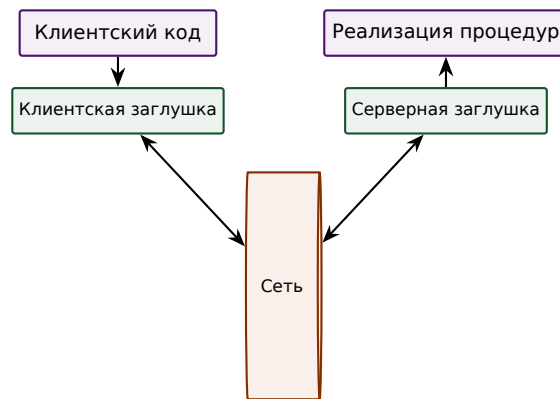
Приём: жить с ненадёжной сетью сегодня [+]

Заблуждения по-прежнему канон (восходят к Дойчу и Гослингу, Sun). Современные средства борьбы: **экспоненциальная выдержка** + **jitter** (случайный разброс, чтобы клиенты не ретраили синхронно), **circuit breaker** и таймауты (библиотеки Polly в .NET, resilience4j в Java), **ключи идемпотентности** для безопасных повторов. Сквозные задачи (ретраи, mTLS, трассировка) выносят в **service mesh** (Istio, Linkerd) и **OpenTelemetry**.

2. RPC, RMI. Пример: gRPC

Определение: RPC и RMI

RPC (Remote Procedure Call) — удалённый вызов процедуры, выглядящий как локальный. Программист пишет клиент, реализации процедур на сервере и *языконезависимое описание интерфейса*; по нему система генерирует *заглушки* (stubs). **RMI** (Remote Method Invocation) — тот же RPC, но знающий про ООП (исключения, иногда распределённые ссылки на объекты, передаваемые как параметры).



Программист пишет только верхние блоки; заглушки генерируются по описанию интерфейса и берут на себя сериализацию параметров и сетевое соединение.

Прозрачность под вопросом. Цель первых RPC — *location and access transparency* (всё равно, где объект). Но удалённый вызов: (1) уязвим к отказам — сервиса (критично: диагностика, аккуратное завершение) и сети (надо просто обрабатывать, стратегия **exponential backoff** — удваивать паузу между повторами); (2) *не мгновенен* — проектируем *асинхронно*, с отменой. Поэтому советуют явно *маркировать* удалённые вызовы и указывать **семантику исполнения**: *at least once* (можно повторять — для запросов без изменения состояния), *at most once* (не более раза, потеря допустима), *exactly once* (ровно один раз — труднее всего в ненадёжной сети).

protobuf и gRPC

Protocol Buffers (protobuf) — бинарный формат сериализации от Google; до $\sim 10\times$ компактнее XML (хитрая упаковка, Base 128 Varints), но сообщение почти не содержит информации о структуре — *нужна схема*. Схема — декларативный .proto-файл; на сборке protoc генерирует код. У каждого поля есть *номер* — для эволюции формата и обратной совместимости (сервер игнорирует незнакомые поля). Умеет enum, repeated (списки), вложенные сообщения, oneof.

```

syntax = "proto3";
message Person {
  string name = 1; // field numbers, not order, define the wire format
  int32 id = 2;
  string email = 3;
}
  
```

gRPC работает поверх protobuf (реализован как расширение protoc). Описывает *сервис*: методы, типы параметра и результата (protobuf-сообщения). Поддерживает 4 режима передачи:

Режим	Сигнатура	Смысл
Unary (запрос-ответ)	(Point) returns (Feature)	одно сообщение туда, одно обратно
Server streaming	(Rectangle) returns (stream Feature)	сервер пишет в канал поток ответов
Client streaming	(stream Point) returns (RouteSummary)	клиент шлёт поток, сервер — один ответ
Bidirectional	(stream RouteNote) returns (stream RouteNote)	обе стороны пишут независимо

```

service RouteGuide {
  rpc GetFeature(Point) returns (Feature) {}
  rpc ListFeatures(Rectangle) returns (stream Feature) {}
  rpc RecordRoute(stream Point) returns (RouteSummary) {}
  rpc RouteChat(stream RouteNote) returns (stream RouteNote) {}
}
  
```

Сервер наследует сгенерированную заглушку RouteGuideImplBase и переопределяет методы. Ответы отдаются через StreamObserver (паттерн «Наблюдатель»: onNext, onComplete, onError). Для unary — один onNext + onComplete; для server-streaming — onNext в цикле, затем onComplete:

```

@Override
public void getFeature(Point request, StreamObserver<Feature> obs) {
  obs.onNext(checkFeature(request)); // one response message...
  obs.onCompleted();                // ...then signal end of stream
}

@Override
public void listFeatures(Rectangle req, StreamObserver<Feature> obs) {
  for (Feature f : features)
    if (inside(f, req)) obs.onNext(f); // many, possibly spaced in time
  obs.onCompleted();
}
  
```

На клиенте генерируются *синхронная* (BlockingStub) и *асинхронная* (Stub) заглушки; соединение строит библиотечный ManagedChannelBuilder. Наследоваться не надо — зови методы заглушки, будто сервис локальный.

Приём: RPC сегодня [+]

gRPC — рекомендованная Microsoft замена WCF, работает поверх **HTTP/2**; для браузеров (нет сырого HTTP/2) — **gRPC-Web** или протокол **Connect**. В C# актуален **grpc-dotnet** (Grpc.Net.Client / Grpc.AspNetCore); старый Grpc.Core — EOL. У protobuf теперь **editions** вместо деления proto2 / proto3. Конкуренты формата: Apache Thrift, Cap'n Proto, FlatBuffers, Avro (со schema registry в экосистеме Kafka).

3. Веб-сервисы, SOAP. WCF

Определение: веб-сервис

Веб-сервис — отдельная система, предоставляющая сетевой интерфейс по стандартному протоколу в стиле RPC / RMI, *языконезависимая*. Светлая цель — собирать приложения из веб-сервисов «как автомобили из запчастей» (не сбылась, как и компонентная модель). Сегодня почти любое распределённое приложение — набор веб-сервисов (своих и сторонних), часто сотни.

Технически: «запрос-ответ» поверх HTTP, человекочитаемая сериализация (XML или JSON); все запросы асинхронны, каждый сервис сам обрабатывает временные отказы и ошибки соседей («каждый сам за себя»). Часто между сервисами вставляют очереди (тогда это уже событийная архитектура). XML — в старых системах (SOAP, XML-RPC), JSON — в современных (REST).

Определение: SOAP

SOAP — протокол передачи XML-сообщений (из XML-RPC, 1998) поверх транспорта (обычно HTTP, но можно SMTP, FTP). Сообщение — *конверт* (Envelope) с *Header* (метаинформация) и *Body*. Стек: SOAP + **WSDL** (XML-описание возможностей сервиса, аналог .proto, стандарт W3C) + **UDDI** (глобальный реестр сервисов — умер в начале 2010-х вместе с идеей сборки из чужих сервисов).

UDDI — реестр

WSDL — описание

SOAP — конверт XML

HTTP / SMTP / FTP

```
<env:Envelope xmlns:env="http://www.w3.org/2003/05/soap-envelope">
  <env:Header>
    <!-- metadata: priority, TTL... -->
    <n:alertcontrol xmlns:n="http://example.org/alertcontrol">
      <n:priority>1</n:priority>
    </n:alertcontrol>
  </env:Header>
  <env:Body>
    <!-- the only payload is m:msg -->
    <m:alert xmlns:m="http://example.org/alert">
      <m:msg>Get up at 6:30 AM</m:msg>
    </m:alert>
  </env:Body>
</env:Envelope>
```

+ Достоинства

Генерация клиентских заглушек по WSDL; генерация WSDL по коду
Валидация в три слоя: парсер XML, XSD-схема, сам сервис
Транспорт HTTP — всё человекочитаемо, смотрится в браузере
Главное — *стандартный*, его все умеют

– Недостатки

Огромный размер сообщений (пространства имён, обязательные теги)
Без инструментов WSDL руками писать тяжело
Нет транзакций; модель «запрос-ответ» беднее, чем стримы gRPC
Тяжело мигрировать клиенты (решают версиями на разных эндпойнтах)

WCF (Windows Communication Foundation) — фреймворк .NET для веб-сервисов; конфигурируемый из трёх компонентов — **ABC**: **Address** (эндпойнт, URL), **Binding** (транспорт, сериализация, безопасность), **Contract** (исполнение). Контракт размечают атрибутами; по ним генерится WSDL:

```
[ServiceContract(Namespace = "http://Microsoft.ServiceModel.Samples")]
public interface ICalculator {
    [OperationContract] double Add(double n1, double n2);
    [OperationContract] double Subtract(double n1, double n2);
}
// hosting: ServiceHost is an in-process web server (no external IIS needed)
// client: svcutil generates a proxy; endpoint A/B/C lives in app.config
```

Осторожно: WCF устарел [+]

WCF *глубоко* устарел с .NET 5 и отсутствует в современном .NET (6+ / .NET 10) на стороне сервера — жив только на .NET Framework 4.8.x для legacy. **CoreWCF** (community + Microsoft + AWS, 2022) — лишь *мост* для миграции SOAP, подмножество WCF, для новых проектов не рекомендован. Замена от Microsoft — **gRPC** или **ASP.NET Core Web API (REST)**. Миграция: Strangler + API-gateway (YARP), FaultException → статус HTTP / gRPC, DataContract → JSON / protobuf, WS-Security → OAuth2 / OIDC / mTLS. Сам **SOAP** жив во внутрикорпоративных системах (банки, госсектор).

4. Веб-сервисы, REST. ASP.NET Web APIs

Определение: REST

REST (Representational State Transfer, Roy Fielding, диссертация 2000) — представление состояния передаётся сервер↔клиент, и именно оно *полностью* определяет состояние протокола: сервер **не хранит** информацию о клиентской сессии (stateless). Архитектурно сервис — конечный автомат: по запросу и состоянию с клиента выполняет переход и возвращает следующее состояние. Всю сессию хранит клиент ⇒ сервису всё равно, с одной машины с ним общаются или с десятка.

REST требует активного использования семантики HTTP; запросы намеренно просты (часто хватает GET из адресной строки), параметры — в URL, результат — код возврата HTTP, тело — обычно JSON (иногда protobuf). Statelessness + параметры в URL ⇒ мощное **кеширование** (критично для сотен микросервисов). Всё строится вокруг **ресурсов**, объединённых в **коллекции**; URL указывает коллекцию / id, HTTP-метод — действие.

Метод	Для коллекции	Для элемента
GET	список ресурсов коллекции	подробности об элементе
POST	создать новый элемент (из тела)	неприменимо
PUT / PATCH	обновить элементы (PUT — целиком, PATCH — часть полей)	обновить элемент, если существует
DELETE	удалить всю коллекцию	удалить элемент

Коды ответа: 200 ок (тело — результат запроса), 201 ресурс создан, 204 ок без тела (для команд), 404 не найдено, 400 неверный запрос, 409 конфликт (нарушение целостности). Пример — Google Drive REST API: GET /drive/v3/files (список), GET /drive/v3/files/{id} (метаданные), POST (загрузить), PATCH (обновить), DELETE (удалить). Данные запроса — прямо в URL (?key=...&fields=*).

ASP.NET Web APIs — серверный фреймворк .NET для REST. Поддерживает **минимальные API**: MapGet отображает роут (путь URL) на лямбду-обработчик — весь сервис уместается в одном файле.

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();
// map a route (relative URL) to a handler lambda; result is JSON, status 200
app.MapGet("/weatherforecast", () =>
    Enumerable.Range(1, 5).Select(i =>
        new WeatherForecast(DateTime.Now.AddDays(i),
            Random.Shared.Next(-20, 55), "Mild")).ToArray());
app.Run();
// C# record == data class with auto-generated members
internal record WeatherForecast(DateTime Date, int TempC, string? Summary);
```

Осторожно: контроллер — не свалка

Схема «контроллер → Data Access Layer» годится лишь для простейших сервисов и противоречит DDD. Правильно: контроллер — только *средство доставки* (принять запрос, десериализовать, передать доменной модели через адаптеры). Различайте модели: *представления* (DTO для сериализации), *доменную* (Model в MVC) и *персистентности* (с атрибутами ORM).

Приём: REST сегодня [+]

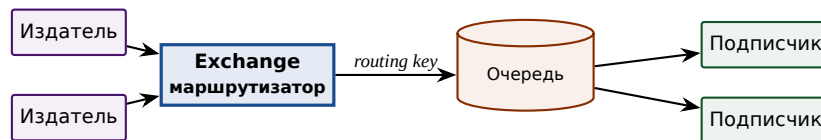
REST остаётся стандартом де-факто для публичных API. Контракт и документацию описывают **OpenAPI** (бывший Swagger) — генерация встроена в ASP.NET (с .NET 9). Альтернативы: **GraphQL** (один эндпойнт, клиент сам выбирает поля — против *over-fetching*), **gRPC** (для внутренних вызовов), **trRPC** (для TypeScript). Транспорт — **HTTP/2**

и **HTTP/3** (поверх QUIC). Минимальные API стали мейнстримом; поверх — идиоматичность, ETag-кеширование, OAuth2 / OIDC.

5. Очереди сообщений. RabbitMQ, Apache Kafka

Определение: очередь сообщений

Очередь сообщений — прослойка между клиентом и сервером, обеспечивающая *гарантированную доставку*, даже если отправитель и получатель онлайн в разное время. Отправитель не шлёт серверу напрямую, а кладёт сообщение в очередь; получатель забирает, когда готов. Снимает связь по времени, допускает много отправителей и получателей («издатель-подписчик», шина событий), умеет маршрутизацию, фильтрацию и преобразование (разветвители, агрегаторы, переупорядочиватели).



RabbitMQ

Отдельный сервер (на Erlang), протокол **AMQP**, «кролик». Сообщение публикуется не в очередь, а в **exchange** — именованный маршрутизатор, который по умолчанию шлёт в одноимённую очередь (можно не знать про exchange вовсе), но умеет раскидывать по ключам, фильтровать (типы: direct, topic, fanout, headers). Получатель забирает сообщение и (обычно) удаляет его. Параметры очереди: durable (хранить на диск), exclusive, autoDelete.

```
// --- producer ---
var channel = connection.CreateModel(); // virtual AMQP connection
channel.QueueDeclare("hello", durable: false, exclusive: false,
    autoDelete: false, arguments: null);
var body = Encoding.UTF8.GetBytes("Hello World!");
channel.BasicPublish(exchange: "", routingKey: "hello",
    basicProperties: null, body: body);

// --- consumer ---
var consumer = new EventingBasicConsumer(channel);
consumer.Received += (model, ea) =>
{
    Console.WriteLine(Encoding.UTF8.GetString(ea.Body)); // deserialize + use
    channel.BasicConsume("hello", autoAck: true, consumer: consumer);
}
```

Apache Kafka

Та же ниша, но иной подход: Kafka — не очередь, а **распределённый лог событий**. Сообщение *записывается* и хранится, пока политика чистки не удалит (по умолчанию ~7 дней, можно вечно), а не вычитывается-и-удаляется. Каждый подписчик читает лог последовательно и сам хранит **индекс** (offset) последнего прочитанного (на сервере, но привязан к получателю) — можно *отмотать назад* и переиграть события (чего нет в RabbitMQ; удобно перезаполнить БД).

Архитектурно — набор **брокеров**, каждый хранит свою часть лога на диске (Kafka рассчитывает на кеш ФС, не кеширует в RAM). Лог делится на **топики** (каналы), топика — на **разделы** (partitions). Ключ сообщения определяет раздел по хешу (одинаковые ключи → один раздел). «Группа подписчиков» координируется так, чтобы событие дошло только одному участнику группы (балансировка). Практически немутабельный лог + распределённость ⇒ высокая скорость и естественная поддержка **Event Sourcing** (хранить не состояние, а последовательность событий) — идеально для IoT и потоков.



```
// --- producer ---
using var p = new ProducerBuilder<Null, string>(config).Build();
await p.ProduceAsync("test-topic", new Message<Null, string> { Value = "test" });
// --- consumer: needs a group id + offset reset policy ---
var conf = new ConsumerConfig { GroupId = "test-consumer-group",
    BootstrapServers = "localhost:9092", AutoOffsetReset = AutoOffsetReset.Earliest };
using var c = new ConsumerBuilder<Ignore, string>(conf).Build();
c.Subscribe("my-topic");
while (true) { // poll loop, cancellable via token
    var r = c.Consume(cts.Token);
    Console.WriteLine(r.Message.Value);
}
```

	RabbitMQ	Apache Kafka
Модель	очередь: прочитал — удалил	лог: запись хранится, читается по offset
Переигрывание	нет	да (отмотать offset)
Сила	гибкая маршрутизация, надёжная доставка <i>одному</i>	огромный поток, агрегация, Event Sourcing
Где	информационные системы (событий немного)	IoT, потоки (сотни тысяч событий/мин)

Приём: брокеры в 2026 [+]

Kafka 4.0 (март 2025) *полностью убрала ZooKeeper* — координацию ведёт встроенный **KRaft** (Raft); это та самая обещанная в лекции цель. Добавлены **Share Groups** («Queues for Kafka», KIP-932) — очередная семантика, конкурирующая с RabbitMQ; брокеру нужен Java 17. **RabbitMQ 4.x** обзавёлся нативными *streams*, **AMQP 1.0** и MQTT 5.0 — системы сближаются. Прочие брокеры: **NATS**, **Redis Streams**, облачные (SQS, Google Pub/Sub, Azure Service Bus). Рядом — schema registry (Avro), dead-letter-очереди и паттерн **Outbox** для надёжной публикации событий из транзакции БД.

Итоговый чек-лист

- ☐ Распределённая система — компоненты в сети, обмен сообщениями. Отличия: многопроцессность без общей памяти, *независимые отказы*, нет единого времени (гонки). 8 заблуждений (сеть надёжна, задержка/расходы нулевые, ...). [+] backoff + jitter, circuit breaker, service mesh.
- ☐ **RPC** — удалённый вызов как локальный (заглушки по описанию интерфейса); **RMI** — + ООП. Маркировать удалённое; семантика at-least / at-most / exactly once; backoff. **protobuf** — бинарная схема (.proto, номера полей); **gRPC** — 4 режима (unary, server/client-stream, bidi), StreamObserver. [+] HTTP/2, gRPC-Web, grpc-dotnet.
- ☐ **Веб-сервис** — сетевой интерфейс по стандартному протоколу. **SOAP** — XML-конверт (Header + Body) + WSDL + UDDI(мёртв); валидация в 3 слоя, но огромен. **WCF** — ABC (Address, Binding, Contract). [+] WCF устарел → gRPC / ASP.NET Core; CoreWCF — мост.
- ☐ **REST** (Fielding, 2000) — stateless, представление состояния в URL + теле (JSON); ресурсы и коллекции, семантика HTTP-методов и кодов; кеширование. **ASP.NET Web APIs** — минимальные API (MapGet → лямбда); контроллер — средство доставки. [+] OpenAPI, GraphQL, HTTP/3.
- ☐ **Очередь сообщений** — гарантированная отложенная доставка, развязка по времени. **RabbitMQ** — exchange → очередь (AMQP, прочитал-удалил, гибкая маршрутизация). **Kafka** — распределённый лог (топики / разделы / offset, переигрывание, Event Sourcing). [+] Kafka 4 — KRaft без ZooKeeper; RabbitMQ 4 — streams, AMQP 1.0.

Лекция 13. Распределённые приложения: архитектура

Рабочая тетрадь

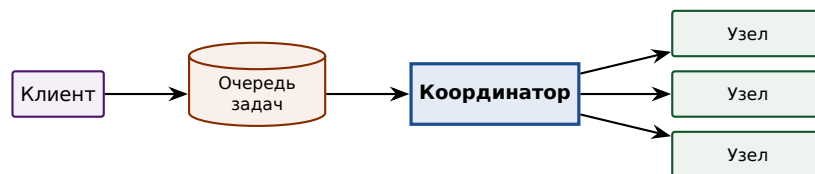
Лекция — «стратегические» аспекты больших облачных приложений: типовые архитектурные стили (Big Compute, Big Data, Web-queue-worker, N-звенная, микросервисы), хорошие практики REST-API и общие принципы дизайна (самовосстановление, Circuit Breaker, избыточность, минимизация координации, CQRS, CAP / ACID / BASE, проектирование для обслуживания). Изложение — по гайдлайнам Azure Architecture Center без вендор-специфики. Все вопросы билета раскрыты лекцией. Метка [+] помечает то, чего в лекции нет (актуальные практики и инструменты на 2026 год).

1. Архитектурные стили: Big Compute, Big Data

Все эти стили — подвиды слоистого или событийно-ориентированного, но в жизни используются более специфичные. Ниже — по возрастанию «обычности» для облака.

Определение: Big Compute

Большие вычисления — для задач массовой параллельности (физические / химические расчёты, симуляция, огромные дифуры). Архитектура проста: клиент → очередь задач → **координатор** (самая сложная часть: делит задачу на подзадачи) → **вычислительные узлы** (десятки–десятки тысяч) → агрегация результата.



Когда брать: *embarrassingly parallel* задача (в идеале — Map-Reduce; узлы считают без координации → нужен мощный процессор, но почти не нужна сеть). Если нужна координация узлов — требуется *высокопроизводительная сеть* (вычислительный кластер). Стил специфичен (сложные вычисления редки и дороги), но часто иначе задачу не решить — только грубой силой.

Определение: Big Data

Большие данные — обратная сторона Big Compute: вычисления просты, но данных очень много (не лезут в обычную СУБД). Данные импортируются из многих источников, далее два пути: **на лету** (поток → очередь Kafka как буфер + балансировщик → потоковые обработчики → аналитическое хранилище) или **пакетно** (распределённое хранилище → периодическая обработка).

Ключевые идеи: распределённое хранение и обработка — *не писать с нуля* (нужна серьёзная теория: отказоустойчивость, безопасность), брать готовое: **Hadoop** (HDFS + MapReduce, для пакетной обработки), **Spark** (в памяти, быстрее, для реального времени). **Schema-on-read** — хранить данные как пришли (сырые логи, документы), преобразовывать при чтении (термин «Data Lake»; антипод — «Data Swamp»). Вместо ETL — **TEL** (обработка на стороне хранилища). *Личные данные удалять как можно раньше* (украдут — пусть никто не пострадает). Пример — IoT: устройства → Field Gateway (локальная обработка) → Cloud Gateway → потоковая обработка (+ ML для детекции аномалий).

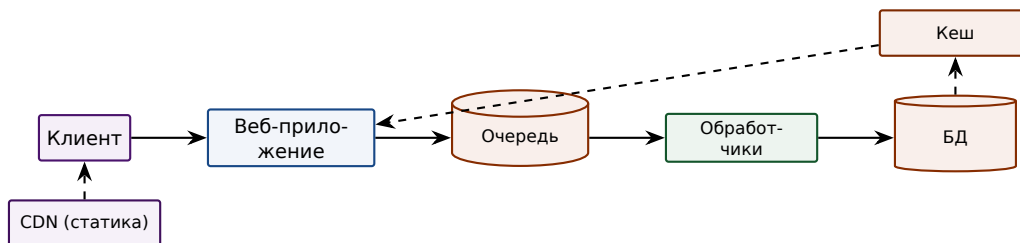
Приём: большие данные сегодня [+]

Big Compute в облаке — managed-сервисы (AWS Batch, Azure Batch) + MPI / GPU-кластеры. В Big Data **Hadoop** уступил место **Spark** (часто на Databricks) и **Flink** (стриминг); вместо разделения «озеро vs склад» — **Data Lakehouse** с ACID прямо на озере (**Apache Iceberg**, Delta Lake, запросы через Trino). Schema-on-read и ELT остаются базой.

2. Web-queue-worker, N-звенная архитектура

Определение: Web-queue-worker

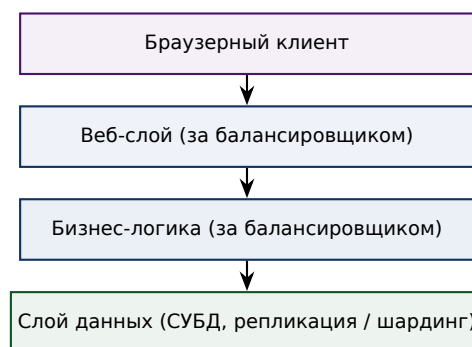
Уточнение событийного стиля для *вычислительно сложных* задач при несложной предметной области (пакетная обработка, конвертация PDF). Клиент (браузер) → **веб-приложение** (принимает работу) → **очередь** → **обработчики** → **БД** (через **кеш**). Плюс внешние веб-сервисы и **CDN** для статики.



Тонкости: очередь и обработчик *не обязательны* (без сложных вычислений — обычная трёхзвенка); но очередь — буфер пиков + балансировщик, а разделение веб-части и обработчиков даёт *независимое масштабирование*. Запросы *асинхронны*. Готовый стек (ASP.NET / Spring + RabbitMQ + MariaDB + Redis + CDN) собирается за выходные. Минус — при росте сложности веб-часть или обработчик разрастается в «ком грязи».

Определение: N-звенная архитектура

Браузерный клиент → **веб-слой** → слой **бизнес-логики** (сколь угодно сложный) → **слой данных** (+ внешние сервисы, кеш на чтение). Особенность: компоненты физически на *разных машинах* → масштабируемость и надёжность. Каждый слой — пул машин за **балансировщиком** (производительность, надёжность, авто-масштабирование по метрикам).



Когда брать: есть монолит, который надо с минимумом усилий перенести в облако; бизнес-логика несложна, но требует структуры (Web-queue-worker не подойдёт). **Минусы:** масштабировать можно только *крупные куски* (один редкий метод тащит за собой целый сервер бизнес-логики); слой бизнес-логики склонен пухнуть в «ком грязи». На Azure: балансировщик → NVA в DMZ (файервол) → виртуальная сеть (веб-пул / бизнес-пул / кластер СУБД), админ-доступ через Bastion.

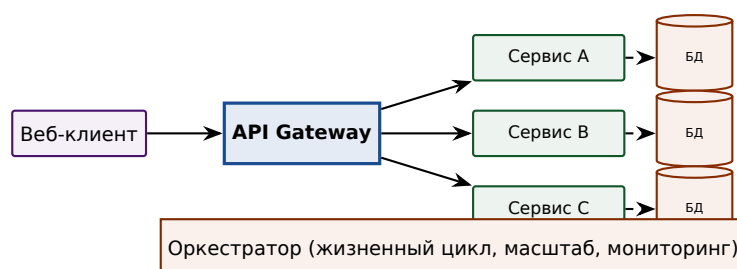
Приём: эти стили сегодня [+]

Web-queue-worker часто делают *бессерверным*: обработчик — функция (FaaS, AWS Lambda / Azure Functions), очередь — managed (SQS, Service Bus), масштабируется до нуля. **N-звенная** в облаке — это контейнеры за managed-балансировщиком или PaaS (App Service); стиль жив для переноса монолитов «как есть» (lift-and-shift).

3. Микросервисная архитектура

Определение: микросервисы

Приложение делится на маленькие *отдельные веб-сервисы*, каждый — размером «одна команда за обозримый срок», со *своей* моделью предметной области (это **ограниченный контекст** из DDD). Приложение собирается из микросервисов, вызывающих друг друга. Клиент → **API Gateway** (единый фасад, без бизнес-логики) → микросервисы в виртуальной сети; **оркестратор** (Kubernetes, Docker Compose) управляет жизненным циклом, масштабированием, балансировкой, мониторингом.



Особенности (плюсы): свой язык / стек у каждого (но «зоопарк» усложняет инфраструктуру); каждый *хранит свои данные сам* (не делит схему — как объект в ООП: состояние + поведение через публичный интерфейс); *независимое*

масштабирование (оркестратором, в контейнерах — дёшево); *независимый деплой* (релизы не согласуют → несколько релизов в день, «коммит сразу в production»); *независимые отказы* (локализуются); маленькая кодовая база.

Осторожно: за всё надо платить

Сложность переезжает с кода на оркестрацию (а её инструменты ещё незрелы): неочевидные зависимости, трудно отлаживать / мониторить, нужна культура DevOps. Само приложение *сложнее* эквивалента (оверхед на инфраструктуру); разработка / тестирование — mock-и или запуск всех зависимостей. Требовательны к сети (локальные вызовы становятся сетевыми — *нельзя сделать каждый класс микросервисом*). Целостность данных не гарантируют — строят на **eventual consistency**.

Приём: микросервисы сегодня [+]

Маятник «микросервисы ↔ монолит» качнулся назад: по опросу CNCF (2025) около 42 % команд *вернули* часть сервисов в более крупные единицы; популярен **модульный монолит** (порог выгоды микросервисов — примерно от 50 инженеров; знаменит откат Amazon Prime Video в 2023). Инфраструктура повзрослела: **service mesh** (Istio, Linkerd) выносит ретрай / mTLS / трассировку из кода; API-шлюзы (Kong, Envoy); паттерны **BFF** (backend-for-frontend) и **sidecar**.

4. Хорошие практики проектирования REST-интерфейсов

REST любим микросервисами: простая коммуникация, малая нагрузка на сеть (слабое место стиля). Рекомендации, чтобы API был *действительно* REST и удобен:

- **Вокруг ресурсов, а не действий.** /customers/ — хорошо, /get_customer/ — плохо (действие — это HTTP-метод, не элемент URL).
- **Отношения — ресурсы в URL:** /customers/5/orders. Но не увлекаться: /customers/1/orders/99/products — уже плохо. Нужно больше одного перехода — пусть клиент делает несколько запросов (получил Id заказа → запрос к /orders).
- **API — модель предметной области, не данных.** Как в ООП, не раскрывать детали реализации (не отображать запросы прямо в таблицы БД).
- **Стандартная семантика HTTP** — заголовки Content-Type (формат тела) и Accept (желаемый формат ответа); стандартные коды ошибок.
- **Фильтрация и пагинация** для больших коллекций: какие элементы и какие поля вернуть (в Google Drive — fields), порциями (pageSize, pageToken), сортировка (orderBy).
- **Partial Content** для больших элементов: HEAD возвращает длину, клиент шлёт GET с заголовком Range (докачка, отмена).
- **HATEOAS** — сервис сообщает о доступных дальше операциях (переходы автомата REST) ссылками в ответе. Но: никто толком не поддерживает, формат не стандартизован.
- **Версионирование** — старые клиенты должны продолжать работать; обычно все версии запущены параллельно (/v3/ в URL — не лучший, но частый способ).

```
GET http://api.example.com/orders/2 HTTP/1.1
Accept: application/json
--- HATEOAS: the response advertises what you can do next ---
{ "orderId":3, "productId":2, "quantity":4,
  "links":[ { "rel":"customer",
              "href":"http://api.example.com/customers/3",
              "action":"GET", "types":["application/json"] } ] }
```

Приём: REST-API сегодня [+]

Контракт и документацию задают через **OpenAPI** (бывший Swagger) — из него генерируют клиентов и серверные заглушки. Для больших коллекций предпочитают **cursor-пагинацию** (стабильнее offset при изменяющихся данных). HATEOAS так и остался редким; где гипермедиа нужно — смотрят на JSON:API. Где REST тесен — **GraphQL** (клиент сам выбирает поля, против over-fetching) или **gRPC** (внутренние вызовы).

5. Принципы дизайна: самовосстановление

Сеть ненадёжна ⇒ приложение проектируют с учётом отказов. Виды отказов — разные стратегии: *временные* (потеря пакетов / связи), *постоянные* (сервис не работает совсем), *перегрузка* (не успевает обрабатывать).

- **Повтор с экспоненциальным откатом** (временные отказы) — но надо отличать временный отказ от постоянного (неверный пароль повтором не лечится); поэтому это редко делают библиотеки (нужно знание области).
- **Методы самодиагностики:** от ping (ответил 200 / 204 — жив) до бизнес-статуса (длина очереди, среднее время обработки). Использует и приложение, и оркестратор (для масштабирования / перезапуска).

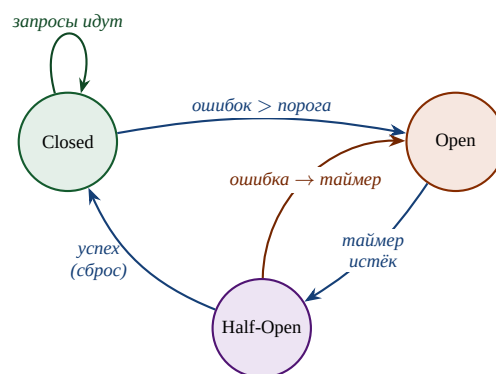
- **Изоляция ресурсов** (bulkhead): если один сервис сожрёт весь процессор, остальные должны жить — оркестратор лимитирует ресурсы.
- **Буферизация очереди** при неравномерной нагрузке («столовка в большой перерыв»): сглаживает пики; длину очереди *ограничивать* (лучше отказать части клиентов, чем уронить всё).
- **Переключение на резерв** (failover): минимум 2 экземпляра, авто-переключение; обратно — *вручную* (убедиться, что основной восстановился и не испортит данные). Резерв в *другом регионе* (экскаватор перерубил оптоволокно — сервера в другом месте работают), но это дорого.
- **Checkpoints** — промежуточное сохранение долгих операций (продолжить, а не начать заново).
- **Graceful degradation** — отказывать *плавно*: упал сервис картинок — показываем заглушки; упал сервис рекомендаций — работаем без него.
- **Намеренный ввод отказов**: раз в месяц «грохать» прод и поднимать из бэкапов (случайным членом команды — чтобы все умели); инструменты непрерывно убивают сервисы (**Chaos Monkey** от Netflix).

Приём: самовосстановление сегодня [+]

Chaos engineering повзрослел: Chaos Monkey (часть Simian Army, во многом ушёл в Spinnaker) сменили **Gremlin**, **Chaos Mesh**, **LitmusChaos**, **AWS FIS**, Chaos Toolkit; ключевой приём — малый *blast radius* + гипотеза + наблюдаемость. В Kubernetes самодиагностика — это **liveness / readiness / startup-пробы**. Повтор делают с *jitter* (случайный разброс), изоляцию — паттерном **Bulkhead**; готовые библиотеки — Polly (.NET), resilience4j (Java).

6. Паттерн «Circuit Breaker»

Если отказ вызван перегрузкой или временным выходом сервиса из строя, лавина повторов от всех клиентов *усугубит* ситуацию. **Circuit Breaker** (предохранитель) — прокси-объект с тремя состояниями:



- **Closed** — запросы идут сервису как обычно; ошибка увеличивает счётчик; превысил порог → *Open*.
- **Open** — запросы *не* идут сервису, предохранитель сам бросает исключение; запускается таймер, по истечении → *Half-Open*.
- **Half-Open** — пропускается *немного* запросов: успех → *Closed* (счётчик сброшен), ошибка → *Open* (таймер перезапущен).

Можно сочетать с повторами, но полезно дать коду повторов узнать, что цепь *разомкнута* — повторять уже бессмысленно.

Приём: Circuit Breaker сегодня [+]

В прикладном коде — готовые библиотеки **Polly** (.NET) и **resilience4j** (Java), где предохранитель комбинируют с *retry*, *timeout* и *bulkhead* в единую «политику устойчивости». В микросервисах его всё чаще делают на уровне инфраструктуры — **service mesh** (Istio / Envoy) размыкает цепь между сервисами без изменения их кода.

7. Принципы дизайна: избыточность

Лучший способ отказоустойчивости — **избыточность** (несколько копий каждого сервиса). Но она стоит денег, поэтому сначала — *бизнес-решение*: задать целевые показатели надёжности (баланс цены эксплуатации и выгоды):

Показатель	Смысл
RTO (Recovery Time Objective)	сколько максимум времени от <i>обнаружения</i> отказа до восстановления
RPO (Recovery Point Objective)	данные за какой период до отказа можно потерять («как часто жать Ctrl-S»)
MTO (Maximum Tolerable Outage)	сколько бизнес-процесс может быть недоступен без ущерба (успех — даже ручное исполнение)

Затем закладывают механизмы достижения:

- **Балансировщики нагрузки** — самый универсальный приём: stateless-сервис в 2–10 копий за балансировщиком, который перестаёт слать запросы упавшему (может и логику Circuit Breaker). Плюс к скорости (минус к цене).
- **Репликация БД** — периодические копии (в горячую СУБД или снапшоты); частота задаётся *RPO*. Обязательна, часто делается провайдером сама.
- **Разделение по регионам** — от падения самолёта на здание не спасут ни балансировщик, ни репликация. Гео-распределение + DNS (ближайший сервер) — надёжность и скорость, но очень дорого.
- **Шардирование** — размазать данные по нескольким БД. Аналогия RAID: репликация = **RAID 1** (копия для надёжности), шардирование = **RAID 0** (распределение для скорости — параллельные операции над шардами). Потеря шарда — потеря *части* данных (10 % лучше 100 %). Политика: по регионам, по первой букве, по хешу. Шардирование и репликация — *вместе*.

Приём: избыточность сегодня [+]

Облака дают избыточность из коробки: **multi-AZ** (зоны доступности в регионе) почти даром, **multi-region** — дорого, но для критичного. Managed-БД реплицируются и снапшотятся сами; авто-масштабирование по метрикам — штатно. **NewSQL** (Spanner, CockroachDB) даёт шардирование + репликацию + почти-ACID «под капотом», скрывая ручную работу.

8. Принципы дизайна: минимизация координации

Компоненты решают общую задачу совместно, но *слишком* тесная координация бьёт по производительности и надёжности. Приёмы борьбы:

- **Событийный стиль вместо прямых вызовов**: вместо вызова чужих методов — объявить **доменное событие** (domain event), на которое подписываются и обрабатывают асинхронно; источник даже не знает, интересно ли событие кому-то. Обычно для «дополнительных» сценариев (регистрация — прямой вызов, нотификация админу — событие). Библиотеки: **MediatR** (.NET).
- **Асинхронность** всех мультисервисных операций — иначе *каскад задержек* (один ждёт второго, второй третьего...).
- **Идемпотентные операции** — безопасно повторяются без изменения состояния (все запросы; «включить» / «выключить» — да; «переключить» — нет). При сбое другой обработчик берёт очередь первого и продолжает.
- **Шардирование** уменьшает координацию — запросы не толкаются вокруг одной БД.
- **Избегать распределённых транзакций**. Атомарность через несколько сервисов заманчива, но «никто не умеет»; современный подход — *избегать*. Лучше оптимистично выполнять локально, а при ошибке — **компенсационная транзакция** (вернуть как было). Семантика — **eventual consistency**.

Приём: координация сегодня [+]

Компенсационные транзакции + eventual consistency сегодня известны как **паттерн Saga** (Гарсиа-Молина, Салем, 1987): цепочка локальных транзакций, при сбое — компенсации в обратном порядке. Две формы: **хореография** (на событиях, через Kafka) и **оркестрация** (движок-дирижёр, напр. **Temporal**). Это альтернатива блокирующему **2PC** (двухфазный коммит почти никто не поддерживает). Надёжную публикацию событий из транзакции БД дают паттерн **Outbox** и ключи идемпотентности.

9. Command and Query Responsibility Segregation (CQRS)

Определение: CQS и CQRS

CQS (Command and Query Separation) — команда *меняет* состояние и *ничего не возвращает*, запрос *возвращает* состояние и *ничего не меняет* (хорошая практика ООП вообще). **CQRS** (... Responsibility Segregation) — эволюция: для команд и запросов можно держать *разные схемы и даже разные хранилища* (реляционная для чтения, другая для записи), что кратно ускоряет операции (у запросов нет конфликтов транзакций — они ничего не меняют). CQRS предполагает явные *доменные команды* с бизнес-логикой.



Поддержание двух баз в согласии непросто — используют **eventual consistency**. Сильно помогает **нечетность** данных (записали раз — всё): нет конфликтов записи, легко параллелить и хранить (как Kafka). Это — подход **Event Sourcing**: хранить не состояние, а *набор событий*, приведших к нему; текущее состояние строится из

начального + события. Для скорости — read-only снапшоты, обновляемые из потока (опять eventual consistency). Эффективно, когда событий и обработчиков много.

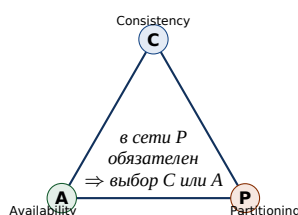
Приём: CQRS сегодня [+]

CQRS обычно не нужен всему приложению — его применяют точно, к контекстам с резко разным профилем чтения и записи. Read-store рассматривают как **проекцию** (read model), пересобираемую из событий. Event Sourcing + CQRS поддерживают фреймворки **Axon** (Java) и **EventStoreDB**, часто в паре с Kafka как журналом событий.

10. CAP-теорема, модели ACID и BASE

Определение: CAP-теорема

В распределённой системе можно обеспечить не более двух из трёх: **Consistency** (во всех узлах данные согласованы), **Availability** (любой запрос завершается корректно, но без гарантии одинаковости ответов узлов), **Partition tolerance** (потеря связи между узлами не портит корректность).



P обязателен в распределённой системе (сеть ненадёжна) — значит, выбираем между **C** и **A** (традиционные СУБД на одной машине не устойчивы к разделению, зато дают C + A). Согласованность — остановить узлы до договорённости (нетривиально при отказах, ср. задача византийских генералов) — бьёт по доступности. Но и голая доступность плоха (полбанка думает, что у клиента 100 руб., полбанка — 1000 руб.).

ACID (централизованные)

Atomicity — транзакция не применится частично

Consistency — завершённая транзакция не рушит целостность

Isolation — параллельные транзакции не мешают

Durability — данные завершённой транзакции не теряются

BASE (распределённые)

Basically Available — отказ узла → некорректный ответ, но только его клиентам

Soft-state — состояние меняется само, согласованность не гарантируется

Eventually consistent — целостность гарантируется лишь *когда-то*

ACID удобен, его пытались перенести в распределёнку — CAP не дал. Поэтому современные распределённые приложения берут более слабую **BASE**: каждый сервис держит целостными свои данные и асинхронно координируется. **Eventual consistency** — часть BASE.

Приём: за пределами CAP [+]

CAP описывает только поведение *при разделении*. Точнее — **PACELC** (Абади): *если Partition, выбирай A или C; иначе (Else) выбирай между Latency и Consistency*. Сама согласованность — спектр (strong / causal / eventual). **NewSQL** (Google Spanner, CockroachDB) и системы с *настраиваемой* согласованностью (Cassandra, DynamoDB) позволяют выбирать точку компромисса под задачу.

11. Принципы дизайна: проектирование для обслуживания

Распределённые приложения трудно контролировать («три секунды грузится страница — ок или нет?»). Поэтому их сразу проектируют так, чтобы ими можно было *управлять и обслуживать* в проде — **Design for operations**, основа DevOps. Главный принцип — **делать всё наблюдаемым**:

- **Логирование** всего в каждом сервисе + *централизованный* сбор и анализ (подсистема логов сама — распределённое приложение). Пример — **Elastic Stack**: Logstash (сбор) + Elasticsearch (индексация) + Kibana (визуализация); в сервисах — log4j / slf4j, log4net.
- **Трассировка** — путь *одного запроса* через много сервисов; нужна **корреляция** (сквозной идентификатор). Лог — *горизонтальный* срез (один сервис, много запросов); трасса — *вертикальный* (один запрос, много сервисов). Локализует отказы, узкие места, задержки.
- **Метрики и мониторинг** — по логам, трассам и счётчикам строят мониторинг численных метрик в реальном времени: запросов/с, ошибок/с, uptime, доля HTTP 5xx. Метрики специфичны для приложения — выбор правильных — искусство, но мониторинг нужен *какой-то* обязательно.
- **Стандартный формат** логов и метрик для всех сервисов (в микросервисах сложно — разные технологии; потому ценны архитекторы).

- **Автоматизировать всё** (развёртывание, мониторинг) — в идеале всё делается *одной командой* (ручные 4-часовые процедуры ⇒ ошибки).
- **Конфигурация как код** — хранить, версионировать, применять в CI / CD. Оркестраторы (Docker Compose, Kubernetes) описывают в YAML / JSON, какие сервисы, в скольких экземплярах, в каком порядке; кладут в систему контроля версий и применяют одной командой — `docker compose up` или `kubectl apply`. Руками лезть в конфигурацию работающей системы — плохо.

Приём: наблюдаемость сегодня [+]

Стандарт наблюдаемости — **OpenTelemetry** (CNCF): единые SDK и протокол для *трёх столпов* — логов, метрик, трасс. Типовой open-source стек: **Prometheus** (метрики) + **Grafana** (дашборды) + **Loki** (логи) + **Tempo** / **Jaeger** (трассы) — во многом сменил Elastic Stack. Цели надёжности формализуют как **SLI** / **SLO** / **SLA** с *error budget* (практика SRE). «Конфигурация как код» выросла в **GitOps** (ArgoCD, Flux) и **IaC** (Terraform).

Итоговый чек-лист

- ☐ **Big Compute** — массовая параллельность (клиент / очередь / координатор / узлы; embarrassingly parallel, мощный CPU, слабая сеть). **Big Data** — данных много (Kafka-поток или хранилище; schema-on-read, Data Lake, Hadoop / Spark). [+] Spark / Flink, Data Lakehouse (Iceberg).
- ☐ **Web-queue-worker** — веб / очередь / обработчик (буфер + независимое масштабирование). **Н-звенная** — веб / бизнес / данные на разных машинах за балансировщиками; масштаб крупными кусками. [+] serverless, контейнеры / PaaS.
- ☐ **Микросервисы** — мелкие сервисы = ограниченные контексты; своя БД, независимый деплой / масштаб / отказ; сложность → оркестрация (K8s), eventual consistency. [+] маятник к модульному монолиту, service mesh, BFF.
- ☐ **REST-API** — ресурсы (не действия), отношения в URL (без перебора), модель не данных, семантика HTTP, фильтрация / пагинация, Partial Content, HATEOAS (редок), версии. [+] OpenAPI, GraphQL.
- ☐ **Самовосстановление** — повтор + backoff, самодиагностика, изоляция ресурсов, буфер-очередь, failover (резерв в др. регионе), checkpoints, graceful degradation, намеренные отказы (Chaos Monkey). [+] chaos-инструменты, K8s-пробы, jitter, Polly.
- ☐ **Circuit Breaker** — Closed (счётчик ошибок > порог → Open) / Open (таймер → Half-Open) / Half-Open (успех → Closed, ошибка → Open). [+] Polly / resilience4j, service mesh.
- ☐ **Избыточность** — RTO / RPO / MTO; балансировщики, репликация (RAID 1), регионы, шардирование (RAID 0). [+] multi-AZ / region, NewSQL.
- ☐ **Минимизация координации** — доменные события, асинхронность, идемпотентность, шардирование, *избегать* распределённых транзакций → компенсационные + eventual consistency. [+] Saga (хореография / оркестрация), 2PC, Outbox.
- ☐ **CQRS** — CQS (команда меняет / не возвращает, запрос возвращает / не меняет) → разные хранилища чтения и записи; Event Sourcing (хранить события, не состояние; снапшоты). [+] Axon, EventStoreDB, проекции.
- ☐ **CAP** — из C / A / P не более двух; P обязателен ⇒ выбор C или A. **ACID** (централизованные) vs **BASE** (Basically Available, Soft-state, Eventually consistent). [+] PACELC, NewSQL.
- ☐ **Проектирование для обслуживания** — наблюдаемость: логи (Elastic Stack) + трассы (корреляция) + метрики / мониторинг; стандартный формат; автоматизировать всё; конфигурация как код (K8s YAML). [+] OpenTelemetry (3 столпа), Prometheus / Grafana, SLO / error budget, GitOps.

Лекция 14. Развёртывание распределённых приложений

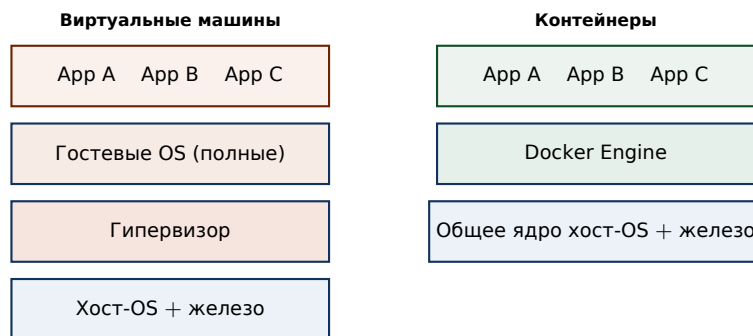
Рабочая тетрадь

Лекция — практика: как и куда развёртывать приложение. Контейнеризация (Docker), оркестрация (Docker Compose / Swarm, Kubernetes) и облачная инфраструктура (модели IaaS / PaaS / SaaS, AWS, Infrastructure as Code / Terraform). Все три вопроса билета раскрыты лекцией. Комментарии в листингах — по-английски. Метка [+] помечает то, чего в лекции нет (актуальные версии и инструменты на 2026 год).

1. Docker, Docker Compose

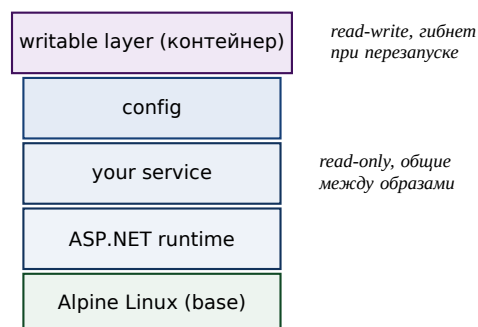
Определение: Docker

Docker — легковесная «виртуальная машина», виртуализирующая не машинные команды, а *системные вызовы* (использует ядро ОС хоста). Хуже изоляция, чем у обычных VM, но *гораздо быстрее*. Контейнер — виртуализированный *процесс* (внутри один процесс), а не виртуализированная машина. Образы *read-only*; стандарт де-факто для деплоя.



Контейнеры разделяют ядро хоста — нет гостевых ОС, поэтому легче и быстрее VM.

Образ (Image) — запускаемый образ приложения со всем окружением (как навороченный .exe). Состоит из *слоёв* read-only (напр. Alpine → рантайм ASP.NET → ваш сервис → конфигурация); файловая система — наложение слоёв. Слои *разделяются* между образами (как динамические библиотеки) ⇒ экономия места и трафика (скачивание послойное). Любой образ — база для другого. **Контейнер (Container)** — запущенный экземпляр образа (как запущенный .exe); имеет *писчий* слой для временных данных (гибнет при перезапуске). **Docker Hub** — центральный репозиторий образов (как GitHub; альтернативы — GitHub Registry, AWS ECR, Azure ACR, свой).



CLI: `docker run` (-d фон, -p host:container проброс порта, -it интерактивно; если образа нет — pull с Docker Hub); `docker ps` (запущенные контейнеры); `docker stop` (SIGTERM → SIGKILL); `docker exec` (доп. процесс в контейнере; для ручного ковыряния добавляют **BusyBox** — ~2 МБ, все базовые утилиты Unix).

Dockerfile и двухфазная сборка

Dockerfile — конфиг сборки образа (как Makefile). Инструкции: FROM (базовый образ + тег), WORKDIR, COPY/ADD, RUN (выполнить при сборке внутри контейнера), EXPOSE (открыть порт), ENV (переменная окружения), CMD (через шелл) / ENTRYPOINT (сразу процесс).

```
# stage 1: build in the SDK image (has the compiler + sources)
FROM mcr.microsoft.com/dotnet/sdk:6.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish "App.csproj" -c Release -o /app/publish
# stage 2: copy only the artifacts into the slim runtime image
FROM mcr.microsoft.com/dotnet/aspnet:6.0 AS final
WORKDIR /app
COPY --from=build /app/publish . # no SDK, no sources -- tiny image
ENTRYPOINT ["dotnet", "App.dll"]
```

Двухфазная (multi-stage) сборка: FROM сбрасывает состояние, AS запоминает слой. Собираем в образе с SDK, копируем *только результат* в тонкий рантайм-образ — никаких исходников и SDK. Рекомендуется для компилируемых языков: инструментарий фиксирован, образ собирается, имея лишь исходники и Docker (даже SDK ставить не надо).

Docker Compose и Swarm

Docker Compose — оркестратор из коробки; описывает многоконтейнерное приложение в файле `docker-compose.yml`: **services** (образ, deploy: реплики, лимиты, restart_policy), **ports**, **networks** (виртуальная сеть; реплики одного образа — за балансировщиком).

```
services:
  web:
    image: username/repo:tag
    deploy:
      replicas: 5 # five instances behind a load balancer
      resources: { limits: { cpus: "0.1", memory: 50M } }
      restart_policy: { condition: on-failure }
    ports: ["80:80"]
    networks: [webnet]
networks:
  webnet:
```

Docker Swarm — кластер из машин (тоже из коробки): `docker swarm init` на *менеджере* → токен → `docker swarm join` на рабочих узлах. На каждой машине — балансировщик; запрос к порту одной машины может уйти на любую. Compose + Swarm — вполне настоящая распределённая система, достаточная для многих корпоративных ИС.

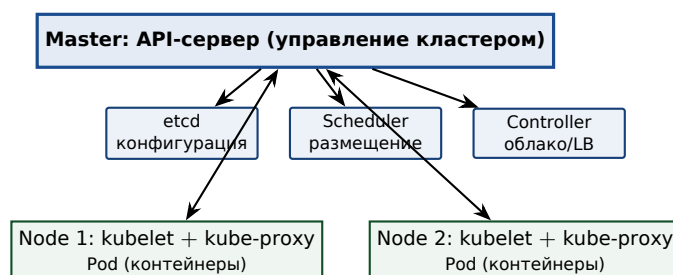
Приём: контейнеры сегодня [+]

Экосистема стандартизована вокруг **OCI** (image-spec + runtime-spec, эталон — `runc`). Docker — уже не единственный: **Podman** (без демона, rootless), **containerd**, **Buildah/kaniko** (сборка образов без демона). **Docker Desktop** с 2021 г. платен для компаний от 250 сотрудников (катализатор перехода на Podman / Rancher Desktop). **Compose v2** — это `docker compose` (плагин на Go), поле `version`: устарело. **Swarm** в больших системах почти вытеснен Kubernetes.

2. Kubernetes

Определение: Kubernetes

Kubernetes — продвинутый оркестратор для больших высоконагруженных систем. Задачи те же, что у Compose + Swarm (раскидать контейнеры по машинам, управлять связями, балансировать, следить за временем жизни, запускать / останавливать реплики), но в разы функциональнее и сложнее в конфигурировании — стандарт де-факто для больших приложений. Вырос из Google Borg, open source, написан на Go.



Master (реплицирован): **etcd** (БД конфигурации), **Controller Manager** (связь с облачным провайдером, балансировщики), **Scheduler** (раскидывает контейнеры). **Node**: **kubelet** (запускает контейнеры, следит за статусом), **kube-proxy** (сеть, маршрутизация), **cAdvisor** (метрики), **Pod** (набор Docker-контейнеров, работающих вместе — единица развертывания).

Объекты (хранятся в etcd, применяются YAML-манифестами): **Deployment** (деплой одного Pod-а; управляет контейнерами, репликацией, выкаткой; создаёт ReplicaSet), **ReplicaSet** (реплицированные группы Pod-ов), **Pod** (конфигурация контейнеров — образ, порты, переменные, монтирование). **Service** (прокси + балансировщик с фиксированным IP, перенаправляет реплике), **Ingress** (умная маршрутизация + TLS).

```
apiVersion: apps/v1
kind: Deployment      # declares the desired state
metadata: { name: demo, labels: { app: demo } }
spec:
  replicas: 1
  selector: { matchLabels: { app: demo } }
  template:
    spec:
      containers:
        - name: demo
          image: cloudnated/demo:hello # never use the :latest tag!
          ports: [{ containerPort: 8888 }]
---
apiVersion: v1
kind: Service          # proxy + load balancer, fixed IP
spec:
  ports: [{ port: 9999, targetPort: 8888 }]
  selector: { app: demo } # routes to every Pod labelled app=demo
```

`kubectl apply -f <file>` обновляет желаемую конфигурацию в etcd, считает разницу с текущей и выполняет переход. **Метки (labels)** — текстовые строки для выборки ресурсов.

Пробы здоровья (их выполняет Kubelet):

- **livenessProbe** — жив ли контейнер; неудача ⇒ *перезапуск*;
- **readinessProbe** — готов ли принимать запросы; неудача ⇒ *не слать* ему запросы из Service (дать приготовиться).

```
livenessProbe:
  httpGet: { path: /healthz, port: 8888 } # 200 => alive
  initialDelaySeconds: 3 # wait for startup
  periodSeconds: 3 # then check every 3s
```

Стратегии обновления (плавное переключение версий):

Стратегия	Суть
Blue/Green	поднять новую версию рядом (green), проверить, переключить трафик (Service / Ingress по меткам); откат — вернуть селектор
Rainbow	обобщение Blue/Green: несколько версий сразу (для долгих задач / соединений), старые гасят по мере завершения работы
Canary	завернуть малый % трафика на новую версию, постепенно до 100 % (Istio); годится и для A/B-тестов

Никогда не используйте тег latest — фиксируйте версию (latest незаметно меняется, чужой latest отдаёт систему на милость сторонним разработчикам).

Инструменты: **Helm** («пакетный менеджер» — чарты = шаблоны манифестов + конфиг), **Dashboard** (веб-UI; *защищать!* — расскажет все секреты), **Prometheus** (мониторинг), **Clair** (статический сканер уязвимостей контейнеров), **Velero** (бэкап etcd). **Метрики:** **RED** (Requests / Errors / Duration — внешнее поведение) и **USE** (Utilization / Saturation / Errors — внутреннее состояние).

Осторожно: конфигурация развёртывания — это код

`kubectl` умеет писать прямо в etcd, но *руками* менять реплики / порты / ресурсы — путь к катастрофе: перезапустите кластер — и система не работает (вручную навносили и забыли). Только декларативно: правите манифест → коммит в систему контроля версий → применяете (или это делает CI). Всегда актуальный конфиг + история, по которой видно, кто сломал.

Приём: Kubernetes сегодня [+]

В K8s 1.24 (2022) убрали **dockershim** — Docker как *рантайм* больше не используется (образы, собранные Docker, работают по стандарту OCI); рантайм по умолчанию — **containerd** (~95 % кластеров) или **CRI-O** (OpenShift), под капотом `runsc`. Кластеры обычно берут managed (EKS / GKE / AKS). Манифесты применяют через **GitOps** (ArgoCD, Flux); шаблонизация — Helm и **Kustomize**; безопасность — **Trivy** (сканер вместо Clair), **Falco** (рантайм), Pod Security Standards; наблюдаемость — OpenTelemetry + Grafana.

3. Облачная инфраструктура, AWS, Terraform

Содержать свои вычислительные ресурсы под большое приложение обычно *накладнее* самого приложения; ноут «в углу» не защищён от отключения электричества и порчи диска. Лучше арендовать облако. Услуги делятся на три категории:

Модель	Что даёт провайдер / что делает вы
IaaS (Infrastructure)	ВМ, диск, трафик — дальше сами (ставите Docker, настраиваете K8s-кластер)
PaaS (Platform)	готовые сервисы (K8s-кластер — конфигурируете и запускаете контейнеры, не зная ОС); дешевле и быстрее, <i>если знаете, что делаете</i>
SaaS (Software)	готовое приложение (Google Docs, облачные IDE)

Модели свободно смешивают. **Провайдеры: AWS** (лидер рынка), **Azure** (интеграция с инструментами Microsoft), **Google Cloud** (GKE — старейший и зрелый K8s; Colab — бесплатные GPU студентам), **Yandex.Cloud** (персональные данные по закону РФ хранить в России), **Heroku** (дружелюбна к новичкам, свой оркестратор).

Экосистема AWS (примеры из сотен сервисов):

Категория	Сервисы
Вычисления	EC2 (аренда ВМ), ECS (запуск Docker-контейнеров + ECR — свой Docker Hub)
Сеть	VPC (виртуальная сеть), ELB (балансировщик), API Gateway
Хранение	EFS (распределённая ФС), EBS (облачный диск)
БД / SaaS	RDS (готовые СУБД), DynamoDB (NoSQL), OpenSearch (поиск по логам)

Определение: Infrastructure as Code (IaC)

Манифесты Kubernetes описывают развёртывание *на готовой* инфраструктуре; **IaC** идёт дальше — описывает кодом *саму инфраструктуру* («нужен K8s-кластер из 5 узлов, балансировщик и сервер NoSQL»). Модель — независимо от провайдера, коммитится в систему контроля версий, применяется к выбранному провайдеру. Даёт *воспроизводимое* развёртывание и переход на другого провайдера (в идеале — одной командой).

Terraform — инструмент IaC (~135 провайдеров), консольный: логинится в провайдера и применяет желаемую конфигурацию (на языке HCL). Специфика провайдера всё же нужна (ресурсы называются по-разному), но править один файл проще, чем кликать в GUI.

```
# Terraform (HCL): a managed Kubernetes cluster on Azure
resource "azurerm_kubernetes_cluster" "default" {
  name           = "demo-aks"
  dns_prefix     = "demo-k8s"
  default_node_pool {
    name         = "default"
    node_count   = 2           # desired infrastructure, version-controlled
    vm_size      = "Standard_D2_v2"
  }
  role_based_access_control { enabled = true }
}
```

Приём: облако и IaC сегодня [+]

Доли рынка сместились: AWS уже не «почти 50%», а около **30%** (Q2 2025), Azure ~20%, Google Cloud ~13% — «большая тройка» держит около двух третей, отрыв AWS сокращается. **Terraform** в 2023 г. сменил лицензию (MPL 2.0 → BSL) — из версии 1.6 форкнули **OpenTofu** (Linux Foundation, MPL 2.0, drop-in). Альтернатива — **Pulumi** (IaC на обычных языках: Python, Go, TypeScript, C#) и AWS CDK. **Heroku** закрыл бесплатный тариф (2022). Поверх — serverless (Lambda) и **FinOps** (контроль облачных расходов).

Итоговый чек-лист

- ☐ **Docker** — виртуализирует системные вызовы (ядро хоста), контейнер = один процесс. **Образ** — read-only слой (общие между образами), **контейнер** — + писчий слой. CLI: run (-d/-p/-it), ps, stop, exec. **Dockerfile** — двух-фазная сборка (SDK → тонкий рантайм). **Compose** — services/networks; **Swarm** — кластер. [+] OCI, Podman, Compose v2.
- ☐ **Kubernetes** — оркестратор больших систем (из Borg, на Go). **Master**: etcd, scheduler, controller. **Node**: kubelet, kube-proxy, Pod. Объекты: Deployment → ReplicaSet → Pod, Service, Ingress; kubectl apply (желаемое состояние), метки. Пробы liveness / readiness; стратегии Blue/Green / rainbow / canary; не latest; Helm / Prometheus / Clair. [+] containerd (dockershim убран), GitOps, managed K8s.
- ☐ **Облако** — **IaaS** (ВМ) / **PaaS** (готовые сервисы) / **SaaS** (готовое приложение). Провайдеры: AWS, Azure, GCP, Yandex, Heroku. **AWS**: EC2 / ECS, VPC / ELB, EFS / EBS, RDS / DynamoDB. **IaC** — кодом описывают *саму инфраструктуру*; **Terraform** (HCL). [+] AWS ~30%, OpenTofu / Pulumi, Heroku без free-tier.

Лекция 15. Основы сетевой безопасности

Рабочая тетрадь

Лекция — основы криптографии и сетевой безопасности для прикладного программиста: базовые понятия, симметричное и асимметричное шифрование, Диффи-Хеллман, цифровые подписи, сертификаты, аутентификация, OAuth 2. Изложение — в основном по Таненбауму. Метка [+] помечает то, чего в лекции нет (актуальные стандарты на 2026 год).

1. Основные понятия. Шифрование с симметричным ключом

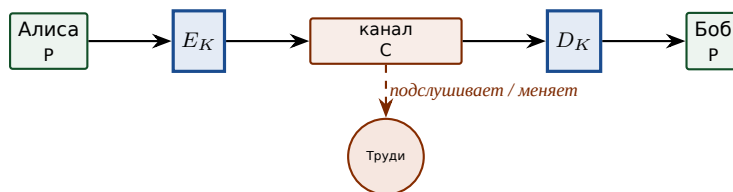
Определение: четыре свойства безопасности

Аутентификация — установление *идентичности* участника (тот ли это пользователь; часто взаимна — и клиент не уверен, что сервер не подделан). **Авторизация** — установление *прав* на операцию (уже после аутентификации; легитимный пользователь Gmail не вправе читать чужую почту). **Шифрование** — конфиденциальность. Также важны **целостность** (злоумышленник ничего не изменил) и **актуальность** (не проиграл старое сообщение — replay).

Принципы. Взлом «в лоб» безнадёжен — ломают *социально* (подкинутая флешка, давление; большинство атак *изнутри* — от обиженных сотрудников). Это игра против живого умного противника, владеющего инициативой; цель — не сделать взлом *невозможным*, а сделать его *дороже выгоды* (нет ценных данных — нечего защищать). Безопасность конфликтует с удобством (протухающий каждые два часа токен Blackboard раздражает). **Главное правило — не придумывать свои шифры и протоколы:** за ними большая наука, самоделка взламывается за часы.

Определение: модель шифрования

Открытый текст **P** (Plaintext) шифруется алгоритмом **E** с ключом **K**: $C = E_K(P)$, где **C** (Ciphertext) — шифртекст; дешифрование — **D** (тем же или другим ключом). **Принцип Керкгоффа:** *сам алгоритм известен*, секретен только ключ (хороший алгоритм разрабатывают годами; взломали ключ — сменили ключ, взломали алгоритм — переделывать всё). Криптостойкость повышают *длиной ключа*, а не «кручу-верчу» (последовательность из 20 шифров может стойкость и *понизить*).



Симметричные шифры

Определение: симметричный шифр

Один ключ для шифрования и дешифрования. *Очень быстрые* $\Rightarrow$ ими шифруется *весь* интернет-трафик (страницы, файлы, видео — десятки ГБ в реальном времени; есть аппаратная поддержка в CPU). Прикладной программист использует их как *чёрный ящик* (библиотеки OpenSSL, Bouncy Castle, System.Security.Cryptography) — *своё не писать*.

Шифр	Год / ключ	Замечание
DES	1975, 56 бит	к 1999 ломался за часы (устройство «на базаре»)
3DES	1995	тройной DES: $E_{K3}(D_{K2}(E_{K1}(P)))$ ($K1=K2$ — совместимость с DES); приемлем, но «от правительства»
AES	2001, 128/192/256	он же Rijndael; открытый конкурс, два бельгийца — считается без backdoor; <i>основной сейчас</i>

Задача шифра — *перемешать* биты так, чтобы каждый бит выхода зависел от каждого бита входа и ключа (перестановки + итерации + хог с ключом). Доказывать статистические свойства результата — ад; потому *свой шифр писать нельзя*.

Режимы шифрования

Идеальный шифр прост: ключ = случайная строка длины сообщения, побитовый хог (**one-time pad**: $a \oplus b \oplus b = a$, неотличим от шума). Но передать такой ключ не проще, чем сообщение $\Rightarrow$ ключи ограниченной длины применяют по-разному:

Режим	Идея и свойства
ECB	блоки шифруются независимо $\Rightarrow$ подстановочный шифр над большим алфавитом; уязвим к <i>статистической атаке</i> и к <i>перестановке</i> блоков (клерк меняет местами две записи зарплат, не дешифруя). <i>Не применяется</i> .
CBC	блок хог-ится с <i>предыдущим шифрблоком</i> перед шифрованием (первый — с IV , вектор инициализации, не секретен). Стоек; минус: порча бита рушит всё после него (плюс: видно вмешательство).
SCM	поточковый: генерируем keystream , шифруя IV ключом снова и снова, и хог-им с текстом. Быстро, параллелится, порча бита = один бит. <i>Нельзя повторять keystream</i> : $(P \oplus K) \oplus (Q \oplus K) = P \oplus Q$ (Keystream Reuse Attack).
CTR	шифруем IV, IV+1, IV+2, ... и хог-им. <i>Произвольный доступ</i> (зашифрованные ФС — читать блок без расшифровки терабайтов).

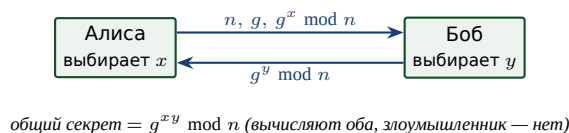
Приём: симметричные шифры сегодня [+]

3DES запрещён NIST для большинства применений после 2023 г.; **DES** давно мёртв. Стандарт — **AES**, причём в **AEAD**-режимах (шифрование + аутентичность сразу): **AES-GCM** (счётчик + тег целостности) и **ChaCha20-Poly1305** (на устройствах без аппаратного AES). Голый ECB/CBC в новых протоколах не используют.

2. Алгоритм Диффи-Хеллмана. Шифрование с открытым ключом

Определение: алгоритм Диффи-Хеллмана

Позволяет договориться об *общем секретном ключе по открытому каналу*, «на глазах» у злоумышленника. Алиса и Боб загадывают случайные x и y , берут большие числа n, g . Алиса шлёт $n, g, g^x \bmod n$; Боб шлёт $g^y \bmod n$. Общий ключ — $g^{xy} \bmod n$ (оба легко вычисляют). Злоумышленник, зная n, g, g^x, g^y , не может за разумное время найти g^{xy} — это *задача дискретного логарифмирования* (полиномиального решения не известно).



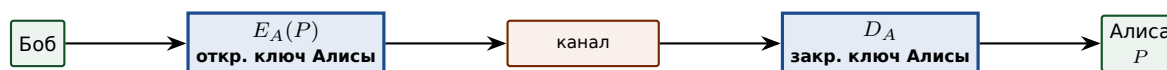
Где: приватные чаты Telegram (даже если провайдер дешифрует HTTPS, сообщения не прочесть); HTTPS иногда тоже использует ДН для выбора симметричного ключа.

Осторожно: атака посредника (Man-in-the-Middle)

ДН работает, *только если уже выполнена взаимная аутентификация*. Иначе злоумышленник представляется Алисе Бобом, а Бобу Алисой, загадывает своё z и выполняет ДН дважды — получает два общих ключа и читает / перешифровывает всё. Алиса и Боб ничего не замечают. Поэтому ДН сам по себе ключ выбрать позволяет, а *аутентичность* — нет.

Определение: шифрование с открытым ключом (асимметричное)

Разные ключи у отправителя и получателя. Алгоритм делится на D, E так, что $D(E(P)) = P$, причём ключ от D *невозможно* получить из ключа от E (нужно разложить огромное число на множители или взять дискретный логарифм). Ключ от E — **открытый** (публикуется), ключ от D — **закрытый** (секрет). Стороны *не обмениваются* секретами.



Боб шифрует *открытым* ключом Алисы E_A , только Алиса дешифрует своим *закрытым* D_A . Популярные схемы: **RSA** (разложение на множители), **ElGamal** (дискретные логарифмы), **эллиптические** шифры (алгебра точек на эллиптических кривых). Асимметричные шифры *медленные* — ими аутентифицируются и выбирают симметричный ключ, а трафик потом шифруют симметрично.

Приём: обмен ключами сегодня [+]

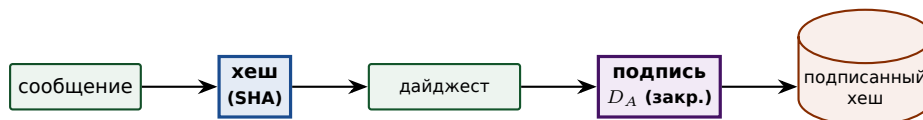
Классический ДН в кольце вычетов вытеснен **ECDH** на эллиптических кривых (**X25519**) — короче ключи, быстрее; в TLS 1.3 — *эфемерный* ECDHE (forward secrecy: вскрытие долгого ключа не раскрывает прошлые сессии). **Угроза — квантовый компьютер:** алгоритм Шора ломает RSA / ECC. NIST в 2024 г. стандартизовал **пост-квантовые** схемы: **ML-KEM** (FIPS 203, бывш. Kyber) для обмена ключами и ML-DSA для подписей; браузеры уже катят *гибридный* обмен X25519 + ML-KEM.

3. Цифровые подписи, сертификаты

Определение: цифровая подпись

Когда нужна не конфиденциальность, а гарантия *авторства и неизменности* (напр. библиотека в NuGet — чтобы вместо NUnit не подсунили вора банковских карт). Алиса шифрует сообщение своим *закрытым* ключом D_A (потом, если надо, открытым ключом Боба E_B). Боб применяет E_A — если получилось осмысленно, автор точно Алиса (только она знает D_A), и она *не отпрётся*.

Шифровать всё асимметрично дорого $\Rightarrow$ подписывают **дайджест** (Message Digest) — криптографический *хеш* сообщения: считается быстро и однозначно, малое изменение меняет хеш до неузнаваемости, по хешу *нельзя* восстановить сообщение (только перебор). Подписывают *хеш* (20 байт — быстро); адресату шлют открытый текст + подписанный хеш. Адресат сам считает хеш и сверяет.

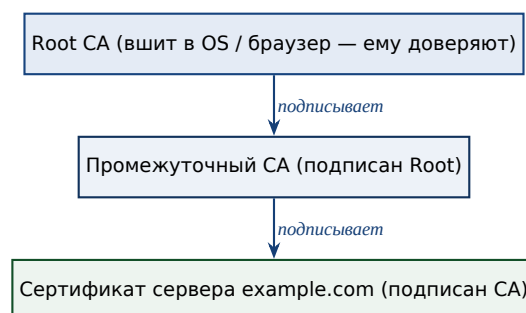


проверка: адресат считает хеш сам + применяет E_A (откр.) — сверяет

Хеш-функции: **MD5** (стар, уязвим — атака дней рождения), семейство **SHA** (SHA-1 тоже сломан; практически — SHA-2, SHA-3). Блочные (SHA-1: блок 512 бит, дайджест 160 бит). **HMAC** — хеш от данных + секретного ключа: проверяет и целостность, и аутентичность.

Определение: сертификат

Как узнать, что открытый ключ собеседника *настоящий*? Скачать со страницы — перехватят (MITM). **Сертификат** — сообщение «предъявитель владеет доменом example.com», подписанное **Certificate Authority (CA)**, которому доверяют обе стороны (формат **X.509**). CA верхнего уровня подписывает CA ниже, те — сертификаты пользователей: **цепочка доверия** до **корневого** сертификата (root), вшитого в ОС / браузер. Передают *всю цепочку* сразу (сетевых запросов для проверки не нужно).



Виды: от «владеет доменом» (бесплатно) до подтверждения личности / финансовой устойчивости (дорого, аудит). Всегда на *ограниченный срок* (обычно год), CA может *отозвать*. **Самоподписанный** сертификат — генерируешь сам (никто не доверяет, но можно добавить в доверенные — так делает Fiddler при дешифровке HTTPS). **Let's Encrypt** — бесплатное CA для подтверждения домена (браузеры доверяют) — неиспользование HTTPS «из-за дороговизны» больше не оправдано.

```
# OpenSSL: generate a self-signed X.509 certificate + RSA-2048 key
openssl req -x509 -nodes -days 365 \
  -newkey rsa:2048 -keyout privatekey.key \
  -out certificate.crt
# -nodes: do NOT encrypt the private key on disk
# -days 365: valid for one year -out: the public certificate
```

Где применяются: HTTPS (сервер доказывает владение доменом), подписывание кода (Windows SmartScreen, Apple Code Signing — ОС проверяет цепочку и что код не менялся), подписывание .NET-сборок («сильное имя»). Форматы (.pem, .p12, .pfx, .der, .cer, .crt) несовместимы, но конвертируются (OpenSSL, PuTTYgen).

Приём: подписи и сертификаты сегодня [+]

SHA-1 полностью сломан (атака SHATTERED, 2017) — только **SHA-256** / SHA-3; MD5 — никогда. Выдача сертификатов автоматизирована протоколом **ACME** (Let's Encrypt доминирует), а сроки их жизни резко *сокращают* (тренд CA/Browser Forum — к десяткам дней) $\Rightarrow$ продление только автоматическое; есть публичные логи **Certificate Transparency**. EV-сертификаты в UI браузеров фактически отменены. На горизонте — *постквантовые* подписи **ML-DSA** (FIPS 204), но они крупные.

4. Аутентификация. OAuth 2

Определение: Challenge-Response

Если у сторон есть *общий секретный ключ*, личность доказывают, не разглашая ключ: Боб шлёт Алисе **challenge** — случайное число **nonce** (number used once, новое каждый раз); Алиса возвращает его *зашифрованным* общим ключом; Боб сверяет. Затем симметрично — Алиса проверяет Боба.

Осторожно: зеркальная атака

«Оптимизированный» протокол (Алиса сразу шлёт и идентичность, и свой challenge — 3 сообщения вместо 5) *ломается*: злоумышленник представляется Алисой, получает challenge Боба, открывает *вторую* сессию и просит Боба зашифровать *его же* challenge, ответ переправляет в первую сессию. Боб «сам себе» доказал, что собеседник — Алиса. Внешне корректный протокол уязвим к неочевидной атаке — поэтому *свои протоколы аутентификации писать нельзя*. (Лечится **HMAC** в три сообщения, запретом параллельных сессий или поппе из разных диапазонов.)

Определение: Basic Authentication

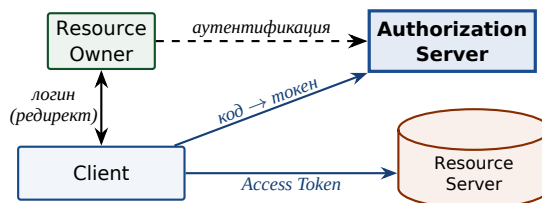
На практике проще: транспорт защищён **HTTPS** (сервер уже доказал идентичность), клиент шлёт логин / пароль *открытым текстом* — их шифрует HTTPS. Подводные камни:

- **взлом клиента**: логин / пароль затирают сразу; сервер выдаёт **Access Token** (случайная строка с *ограниченным* временем жизни) — дальше предъявляют только его;
- **взлом сервера**: пароли *не хранят* — хранят их криптографический **хеш**. Против «радужных таблиц» (пред-посчитанные хеши) добавляют **соль** (salt) — случайное число к паролю перед хешированием (хранится рядом, открыто).

Аутентификация с открытым ключом: секретов не хранить вовсе — сервер хранит *открытые* ключи клиентов (даже если украдут, они не секрет). Так работает SSH на GitHub, Yandex.Cloud.

Определение: OAuth 2 (авторизация)

Позволяет дать доступ к ресурсу, *не раскрывая* логин / пароль владельцу ресурса (вход на сторонний сайт по аккаунту Google / VK). Четыре роли: **Client** (приложение), **Resource Owner** (пользователь), **Authorization Server** (меняет разрешение на токен), **Resource Server** (хранит ресурс).



Режим Authorization Code (основной): Client *перенаправляет* владельца на Authorization Server, где тот логинится (логин / пароль видит только AS, даже Client — нет). AS возвращает Client-у **Access Token** (с **Scope** — набор прав: чтение / запись; протухает быстро) и **Refresh Token** (живёт дольше, можно продлять и отозвать — по нему получают новый Access Token, не логинясь заново). Всё — по HTTPS. Пример — Google OAuth 2.0 (регистрируешь приложение, получаешь Client ID / Secret, Consent Screen).

Приём: аутентификация и авторизация сегодня [+]

Хранение паролей: простой «соль + SHA» недостаточен — нужны *медленные, memory-hard* функции **Argon2id** (рекомендация OWASP), bcrypt или scrypt (против перебора на GPU). **OAuth 2.1** консолидировал практики: **PKCE обязателен** для всех клиентов (против перехвата кода), implicit-grant удалён, ротация refresh-токенов. Поверх OAuth для аутентификации — **OpenID Connect** (OAuth отвечает «что можно приложению», OIDC — «кто пользователь», ID-токен в JWT). Пароли вытесняют **passkeys** (WebAuthn / FIDO2) — закрытый ключ на устройстве, привязан к домену, устойчив к фишингу.

Итоговый чек-лист

- **Понятия**: аутентификация (личность) $\neq$ авторизация (права); + шифрование, целостность, актуальность. Цель — взлом дороже выгоды; ломают социально и изнутри; свои шифры / протоколы не писать.
- **Симметричные** ($C = E_K(P)$, один ключ, быстрые, весь трафик): DES $\rightarrow$ 3DES $\rightarrow$ **AES**. **Режимы**: ECB (плох — статистика, перестановка), CBC (хот с предыдущим, IV), SCM (keystream, не повторять), CTR (произвольный доступ). [+]
AES-GCM / ChaCha20 (AEAD), 3DES запрещён.
- **Диффи-Хеллман** — общий ключ по открытому каналу ($g^{xy} \bmod n$; дискретный логарифм); не от MITM — нужна аутентификация. **Асимметричные** ($D(E(P)) = P$, открытый E / закрытый D): RSA, ElGamal, эллиптические; медленные. [+]
ECDH / X25519, forward secrecy, пост-квантовый ML-KEM.

- **Подпись** — хеш (дайджест), подписанный закрытым ключом (авторство + неизменность + неотрекаемость); MD5 / SHA, HMAC. **Сертификат** — открытый ключ + владелец, подписан CA (X.509); *цепочка доверия* до root; срок ограничен; самоподписанные, Let's Encrypt. [+] SHA-1 сломан → SHA-256, ACME, короткие сроки, ML-DSA.
- **Аутентификация:** Challenge-Response (nonce; зеркальная атака!), **Basic Auth** (HTTPS + токен; пароли — хеш + **соль** против радужных таблиц), аутентификация открытым ключом (SSH). **OAuth 2** (авторизация без раскрытия пароля; Client / Owner / Authorization & Resource Server; Authorization Code, Access + Refresh Token, Scope). [+] Argon2id, OAuth 2.1 / PKCE, OIDC, passkeys.